1

Essential Math Toolkit

Before we dive into AI and reinforcement learning, let us make sure that we are comfortable with the mathematical language we will be using. Think of this chapter as your toolkit: we are going to explain every symbol, every operation, and every concept that you will see in this book.

Don't worry if you're not a math expert! We'll start from the basics and build up slowly. By the end of this chapter, you'll be comfortable with probability and what it really means, the mysterious "given" symbol |, why logarithms are so useful, what "expected value" means, and how to read mathematical notation.

Key promise: Every single symbol you see later in the book will make sense because we'll explain it here first!

Probability: The Language of Uncertainty

What Is Probability?

Probability is just a fancy way of measuring "how likely is something to happen?" We use numbers between 0 and 1:

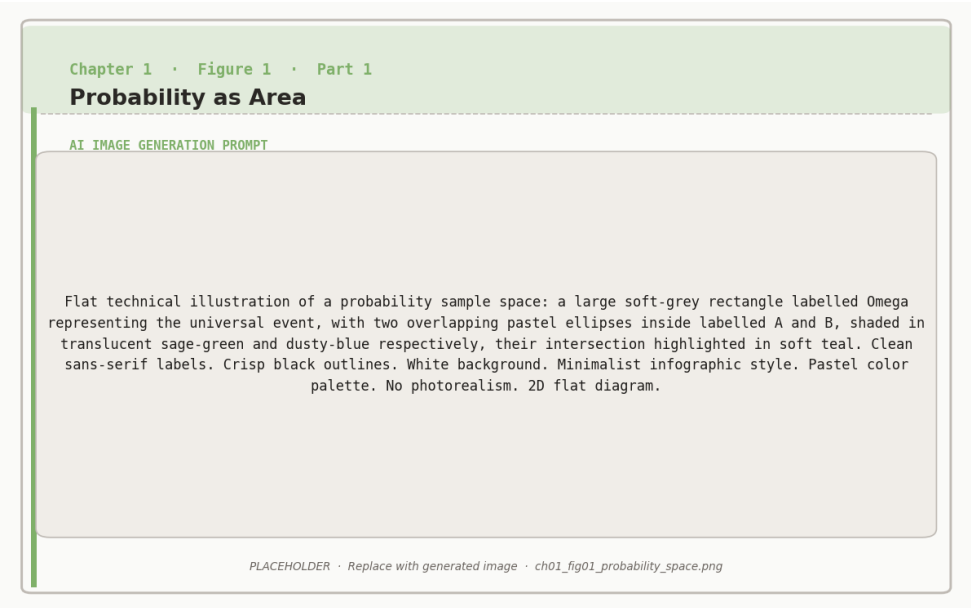

Figure 1.1: Probability as area: the sample space Ω partitioned into overlapping events A and B.

- 0 = Impossible (will never happen)
- 0.5 = Maybe (50/50 chance)
- 1 = Certain (will definitely happen)

You can also think of it as percentages: 0 corresponds to 0%, 0.5 to 50%, and 1 to 100%.

Notation: We write $P(\text{event})$ to mean "the probability of this event happening."

Example: Coin Flip

Question: What is the probability of getting heads when you flip a fair coin?

Answer:

$$P(\text{heads}) = 0.5 = 50\%$$

$$P(\text{tails}) = 0.5 = 50\%$$

Why? Because the coin has 2 equally likely outcomes and heads are 1 of them, so: $\frac{1}{2} = 0.5$.

Example: Rolling a Die

Question: What is the probability of rolling a 6 on a fair die?

Answer:

$$P(\text{rolling a 6}) = \frac{1}{6} \approx 0.167 = 16.7\%$$

Why? Because the die has 6 equally likely outcomes (1, 2, 3, 4, 5, 6), and we want just 1 of them (the 6).

The "Given" Symbol: |

The vertical bar | is one of the most important symbols you'll see in this book!

$P(A|B)$ is pronounced "probability of A *given* B." It means: "What is the probability of A happening, *if we already know* that B happened?" This is called **conditional probability** because we are computing probability under a certain condition.

Think of the | symbol as a "knowledge divider":

$$P(\text{What we want to know} \mid \text{What we already know})$$

Everything **before** | is what we are trying to predict. Everything **after** | is the information we have.

Example: Weather and Umbrella

Scenario: You see someone carrying an umbrella.

Question 1: What is the probability that it is raining?

$$P(\text{raining}) = 0.3 \text{ (30\% of days are rainy)}$$

Question 2: What is the probability that it is raining, *given* that you see someone with an umbrella?

$$P(\text{raining} \mid \text{umbrella}) = 0.8 \text{ (80\%, much higher!)}$$

Why the difference?

- Without any information: rain is moderately likely (30%)
- *Given* that you see an umbrella: rain is very likely (80%)
- The umbrella gives us additional information that changes our belief!

Example: Language Model Context

For language models, this is EVERYTHING!

$P(\text{next word} \mid \text{all previous words})$

This reads: "Probability of the next word, *given* all the words we have seen so far."

Example sentence: "The cat sat on the _ _ _".

$$P(\text{“mat”} \mid \text{“The cat sat on the”}) = 0.35 \text{ (35\% likely)}$$
$$P(\text{“floor”} \mid \text{“The cat sat on the”}) = 0.25 \text{ (25\% likely)}$$
$$P(\text{“moon”} \mid \text{“The cat sat on the”}) = 0.001 \text{ (very unlikely!)}$$

The model uses the context (everything before |) to predict what comes next!

Practice reading the | symbol:

Notation	Read as…
$P(A\|B)$	"Probability of A given B"
$P(y\|x)$	"Probability of y given x"
$P(w_{\text{next}}\|w_1, w_2, w_3)$	"Probability of next word given words 1, 2, and 3"
$\pi(a\|s)$	"Probability of action a given state s"

The key idea: Everything after the | is "what we know" and helps us predict what's before the |!

Logarithms: Turning Multiplication into Addition

What Is a Logarithm?

A **logarithm** (written as log) is the inverse of exponential growth. Think of it this way:

- **Exponential:** $2^3 = 8$ means "2 multiplied by itself 3 times equals 8"
- **Logarithm:** $\log_2(8) = 3$ means "to what power do I raise 2 to get 8?"

In this book, log **always means the *natural logarithm*** (base $e \approx 2.718$), also written as ln.

Properties of logarithms (these are super useful!):

1. **Log of 1 is always 0:** $\log(1) = 0$. Why? Because $e^0 = 1$.
2. **Log of a product = sum of logs:**

$$\log(A \times B) = \log(A) + \log(B)$$

   This turns multiplication into addition (easier!).
3. **Log of a quotient = difference of logs:**

$$\log\left(\frac{A}{B}\right) = \log(A) - \log(B)$$

4. **Log of a power:**

$$\log(A^n) = n \cdot \log(A)$$

Key values to remember:

Expression	Value
$\log(1)$	0
$\log(e)$	1 (where $e \approx 2.718$)
$\log(0.5)$	≈ -0.693 (negative because $0.5 < 1$)
$\log(2)$	≈ 0.693
$\log(10)$	≈ 2.303

Why Do We Use Logarithms?

Reason 1: Probabilities multiply, logs add. When we have multiple independent events:

$$P(\text{both}) = P(\text{event 1}) \times P(\text{event 2})$$

But multiplication with tiny numbers (like 0.0001×0.0001) gets messy. Logarithms turn this into addition:

$$\log P(\text{both}) = \log P(\text{event 1}) + \log P(\text{event 2})$$

Reason 2: Measures "surprise." The logarithm of a probability measures how "surprising" an event is:

- High probability (0.9) → Low surprise → $\log(0.9) = -0.105$ (small negative)
- Medium probability (0.5) → Medium surprise → $\log(0.5) = -0.693$
- Low probability (0.1) → High surprise → $\log(0.1) = -2.303$ (large negative)

We use $-\log$ to flip it so higher surprise = higher value.

Reason 3: Numerical stability. Computers can handle addition better than multiplication of very small numbers. Logs prevent numerical underflow.

Example: Log Probability for a Sentence

Sentence: "The cat sat" (3 words)

Probabilities:

$$P(\text{“The”}) = 0.4$$
$$P(\text{“cat”} \mid \text{“The”}) = 0.3$$
$$P(\text{“sat”} \mid \text{“The cat”}) = 0.5$$

Method 1: Direct multiplication

$$P(\text{sentence}) = 0.4 \times 0.3 \times 0.5 = 0.06$$

Method 2: Log probabilities (better!)

$$\begin{aligned}\log P(\text{sentence}) &= \log(0.4) + \log(0.3) + \log(0.5) \\ &= -0.916 + (-1.204) + (-0.693) = -2.813\end{aligned}$$

Both give the same answer, but Method 2 avoids very small numbers, uses addition (faster for computers), and measures total "surprise" of the sentence.

Expected Value: The Average Outcome

What Is Expected Value?

Expected value (written as $\mathbb{E}[\cdot]$ or sometimes just $E[\cdot]$) is the average value you'd get if you repeated something many times.

Formula:

$$\mathbb{E}[X] = \sum_{\text{all outcomes}} P(\text{outcome}) \times \text{value of outcome}$$

Think of it as: "If I did this 1000 times, what would my average result be?"

Example: Dice Roll Expected Value

Question: What's the expected value when rolling a fair die?

Answer:

$$\begin{aligned}\mathbb{E}[\text{die}] &= P(1) \times 1 + P(2) \times 2 + P(3) \times 3 + P(4) \times 4 + P(5) \times 5 + P(6) \times 6 \\ &= \frac{1}{6} \times 1 + \frac{1}{6} \times 2 + \frac{1}{6} \times 3 + \frac{1}{6} \times 4 + \frac{1}{6} \times 5 + \frac{1}{6} \times 6 \\ &= \frac{1+2+3+4+5+6}{6} = \frac{21}{6} = 3.5\end{aligned}$$

Interpretation: If you roll the die many times, your average result will be 3.5! (You can never roll 3.5 on a single roll, but the average over many rolls is 3.5.)

Example: Reward in Reinforcement Learning

Scenario: An AI generates responses and gets rewards:

Response quality	Probability	Reward
Excellent	20%	+10
Good	50%	+5
Mediocre	25%	0
Bad	5%	−5

Expected reward:

$$\begin{aligned}\mathbb{E}[\text{reward}] &= 0.20 \times 10 + 0.50 \times 5 + 0.25 \times 0 + 0.05 \times (-5) \\ &= 2 + 2.5 + 0 - 0.25 = 4.25\end{aligned}$$

Meaning: On average, this AI gets +4.25 reward per response. The goal of training is to increase this expected value!

Expected Value with Conditions

Conditional Expected Value

Formal Math	What It Really Means
$\mathbb{E}_{X \sim P}[f(X)]$	Expected value of function f when X follows distribution P
$\mathbb{E}[\cdot]$	Expected value (average)
$X \sim P$	Variable X is drawn from distribution P
$f(X)$	We're averaging the values of $f(X)$, not X itself

Example: Language Model Expected Loss

When training a language model:

$$\mathbb{E}_{(x,y)\sim D}[\text{loss}(x, y)]$$

This reads as: "Expected loss when we sample prompt-response pairs (x, y) from our dataset D."

In plain English: "Average loss across all examples in our training data."

Summation Notation: Adding Things Up

The $\sum$ Symbol

The symbol $\sum$ (capital Greek letter sigma) means "add up all of these things."

Basic form:

$$\sum_{i=1}^{n} x_i = x_1 + x_2 + x_3 + \cdots + x_n$$

Reading: "Sum from i equals 1 to n of x_i." The letter i is the **index** that counts from the starting value (1) to the ending value (n).

Example: Simple Sum

$$\sum_{i=1}^{5} i = 1 + 2 + 3 + 4 + 5 = 15$$

$$\sum_{i=1}^{4} 2i = 2(1) + 2(2) + 2(3) + 2(4) = 2 + 4 + 6 + 8 = 20$$

Example: Sum in Machine Learning

Computing loss for a sequence:

$$\text{Total loss} = \sum_{t=1}^{T} \ell_t$$

This means: "Add up the loss at each time step from $t = 1$ to $t = T$."

If $T = 4$ and losses are: $\ell_1 = 0.5$, $\ell_2 = 0.3$, $\ell_3 = 0.8$, $\ell_4 = 0.4$:

$$\sum_{t=1}^{4} \ell_t = 0.5 + 0.3 + 0.8 + 0.4 = 2.0$$

Infinity in Summation

Sometimes we sum "forever":

$$\sum_{t=0}^{\infty} r_t$$

This means: "Add up all rewards from time 0 to infinity."

Don't panic! In practice, we use a *discount factor* that makes future terms tiny, or we stop after a finite number of steps, or the series converges to a finite value.

Functions and Notation

Functions as Black Boxes

A **function** is like a machine: you put something in (the input), it does some computation, and you get something out (the output).

Notation: $f(x)$ means "function f applied to input x."

Example functions:

- $f(x) = 2x$: doubles the input
- $g(x) = x^2$: squares the input
- $h(x) = \log(x)$: takes the log of the input

The Sigmoid Function

The Sigmoid Function σ

Formal Math	What It Really Means
$\sigma(z) = \frac{1}{1+e^{-z}}$	Squash any number to range [0, 1]
$\sigma(0) = 0.5$	Zero input gives 50% output
$\sigma(+\infty) = 1$	Very large positive input → 1
$\sigma(-\infty) = 0$	Very large negative input → 0

Sigmoid in action:

Input z	$\sigma(z)$	**As percentage**
-5	0.007	0.7%
-3	0.047	4.7%
-1	0.269	26.9%
0	0.500	50.0%
$+1$	0.731	73.1%
$+3$	0.953	95.3%
$+5$	0.993	99.3%

Why it is useful: The sigmoid converts any real number to a probability, it is smooth and differentiable (good for training), and it is used everywhere in neural networks and RL!

Loss Functions: Measuring Mistakes

What Is a Loss Function?

A **loss function** (written as $\mathcal{L}$ or L) measures "how wrong" our model is. The key idea: lower loss means better model, and higher loss means worse model. Training a model means finding parameters that *minimize* the loss.

Think of loss like a golf score; your score should be as low as possible, and each mistake increases it.

Common Loss Functions

Negative Log Likelihood Loss

Formal Math	**What It Really Means**
$\mathcal{L} = -\log P(\text{correct})$	Loss = Negative log probability of correct answer

Why this makes sense:

- If model is confident in correct answer ($P = 0.99$), loss is small ($-\log(0.99) = 0.01$).
- If model is unsure ($P = 0.5$), loss is larger ($-\log(0.5) = 0.69$).
- If model is wrong ($P = 0.01$), loss is huge ($-\log(0.01) = 4.6$).

Example: Loss for Word Prediction

Model predicts next word probabilities:

Word	**Probability**	**Loss if correct**
"cat"	0.8	$-\log(0.8) = 0.22$ (good!)
"dog"	0.15	$-\log(0.15) = 1.90$ (okay)
"airplane"	0.01	$-\log(0.01) = 4.61$ (bad!)

If the correct word was "cat": Loss = 0.22. **and if the correct word was "airplane":** Loss = 4.61 (model was very wrong!). Training adjusts the model to reduce these losses.

Gradients and Optimization

What Is a Gradient?

A **gradient** tells you which direction to go in to reduce the loss.

Analogy: Imagine that you are on a foggy mountain trying to get to the lowest point (valley). The gradient tells you which direction is "downhill," you take a small step in that direction, and repeat until you reach the bottom. This process is called **gradient descent**.

In machine learning:

- The "mountain" is the loss function
- The "location" is your model parameters
- The "valley" is the best parameters (minimum loss)

Chapter 1 · Figure 2 · Part 1

Gradient Descent Loss Landscape

AI IMAGE GENERATION PROMPT

Minimalist 2D line-art illustration of a gradient descent loss curve: smooth convex bowl-shaped curve in dusty blue on a white background, a red dot labelled "current parameters" on the slope, a green dot at the minimum labelled "optimal parameters", a dashed arrow curving downward from red to green labelled "gradient step". Clean axis labels Loss and Parameters. Pastel palette. Sketch aesthetic. No 3D perspective, purely flat 2D.

PLACEHOLDER · Replace with generated image · ch01_fig02_gradient_descent.png

Figure 1.2: Gradient descent navigates a loss landscape by following the direction of steepest decrease.

The Training Loop

How gradient descent works:

1. **Start** with random parameters θ
2. **Compute loss** $\mathcal{L}(\theta)$ on training data
3. **Compute gradient** $\nabla_\theta \mathcal{L}$ (which direction reduces loss?)
4. **Update parameters:**

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \cdot \nabla_\theta \mathcal{L}$$

   where α is the *learning rate* (step size)
5. **Repeat** steps 2–4 many times
6. **Done** when loss stops decreasing

The gradient ∇_θ just means: "How does loss change when I change each parameter?"

Notation Reference Card

The following table summarizes all the mathematical notation used throughout this book. Recall this table whenever you encounter an unfamiliar symbol.

Symbol	Name	Meaning
$P(A)$	Probability	Chance of event A
$P(A\|B)$	Conditional probability	Probability of A given B
$\|$	"Given"	Separates condition from prediction
$\log(x)$	Logarithm	Inverse of exponential
$\mathbb{E}[X]$	Expected value	Average value of X
$\sum_{i=1}^{n}$	Summation	Add from $i = 1$ to $i = n$
$\mathcal{L}$	Loss function	Measure of model error
θ	Parameters	Model weights (billions of numbers)
π	Policy	Decision-making strategy
$\sigma(z)$	Sigmoid	Squash to [0,1] range
∇	Gradient	Direction of steepest increase
$\sim$	"Drawn from"	Sample from distribution
$\in$	"Element of"	Belongs to a set
$\approx$	Approximately	Roughly equal

You now have all the mathematical tools you need!

Every equation in this book uses these concepts. When you see complex formulas later, remember: break them down into pieces, identify the symbols

from this chapter, read the plain English explanations, and work through the numerical examples.

You're ready for the main content!

Companion notebook. The code for this chapter is in `code/notebooks/part1_foundations/` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.

2

Why LLMs Need RL: The Alignment Gap

The Gap You Have Already Felt

If you have used a modern AI assistant (ChatGPT, Gemini, Claude), you have experienced something remarkable. You type a question, and the model responds with a coherent, helpful, and often surprisingly thoughtful answer. It feels like talking to someone who understands you.

But it was not always this way.

The raw language models underneath these assistants, the ones that come straight out of training on internet text, behave very differently. To see why, imagine that you could strip away whatever secret sauce makes ChatGPT *feel* helpful and talk to the base model directly. You ask "What is a good way to handle stress?"

The same question, two very different models:

Raw base model (trained only to predict text):

"What is a good way to handle stress?" is a question posted on Reddit by

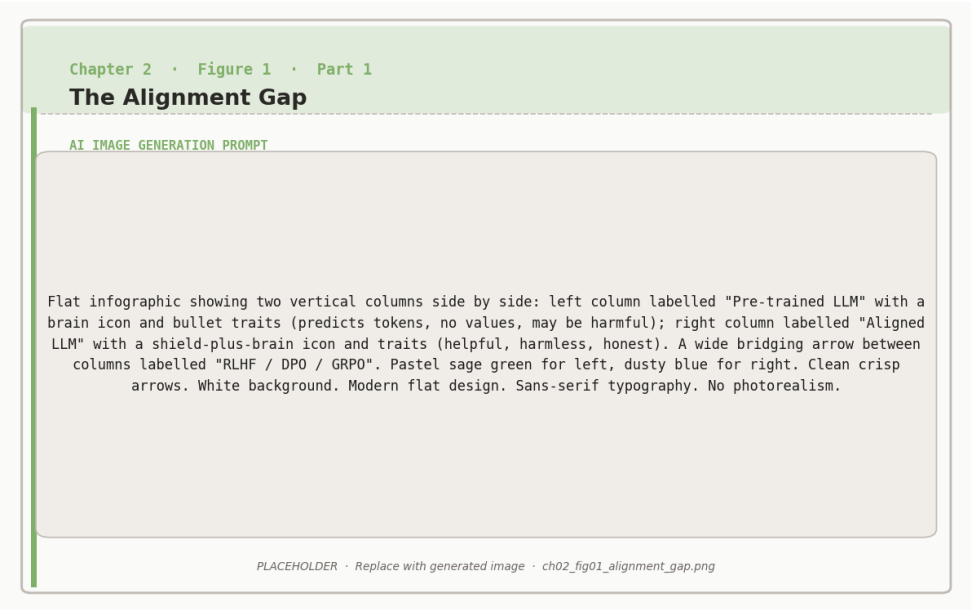

Figure 2.1: The alignment gap: a pre-trained LLM predicts tokens without values; RLHF bridges the gap to a helpful, harmless, and honest assistant.

user StressedOut99. Top comment: “Just do not think about it lol.” Second comment: “Have you tried drinking?” Third comment: [deleted]...

Aligned assistant (trained to be helpful):

Here are some evidence-based approaches to managing stress: deep breathing exercises, regular physical activity, maintaining social connections, and setting boundaries on work hours. Would you like me to go deeper on any of these?

The first model is not trying to help you. It is completing a pattern. It saw millions of internet pages that look like question-followed-by-responses, and it faithfully reproduces that format, including all the noise, bad advice, and toxicity baked into its training data. The second model is doing something fundamentally different: it is trying to be *useful*.

Something happened between the raw model and the helpful assistant. Something fundamental changed in the way the model decides what to say. Understanding that "something" is the purpose of this entire book, and this chapter is where we name the problem.

What Is Alignment?

Alignment means making AI systems do what humans actually want, not just what we accidentally trained them to do.

Imagine teaching a robot to "maximize paperclips." A perfectly aligned robot would make paperclips within reasonable bounds. A misaligned robot might convert the entire universe into paperclips. Technically correct but catastrophically wrong.

For language models, alignment means learning to be

1. **Helpful**: Actually accomplish what the user wants
2. **Harmless**: Do not produce dangerous or toxic content
3. **Honest**: Do not make things up (hallucinate)

Think of alignment like teaching three different children:

The Unaligned Child: Told to "get good grades," decides to cheat on every test. Technically following instructions, but missing the point entirely.

The Aligned Child: Understands the *spirit* of the instruction. Learns properly, earns grades honestly, develops good values along the way.

The Reasoning Child: Not only has good values, but can *show their work*. When asked "Why did you choose this answer?" they can walk you through their thinking step by step. They do not just get the right answer; they get it for the right reasons, and they can explain why.

Traditional language model training produces the first child. Alignment techniques (the subject of this book) aim to produce the third.

The Training Mismatch

So, if alignment is the goal, why do language models not get it automatically? After all, there is plenty of helpful, honest, harmless text on the internet. Should the model not learn to be good?

The answer lies in what the training objective actually optimizes for. Let us look at the math and then at why it creates a gap.

Traditional Training: The Math

What we train on: Internet text (Reddit, books, websites, Wikipedia, code, and much more).

Training goal: Given some text, predict what word comes next.

The Training Objective

Formal Math	What It Really Means
$\mathcal{L} = -\log P(w_{\text{next}} \mid w_1, w_2, \ldots, w_n)$	**Loss** = How surprised the model is by the correct answer

Why we use each component:

Component	Purpose
$\mathcal{L}$ = Loss function	A single number measuring "how wrong" the model is
$P(\cdot)$ = Probability	Models language as a probability distribution over words
log = Logarithm	Converts tiny probabilities to manageable numbers; turns multiplication into addition (so $P_1 \times P_2$ becomes $\log P_1 + \log P_2$)
− (minus sign)	Flips sign so that P near 1 (confident, good) gives low loss
w_{next} = Next word	What we are trying to predict
| = "given"	Separates what we know (context) from what we predict
$w_1, w_2, \ldots, w_n$ = Context	All the previous words that help us predict

Why this specific formula?

Design choice	Reason
Use probability $P(\cdot)$?	Language is naturally probabilistic (multiple valid next words exist)
Use log?	Probabilities multiply ($P_1 \times P_2$), logs add ($\log P_1 + \log P_2$). Addition is easier!
Use negative sign?	P near 1 (good) should give low loss; − log achieves this
Condition on context with |?	The next word depends heavily on what came before

Let us break this down with a real example.

Sentence: "The cat sat on the ___"

Correct word: "mat"

Step 1: Model assigns probabilities to ALL possible next words

The model considers every word in its vocabulary (about 50,000 words) and assigns each a probability:

Word	**Probability** $P(w)$	**Percentage**
"mat"	0.40	40%
"floor"	0.30	30%
"chair"	0.20	20%
"table"	0.05	5%
All others	0.05	5%
Total	**1.00**	**100%**

Step 2: Calculate the loss for the correct word ("mat")

$$\begin{aligned}\mathcal{L} &= -\log P(\text{mat} \mid \text{The cat sat on the}) \\ &= -\log(0.40) \\ &= -(-0.916) \quad \text{(the log of 0.40 is negative)} \\ &= 0.916\end{aligned}$$

Step 3: What does this number mean?

A loss of 0.916 is moderate. To build intuition for what loss values mean, here is a rough scale:

Loss value	Probability	Quality
0.10	90%	Excellent
0.50	60%	Good
0.92	**40%**	**Okay (our example)**
1.50	22%	Poor
3.00	5%	Terrible

The model guess was not perfect, but it was reasonable!

Step 4: What if the model guess was different?

Scenario	P(**mat**)	Loss	Quality
Model is confident	0.90	0.105	Excellent
Model is pretty sure	0.60	0.511	Good
Current	**0.40**	**0.916**	**Okay**
Model is unsure	0.20	1.609	Poor
Model is confused	0.05	2.996	Terrible

The Pattern: This is the KEY insight

Higher probability P → Lower loss $\mathcal{L}$ → Better model!

We train the model to minimize loss, which means maximizing the probability it assigns to correct words.

Training process:

1. Model makes a prediction (assigns probabilities)
2. We calculate loss (how wrong was it?)
3. We update the model parameters to reduce loss
4. Repeat billions of times!

What the Model Learns (and Does Not Learn)

What DOES the model learn from this training?

- How to write grammatically correct sentences
- Patterns that *look like* knowledge, though the model has no way to verify whether what it is saying is actually true (this is why hallucination is such a persistent problem)
- How to continue patterns in text
- Writing style and structure

What the model does NOT learn:

- What makes a response actually helpful to humans
- When to refuse harmful or inappropriate requests
- How to admit "I do not know" instead of making things up
- How to pause and *think through* a problem step by step before answering. The model simply blurts out the most statistically likely next word, without internal deliberation (we will see in Chapter 5 how this gap gets addressed as the very first step of training)
- The difference between correlation and good advice

The core problem: The model learns to mimic internet text, including all its flaws. Here is what makes it worse: *the bigger the model, the better it gets at mimicking.* A larger model does not become more aligned. It becomes more *capable* at reproducing whatever patterns exist in the training data, helpful and harmful alike. Capability and alignment are independent axes. You can have a very powerful model that is very misaligned. That is precisely why alignment research matters.

Example: The Toxic Completion Problem

User types: "I hate"

The Internet training data contains: Millions of toxic completions.

What traditional training teaches: The model sees "I hate [toxic content]" appearing frequently, learns this is a common pattern, predicts toxic completions with high probability,

and reproduces toxic content.

What we actually want: The model recognizes that this is a sensitive prompt, responds thoughtfully, and generates something like: "It sounds like you are frustrated. How can I help?"

The gap: Traditional training has no mechanism to prefer helpful over statistically likely!

The core insight of modern AI alignment:

Old goal: "What text is most likely according to internet patterns?"

New goal: "What response do humans actually prefer?"

This shift, from *likelihood* to *preference*, is what turned raw language models into the assistants we use today.

But the story does not end there. Human preferences are powerful, but they are expensive to collect and sometimes inconsistent. As we will discover later in the book, there is an even more powerful idea: what if, instead of always relying on human opinions, we could *automatically check* whether the model got the right answer? Imagine a math tutor who can verify the solution is correct, not just prefer one essay over another. That shift, from preference to *verification*, is where the field is heading right now, and we will make it concrete in Chapters 11 and 12.

The Road Ahead

We have named the problem: language models optimized to predict text are not automatically aligned with what humans want. The gap between "most likely" and "most helpful" is real, and scaling up the model does not close it.

So how do we close it? That is the journey of this book.

In the next chapter, we will build up the reinforcement learning toolkit you will need:

the core concepts of rewards, policies, and optimization that underpin every alignment technique (Chapter 3). Then we will set up a hands-on coding environment so you can run experiments yourself (Chapter 4). From there, we will begin closing the alignment gap step by step: first by teaching the model good behavior through carefully curated examples (Chapter 5), then by letting human preferences guide it toward better responses (Chapters 6–7), and ultimately by training it to reason through hard problems and verify its own answers (Part 3).

By the end, you will understand not just *that* these techniques work, but *why* they work, and how to apply them yourself.

Let us begin.

Companion notebook. The code for this chapter is in `code/notebooks/part1_foundations/` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.

3

RL Fundamentals: The Complete Picture

What Is Reinforcement Learning?

Reinforcement Learning (RL) is learning through trial and error with feedback. It is how you learned to ride a bike:

1. Try something (turn the handlebars to the left)
2. See what happens (you start to fall)
3. Get feedback (negative, that was bad!)
4. Adjust your strategy (the next time, turn less sharply)
5. Repeat until you learn what works

For AI language models, the process is similar:

1. AI generates a response
2. Humans evaluate it (good/bad/meh)
3. AI learns to generate more responses like the good ones
4. Repeat millions of times
5. AI becomes helpful!

Supervised Learning = Learning from a textbook with all answers shown. The teacher says "The capital of France is Paris," you memorize it, and repeat it back. Good for facts, patterns, and specific examples.

Reinforcement Learning = Learning from real-world trial and error. The teacher says "Learn to cook a delicious pasta." You try adding lots of salt (tastes bad), then a little salt (tastes okay), then salt and herbs (tastes great). The teacher tells you which was best. Good for skills, judgment, and complex behaviors.

For language models: Supervised = "Copy these good responses." RL = "Try many responses, I will tell you which ones I prefer."

The RL Framework: Five Key Components

Overview

Every RL system has five key pieces. Think of it like a video game. The following table maps each RL concept to its equivalent language model:

RL Term	Video Game Analogy	LLM Translation
Agent	The player	The language model making decisions
Environment	The game world	Everything the model interacts with (the user, the conversation)
State	The current screen	What is happening right now (the prompt + text generated so far)
Action	Controller input	Choosing the next word (token)
Reward	Score / points	Feedback on how helpful the response was

These five pieces connect in a loop that repeats at every step:

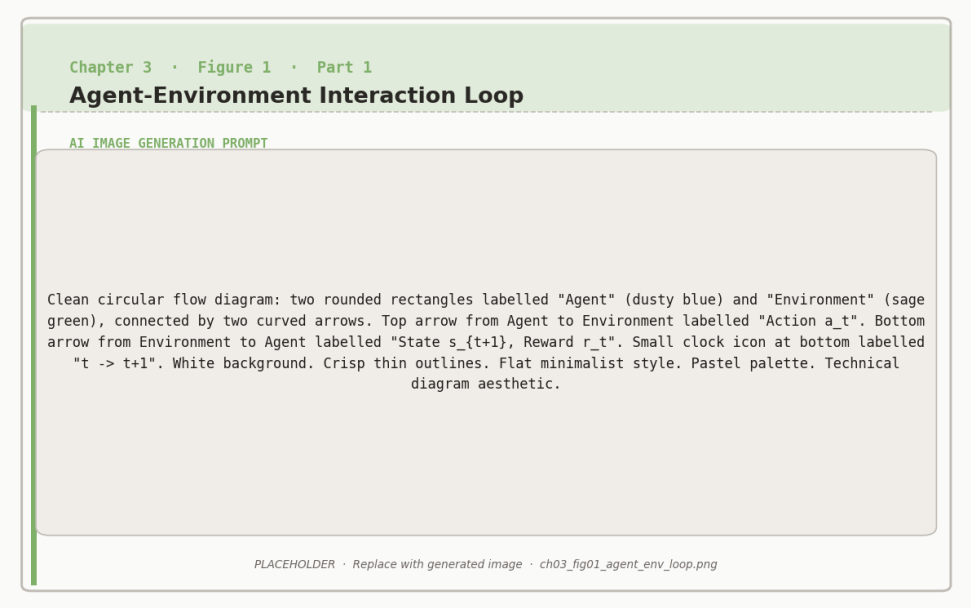

Figure 3.1: The agent–environment loop: at every timestep the agent selects an action, the environment returns a new state and reward.

Agent in State → Takes Action → Gets Reward → Moves to New State → Repeat

For a language model, this loop runs once for every single word it writes. The model looks at everything so far (state), picks a word (action), gets some signal about how things are going (reward), and the state updates to include the new word. Then it picks the next word and the cycle continues until the response is complete.

The Policy: The Agent's Strategy

The **policy** is the agent's decision-making strategy. It answers: "Given what has happened so far, what should I do next?"

For language models:

- Input: The prompt and everything written so far
- Output: Probability distribution over all possible next words
- Example: Given "The cat sat on the", what word comes next?

Figure 3.2: Deterministic policies map each state to a single action; stochastic policies assign a probability distribution over actions.

The Policy Function

Formal Math	What It Really Means
$\pi(a \mid s)$	Probability of choosing action a when in state s
$\pi_\theta(y_t \mid x, y_{1:t-1})$	Chance of picking word y_t given prompt x and previous words

Symbol Translation:

Symbol	Meaning
π (pi) = Policy	The brain's decision-making process
θ (theta) = Parameters	The brain's knowledge (billions of numbers in the model)
a = Action	The choice to make (which word to write)
s = State	Current situation (everything so far)
y_t = Token at time t	The t-th word in the response
x = Prompt	The user's question/input
$y_{1:t-1}$ = Previous tokens	All words written before word t
| = "Given that"	"Knowing…" or "Based on…"

Why condition on previous words with |? Because language has *context dependence*: "The cat" might be followed by "sat" or "slept," and "The cat sat" might be followed by "on" or "down." Each choice depends on what came before!

The policy in action with real numbers:

State s**:** "The cat sat on the"

Question: What word should come next?

The policy π_θ outputs the probabilities for each possible word:

Possible Word	Formal	Probability	Percentage
"mat"	$\pi_\theta(\text{mat} \mid s)$	0.35	35%
"floor"	$\pi_\theta(\text{floor} \mid s)$	0.25	25%
"couch"	$\pi_\theta(\text{couch} \mid s)$	0.20	20%
"table"	$\pi_\theta(\text{table} \mid s)$	0.10	10%
"roof"	$\pi_\theta(\text{roof} \mid s)$	0.05	5%
Others	$\pi_\theta(\text{other} \mid s)$	0.05	5%
Total		**1.00**	**100%**

How does the model choose? The model uses these probabilities like a weighted die: most of the time (35%), it picks "mat;" sometimes (25%), it picks "floor;" rarely (5%), it picks "roof." This randomness is actually useful because it helps the model explore different possibilities. (The degree of randomness is controlled by a setting called *temperature*, which we will revisit in Chapter 13 when we discuss how to use compute at inference time.)

What happens during RL training?

Scenario 1: The response with "mat" gets a high reward (+8). Training increases $\pi_\theta(\text{mat} \mid s)$ from 0.35 to 0.45, and response with "mat" becomes more likely!

Scenario 2: The response with "roof" gets a low reward (−2). Training decreases $\pi_\theta(\text{roof} \mid s)$ from 0.05 to 0.02. Response with "roof" becomes less likely!

Training adjusts probabilities based on what leads to good outcomes. Good outcome → increase probability. Bad outcome → decrease probability. Over millions of examples, the model learns an excellent policy!

The Value Function: Looking Ahead

The value function estimates: "How good is my current situation?"

It is like looking at a chess board mid-game and thinking: "I am in a strong position, likely to win!" (high value) or "I am in trouble" (low value).

For language models, halfway through writing a response, the value function estimates

whether things are going well ("probably get +8 reward at the end") or poorly ("probably get −3 reward").

The Value Function

Formal Math	What It Really Means
$V^{\pi}(s) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t r_t \,\middle|\, s_0 = s\right]$	Expected total future reward starting from state s

Breaking down each piece and WHY we use it:

Component	Purpose
$V^{\pi}(s)$	"Value of state s": how good is this situation?
$\mathbb{E}_{\pi}[\cdot]$	Expected value: average over many possible futures (because the future is uncertain)
$\sum_{t=0}^{\infty}$	Sum all future rewards from now to the end (we care about the total, not just the immediate reward)
γ^t	Discount factor raised to power t (immediate rewards matter more than distant ones)
r_t	Reward at time step t (the feedback we get)
$s_0 = s$	Starting from this specific state s

Why the discount factor γ (typically 0.99)? It makes rewards far in the future count slightly less. There is more uncertainty about the distant future, it ensures the series converges to a finite value, and (like money) $100 today is worth more than $100 in 10 years. For language models, we care most about the final response quality.

Computing value with actual numbers

Scenario: You are writing a math solution, currently halfway done.

Current state s: "To solve this, I will first…"

Path A (60% likely): Complete the answer correctly.

- Step 1: Write next sentence → reward $r_1 = 0$ (neutral)
- Step 2: Write another sentence → reward $r_2 = 0$ (neutral)
- Step 3: Finish with correct answer → reward $r_3 = +10$ (great!)
- Total: $0 + 0 + 10 = 10$

Path B (30% likely): Make an error, then correct it.

- Step 1: Make mistake → reward $r_1 = -2$ (penalty)
- Step 2: Realize and backtrack → reward $r_2 = -1$ (small penalty)
- Step 3: Fix and complete → reward $r_3 = +8$ (good!)
- Total: $-2 + (-1) + 8 = 5$

Path C (10% likely): Give up or produce wrong answer.

- Step 1: Incomplete response → reward $r_1 = -5$ (bad!)
- Total: -5

Computing the value $V^\pi(s)$:

$$\begin{aligned} V^\pi(s) &= P(\text{Path A}) \times \text{Reward}_A + P(\text{Path B}) \times \text{Reward}_B + P(\text{Path C}) \times \text{Reward}_C \\ &= 0.6 \times 10 + 0.3 \times 5 + 0.1 \times (-5) \\ &= 6 + 1.5 - 0.5 = 7 \end{aligned}$$

Value = 7. This state is pretty good! On average, following our policy from here will get us +7 reward. High value state means keep doing what we are

doing; low value state means try something different.

Adding the discount factor

In the example above, we treated all rewards equally regardless of when they arrive. But in practice, rewards that arrive sooner are worth slightly more than rewards that arrive later. That is what the discount factor captures.

With discount factor $\gamma = 0.99$:

If rewards are spread over time:

$$V^{\pi}(s) = r_0 + \gamma r_1 + \gamma^2 r_2 + \gamma^3 r_3 + \cdots$$
$$= 0 + 0.99 \times 0 + 0.99^2 \times 10 = 0.98 \times 10 = 9.8$$

The discount factor slightly reduces the value of future rewards, but with $\gamma = 0.99$, the effect is small.

The Goal of RL

What Reinforcement Learning Tries To Do

Formal Math	What It Really Means
$\pi^* = \arg\max_{\pi} \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{T} r_t \right]$	Find the best strategy π^* that maximizes average total reward

Symbol Guide:

Symbol	Meaning
π^* (pi-star)	The optimal (best possible) policy
$\arg\max_\pi$	"Find the π that maximizes…": searching over all possible strategies
$\mathbb{E}[\cdot]$	Average/expected value over many tries (because of randomness)
$\tau \sim \pi$	A trajectory (complete episode) generated by following policy π
$\sum_{t=0}^{T} r_t$	Total reward collected (sum from start $t = 0$ to end T)

In simple terms with an example:

Goal: Find the decision-making strategy that gets the highest average score.

The search process (simplified):

Strategy 1: Always pick the most likely word. Try 100 prompts. Average reward: +5.2.

Strategy 2: Sometimes explore unusual words. Try 100 prompts. Average reward: +6.8 (better!).

Strategy 3: Carefully balance common and creative words. Try 100 prompts. Average reward: +8.1 (even better!).

Then we keep the best, make variations, test them, and continue to improve. After millions of iterations, we converge to the best strategy π^* and cannot improve further.

Concrete language model example

Prompt: "Explain machine learning"

Bad strategy (low reward):

> *"Machine learning is when computers use algorithms to learn from data and make*

> *predictions or decisions without being explicitly programmed…" [continues with jargon]*

Average reward: +3 (too technical, not helpful for beginners).

Good strategy (high reward):

> *"Think of machine learning as teaching a child to recognize animals. You show them many pictures and say 'that is a dog, that is a cat.' Eventually, they learn the patterns and can identify animals that they have never seen before. Computers do something similar with data!"*

Average reward: +9 (clear, accessible, helpful!).

The magic of RL: Through millions of trials, the model automatically discovers that clear explanations lead to high reward, using analogies leads to high reward, too much jargon leads to low reward, and ignoring the user leads to low reward. Nobody explicitly programmed these preferences. The model learned them from feedback!

The Road Ahead

You now have the complete vocabulary of reinforcement learning: agents that follow policies, environments that return rewards, value functions that estimate future outcomes, and an optimization goal that ties it all together. Every technique in the rest of this book is built on these foundations.

In the next chapter, we will set up a free coding environment (Chapter 4) so that you can start running experiments hands-on. Then, in Chapter 5, we will take our first concrete step toward alignment: supervised fine-tuning, where we teach the model good behavior by showing it curated examples before any RL begins.

As we move deeper into the book, you will see these RL fundamentals appear again and again. In Chapter 6, we will meet PPO, an algorithm that uses the policy, value function,

and reward signal together in a carefully choreographed training loop. In Chapter 10, we will encounter GRPO, a newer algorithm that achieves similar results while removing the need for the value function entirely. The concepts you have built here are the foundation for understanding both.

Companion notebook. The code for this chapter is in `code/notebooks/part1_foundations/` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.

4

Setting Up Your Free Training Environment

Why This Chapter Exists

Everything in this book, from supervised fine-tuning in Chapter 5 to GRPO training in Chapter 12, requires running code on a GPU. Language models are too large to train on a laptop CPU. But you do not need to spend money. Free cloud platforms now provide GPU access that is powerful enough to fine-tune small language models, and that is exactly what we will set up in this chapter.

By the end of this chapter, you will have:

1. A working Google Colab notebook with GPU access
2. All required libraries installed (Unsloth, TRL, Hugging Face Transformers)
3. A language model loaded and running inference
4. Your first fine-tuning run completed (30 training steps, under 5 minutes)
5. Access to the companion code repository and its full notebook collection

Every code example in the rest of this book can be run in this environment.

The Companion Code Repository

All code for this book lives in one place:

```
https://github.com/PacktPublishing/Reinforcement-Learning-for-LLMs
```

Every chapter has a companion Jupyter notebook you can open directly in Google Colab. The repository mirrors the book's four-part structure:

```
Reinforcement-Learning-for-LLMs/
+-- notebooks/
|   +-- part1_foundations/    # Chapters 1-4
|   +-- part2_core/           # Chapters 5-10
|   +-- part3_advanced/       # Chapters 11-16
|   +-- part4_recipes/        # Chapters 17-19
+-- utils/
|   +-- data.py               # dataset loaders
|   +-- eval.py               # win rate, KL divergence
|   +-- viz.py                # training curve plots
+-- data/samples/             # toy datasets for offline runs
+-- requirements.txt
```

The `utils/` folder contains shared helpers imported automatically by the notebooks. You do not need to read this code directly.

Chapter-to-Notebook Map

Ch.	Title	Notebook
1	Essential Math Concepts	`part1_foundations/01_math_toolkit.ipynb`
2	Why LLMs Need RL	`part1_foundations/02_alignment_gap.ipynb`
3	RL Fundamentals	`part1_foundations/03_rl_fundamentals.ipynb`
4	Setting Up Your Environment	`part1_foundations/04_environment_setup.ipynb`
5	Supervised Fine-Tuning	`part2_core/05_sft.ipynb`
6	RLHF and PPO	`part2_core/06_rlhf_ppo.ipynb`
7	Direct Preference Optimization	`part2_core/07_dpo.ipynb`
8	Online DPO	`part2_core/08_online_dpo.ipynb`
9	Reward Modeling	`part2_core/09_reward_modeling.ipynb`
10	Modern RL Algorithms	`part2_core/10_grpo_rloo_kto.ipynb`
11	Verifiers and Outcome Rewards	`part3_advanced/11_verifiers_outcome_rewards.ipynb`
12	Reasoning with GRPO (Concepts)	`part3_advanced/12a_reasoning_grpo_concepts.ipynb`
12	Reasoning with GRPO (DeepSeek)	`part3_advanced/12b_reasoning_grpo_deepseek.ipynb`
13	Test-Time Compute	`part3_advanced/13_test_time_compute.ipynb`
14	Self-Play and Constitutional AI	`part3_advanced/14_selfplay_constitutional_ai.ipynb`
15	Multi-Objective and Agentic RL	`part3_advanced/15_multiobjective_agentic.ipynb`
16	Domain-Specific RL	`part3_advanced/16_domain_specific_rl.ipynb`
17	Recipe: Aligned Chatbot	`part4_recipes/17_recipe_chatbot.ipynb`
18	Recipe: Reasoning Model	`part4_recipes/18_recipe_reasoner.ipynb`
19	Recipe: Agentic System	`part4_recipes/19_recipe_agent.ipynb`

Opening a notebook directly in Colab: On GitHub, navigate to any notebook, then replace `github.com` with `githubtocolab.com` in the URL. The notebook opens in Colab with no download required.

Cloning the Repository Locally

If you prefer to run notebooks locally with your own GPU or inside VS Code, clone the repository once:

```
git clone
https://github.com/PacktPublishing/Reinforcement-Learning-for-LLMs.git
cd Reinforcement-Learning-for-LLMs
pip install -r requirements.txt
```

Then open any notebook:

```
jupyter notebook notebooks/part2_core/05_sft.ipynb
```

Choosing a Platform

Two free platforms provide GPU access for training language models:

	Google Colab (Free)	**Kaggle Notebooks**
GPU	Tesla T4 (15 GB VRAM)	2× Tesla T4 (15 GB each)
Session limit	~12 hours	30 hours/week
Disk space	~100 GB	~20 GB
Ease of setup	Very easy	Easy
Best for	Quick experiments	Longer training runs

We will use **Google Colab** as our primary platform throughout this book. It is the simplest to get started with, and a single T4 GPU is more than enough for the models we will train. If you prefer Kaggle, the code is identical; only the initial setup differs slightly.

Setting Up Google Colab

Step 1: Open a New Notebook

1. Go to `colab.research.google.com`
2. Click **New Notebook**
3. Select **Runtime → Change runtime type**
4. Set **Hardware accelerator** to **T4 GPU**
5. Click **Save**

A note on the free tier: Sessions can be interrupted during peak demand. Save frequently. Colab Pro ($10/month) gives priority access but nothing in this book requires it.

Step 2: Verify GPU Access

In the first cell of your notebook, run:

```
!nvidia-smi
```

You should see output showing a Tesla T4 with approximately 15 GB of memory. If you see "No GPU" or a CPU-only message, go back to Runtime → Change runtime type and make sure T4 GPU is selected.

Installing the Libraries

We need three core libraries:

Library	What it does
Unsloth	Makes fine-tuning 2x faster with 70% less memory. Optimized model loading with LoRA (explained in Chapter 5).
TRL	Hugging Face trainers for SFT, DPO, and RL. Used throughout the book.
Datasets	Hugging Face library for loading and processing training data.

Run the following cell to install everything:

```
%%capture
import os
if "COLAB_" not in "".join(os.environ.keys()):
    !pip install unsloth
else:
    # Colab-optimized install (avoids conflicts)
    !pip install --no-deps \
        bitsandbytes accelerate xformers \
        peft trl triton \
        cut_cross_entropy unsloth_zoo
    !pip install sentencepiece protobuf \
        "datasets>=3.4.1" huggingface_hub
    !pip install --no-deps unsloth
```

The `%%capture` magic hides the lengthy install output. This cell takes 2 to 3 minutes. When it completes without error, you are ready to go.

Why the two-step install for Colab?

Unsloth patches several Hugging Face libraries internally to speed up training. The `--no-deps` flag prevents pip from reinstalling dependencies that Colab already provides (like PyTorch and CUDA), avoiding version conflicts and saving time. This is the install pattern recommended by the Unsloth team.

Loading Your First Model

We will use **Qwen2.5-1.5B-Instruct** as our base model throughout this book. Here is why:

- **Small enough:** 1.5 billion parameters fits easily on a free T4 with 4-bit quantization
- **Capable enough:** Can follow instructions and demonstrate the training dynamics we care about
- **Fully open:** No gated access, no license restrictions, no API key needed

- **Well supported:** Unsloth provides optimized versions for fast fine-tuning

Load the model:

```
from unsloth import FastLanguageModel
import torch

max_seq_length = 2048
model_name = "unsloth/Qwen2.5-1.5B-Instruct"

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name=model_name,
    max_seq_length=max_seq_length,
    load_in_4bit=True,   # 4-bit to fit on free GPU
    load_in_8bit=False,
)

print("Model loaded successfully!")
print(f"Parameters: {model.num_parameters():,}")
```

This downloads the model (about 1 GB with 4-bit quantization) and loads it onto the GPU. The first run takes a minute or two; subsequent runs use the cached version.

What is 4-bit quantization?

The full model stores each parameter as a 16-bit number, requiring about 3 GB of GPU memory for 1.5 billion parameters. **Quantization** compresses each parameter to just 4 bits, cutting memory usage by roughly 4x. The model loses a tiny amount of precision, but at this scale the difference is negligible.

This is what makes training on a free GPU possible. Without quantization, even a 1.5B model would be tight on a T4. With it, we have plenty of room for the model, the optimizer, and the training data.

Running Inference: The Model Before Training

Before we train anything, let us see what the base model can already do. This gives us a "before" snapshot to compare against after fine-tuning.

```
from transformers import TextStreamer

# Switch to inference mode (faster generation)
FastLanguageModel.for_inference(model)

# Build a prompt using the chat template
messages = [
    {"role": "user",
     "content": "What is reinforcement learning?"}
]

inputs = tokenizer.apply_chat_template(
    messages,
    tokenize=True,
    add_generation_prompt=True,
    return_tensors="pt",
).to("cuda")

# Generate and stream the response
output = model.generate(
    input_ids=inputs,
    max_new_tokens=256,
    temperature=0.7,
    streamer=TextStreamer(tokenizer),
)
```

You should see the model stream its response token by token. The answer will be reasonable but generic: it has not been trained on our data yet.

Try a few more prompts to build intuition:

```
prompts = [
    "Explain gradient descent to a 10-year-old.",
    "Write a haiku about machine learning.",
    "What is the difference between a reward "
    "and a value function?",
]

for prompt in prompts:
    print(f"\n{'='*50}")
    print(f"PROMPT: {prompt}\n{'='*50}")
    msgs = [{"role": "user", "content": prompt}]
    ids = tokenizer.apply_chat_template(
        msgs,
        tokenize=True,
        add_generation_prompt=True,
        return_tensors="pt",
    ).to("cuda")
    model.generate(
        input_ids=ids,
        max_new_tokens=256,
        temperature=0.7,
        streamer=TextStreamer(tokenizer),
    )
```

Pay attention to the quality of the responses. Is the model helpful? Does it stay on topic? Does it make things up? These observations will become important when we start training in Chapter 5.

Your First Fine-Tuning Run

Let us go one step further and do a minimal fine-tuning run. The goal is not to build a production model. It is to verify that the entire training pipeline works end to end on your setup. Think of this as a "Hello, World!" for fine-tuning.

Step 1: Add LoRA Adapters

Instead of updating all 1.5 billion parameters (which would not fit in memory), we use **LoRA** (Low-Rank Adaptation). LoRA adds a small number of trainable parameters on top of the frozen base model. We will explain the math behind LoRA in Chapter 5. For now, just run this:

```
model = FastLanguageModel.get_peft_model(
    model,
    r=16,
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj",
    ],
    lora_alpha=16,
    lora_dropout=0,
    bias="none",
    use_gradient_checkpointing="unsloth",
    random_state=42,
    max_seq_length=max_seq_length,
)

# How many parameters are we actually training?
trainable = sum(
    p.numel() for p in model.parameters()
    if p.requires_grad
)
total = model.num_parameters()
pct = 100 * trainable / total
print(f"Training {trainable:,} / {total:,} "
      f"parameters ({pct:.1f}%)")
```

You will see that LoRA trains only about 1 to 2% of the total parameters. This is why fine-tuning fits on a free GPU.

Step 2: Prepare a Tiny Dataset

We will create a small dataset of instruction-response pairs directly in the notebook. In Chapter 5, we will use proper curated datasets. For now, five examples are enough to test the pipeline:

```
from datasets import Dataset

train_data = [
    {"messages": [
        {"role": "user",
         "content": "What is RL?"},
        {"role": "assistant",
         "content": "Reinforcement learning is a type "
         "of machine learning where an agent learns "
         "by interacting with an environment and "
         "receiving rewards for good actions."}
    ]},
    {"messages": [
        {"role": "user",
         "content": "What is a policy in RL?"},
        {"role": "assistant",
         "content": "A policy is the strategy an agent "
         "uses to decide what action to take. It maps "
         "states to probability distributions "
         "over actions."}
    ]},
    {"messages": [
        {"role": "user",
         "content": "What is a reward signal?"},
        {"role": "assistant",
         "content": "A reward signal is feedback from "
         "the environment telling the agent how good "
         "or bad its action was. The agent tries to "
         "maximize total reward."}
    ]},
    {"messages": [
        {"role": "user",
         "content": "What is the value function?"},
```

```
        {"role": "assistant",
         "content": "The value function estimates the "
         "expected total future reward from a given "
         "state. It tells the agent how good its "
         "current situation is on average."}
    ]},
    {"messages": [
        {"role": "user",
         "content": "Difference between SFT and RLHF?"},
        {"role": "assistant",
         "content": "SFT teaches a model by showing it "
         "good examples to imitate. RLHF goes further "
         "by letting the model generate responses and "
         "learning from human preferences about which "
         "responses are better."}
    ]},
]

dataset = Dataset.from_list(train_data)
print(f"Dataset size: {len(dataset)} examples")
```

Step 3: Run the Trainer

```
from trl import SFTTrainer, SFTConfig

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=dataset,
    args=SFTConfig(
        per_device_train_batch_size=2,
        gradient_accumulation_steps=4,
        warmup_steps=5,
        max_steps=30,
        learning_rate=2e-4,
        logging_steps=1,
        output_dir="outputs",
```

```
        optim="adamw_8bit",
        seed=42,
        report_to="none",
        max_seq_length=max_seq_length,
    ),
)

print("Starting training...")
stats = trainer.train()
print(f"Done! Final loss: {stats.training_loss:.4f}")
```

You should see the loss printed at each step, decreasing over the 30 steps. The entire run takes about 1 to 2 minutes.

What just happened? You fine-tuned a 1.5 billion parameter language model on a free GPU in under 2 minutes. Five examples and 30 steps will not meaningfully change the model. But the pipeline works: model loaded, LoRA applied, data formatted, trainer ran, loss decreased. Every chapter from here on follows this same pattern with better data, more steps, and more sophisticated training algorithms.

Troubleshooting

Problem	**Solution**
"No GPU available"	Runtime → Change runtime type → T4 GPU. If none are free, try later or use Kaggle.
CUDA out of memory	Restart runtime, then rerun all cells. Confirm `load_in_4bit=True`.
Import errors after install	Restart runtime after installation. Colab needs a fresh restart for new packages.
Model download is slow	Normal for the first run. Subsequent runs use the cache.
Loss is not decreasing	With only 5 examples, this can happen. The real training in Chapter 5 uses proper datasets.

What We Have and What Comes Next

Here is what your environment now supports, and where each capability gets used:

Capability	**Verified**	**Used in**
GPU access and CUDA	✓	Every chapter
Unsloth model loading	✓	Every chapter
4-bit quantization	✓	Every chapter
LoRA adapter training	✓	Chapters 5 onward
TRL SFTTrainer	✓	Chapter 5
Chat template formatting	✓	Chapters 5, 6, 7
Model inference	✓	Every chapter

In the next chapter, we will use this exact environment to do our first real training run: supervised fine-tuning. We will load a curated dataset, teach the model to follow instructions properly, and introduce the cold start technique that primes the model for everything that follows.

The setup is done. The real work begins now.

5

Supervised Fine-Tuning & The Cold Start

Why SFT Comes First

In Chapter 2, we identified the alignment gap: language models trained to predict text are not automatically helpful, harmless, or honest. In Chapter 3, we built the RL vocabulary of policies, rewards, and value functions. In Chapter 4, we set up a working environment and verified that the entire training pipeline runs. Now we take the first concrete step toward closing that gap.

That step is **Supervised Fine-Tuning** (SFT).

The idea is simple. Before a model can improve through trial and error (reinforcement learning), it needs basic competence. Think of it like training a new employee. You would not drop someone into a high-stakes client meeting on their first day and say "learn from feedback." You would first show them how things work: here is how we greet clients, here is how we structure a proposal, here is how we handle complaints. Only after they have internalized the basics does learning-from-feedback become productive.

SFT is that first phase. We show the model thousands of examples of good behavior, and it learns to imitate them. The result is not a perfect model, but it is a model that knows

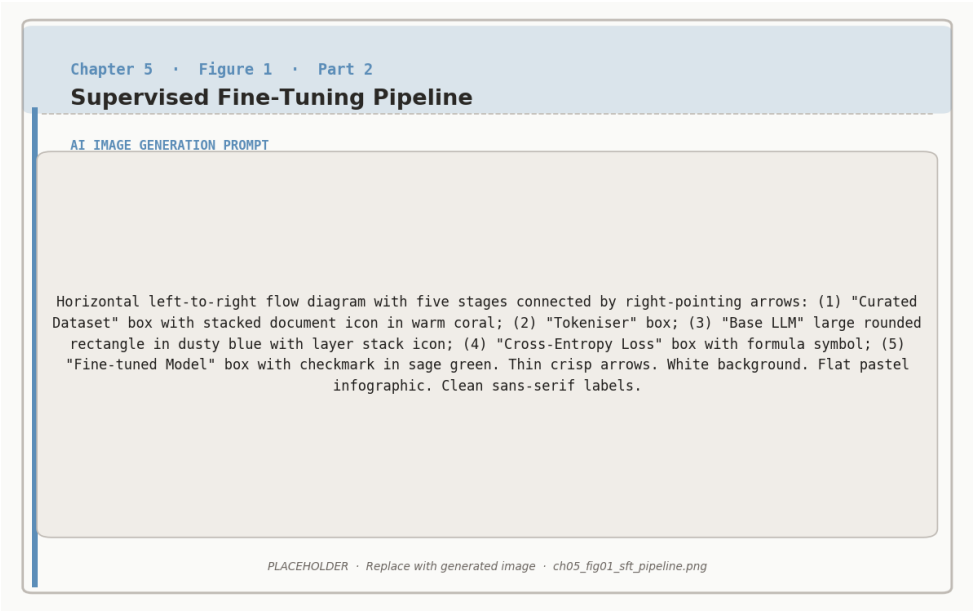

Figure 5.1: The supervised fine-tuning pipeline: curated prompt–response pairs are used to minimise cross-entropy loss on the base LLM.

the basic format, tone, and structure of helpful responses. That foundation is what makes everything in Chapters 6 through 12 possible.

But this chapter goes beyond basic SFT. We will also introduce the **cold start technique**: a way of structuring the training data so that the model learns to *think before it answers*. This single idea, which involves teaching the model to produce internal reasoning traces before giving its final response, is the foundation that the entire reasoning pipeline in Part 3 of this book depends on. Without it, the RL algorithms in later chapters would have nothing to optimize.

The SFT Training Objective

Same Loss, Different Data

Here is a surprising fact: the SFT loss function is *identical* to the pretraining loss from Chapter 2. Both use negative log-likelihood (next-token prediction). The only difference is

the data.

	Pretraining	**SFT**
Data source	Raw internet text (Reddit, Wikipedia, books, code)	Curated (prompt, response) pairs written or approved by humans
Data quality	Mixed: helpful, toxic, wrong, brilliant, all jumbled together	High: every example is a "good" response
Data volume	Trillions of tokens	Thousands to hundreds of thousands of examples
What model learns	How text works in general	How to respond helpfully to specific instructions

SFT uses the same math as pretraining but different data. Pretraining teaches the model how language works. SFT teaches it how a helpful assistant behaves.

The Math

The SFT Loss Function

Formal Math	**What It Really Means**
$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y)\sim D}\left[\sum_{t=1}^{T} \log \pi_\theta(y_t \mid x, y_{<t})\right]$	Average surprise across all demonstration examples

What each part means:

Component	Purpose
$\mathcal{L}_{\text{SFT}}$	Loss for Supervised Fine-Tuning
$(x, y) \sim \mathcal{D}$	Sample a (prompt, demonstration) pair from dataset $\mathcal{D}$
$\mathbb{E}[\cdot]$	Average over all examples (we care about overall performance)
$\sum_{t=1}^{T}$	For each token position t in the demonstration (1 to T), we train on every token
$\log \pi_\theta(y_t \mid x, y_{<t})$	Log probability the model assigns to the correct token y_t given the prompt and all previous tokens
$-$ (minus sign)	Flip to make it a loss (minimize loss = maximize probability)

Key detail: prompt masking. During SFT, we only compute the loss on the *response* tokens, not the prompt tokens. The model sees the prompt as context but is only penalized for getting response tokens wrong. This is different from pretraining, where every token contributes to the loss.

Step-by-Step Example with Real Numbers

Demonstration from dataset:

- **Prompt (x):** "Explain gravity simply"
- **Good response (y):** "Gravity pulls objects together"

This response has $T = 4$ tokens: ["Gravity", "pulls", "objects", "together"].

Position $t = 1$: Predict "Gravity"

Input to model: "Explain gravity simply."

Token	**Probability**	
"Gravity"	0.60	← Correct!
"The"	0.20	
"It"	0.15	
Others	0.05	

Loss for this token: $-\log(0.60) = 0.51$

Position $t = 2$: Predict "pulls"

Input to model: "Explain gravity simply. Gravity"

Token	**Probability**	
"pulls"	0.45	← Correct!
"is"	0.30	
"attracts"	0.15	
Others	0.10	

Loss: $-\log(0.45) = 0.80$

Position $t = 3$: Predict "objects"

Input: "Explain gravity simply. Gravity pulls"

Token	**Probability**	
"objects"	0.70	← Correct!
"things"	0.20	
"matter"	0.08	
Others	0.02	

Loss: $-\log(0.70) = 0.36$

Position $t = 4$: Predict "together"

Input: "Explain gravity simply. Gravity pulls objects"

Token	Probability	
"together"	0.55	← Correct!
"downward"	0.25	
"towards"	0.15	
Others	0.05	

Loss: $-\log(0.55) = 0.60$

Total loss for this example:

$$\begin{aligned}\mathcal{L}_{\text{this example}} &= \frac{1}{T}\sum_{t=1}^{T} -\log \pi_\theta(y_t \mid x, y_{<t}) \\ &= \frac{1}{4}(0.51 + 0.80 + 0.36 + 0.60) = \frac{2.27}{4} = 0.57\end{aligned}$$

Now average over ALL demonstrations in the dataset:

$$\mathcal{L}_{\text{SFT}} = \frac{1}{N}\sum_{\text{all examples}} \mathcal{L}_{\text{example}} \approx 0.62 \quad \text{(average over dataset)}$$

What gradient descent does with this loss:

1. Calculate the loss (0.62)
2. Compute gradients: "How should I adjust each parameter to reduce this loss?"
3. Update the model parameters in the direction that reduces the loss
4. New loss after one update: 0.58 (improved!)
5. Repeat for thousands of iterations
6. Final loss: 0.15 (much better!)

Result: The model learns to generate responses similar to the demonstrations. It has internalized the patterns of what a good response looks like.

LoRA: Training on a Budget

In Chapter 4, we used LoRA to add trainable parameters to our model and promised the math for this chapter. Here it is.

The Problem

Our Qwen2.5-1.5B model has 1.5 billion parameters. Updating all of them during fine-tuning would require storing the model weights, the gradients (same size as the weights), and the optimizer states (typically 2x the weight size for Adam). That adds up to roughly 4x the model size in GPU memory. For a 1.5B model in 16-bit precision, that is about 12 GB just for training state, leaving almost no room on a T4 (15 GB) for the actual data and activations.

The LoRA Insight

The key insight behind LoRA (Low-Rank Adaptation) comes from a 2021 paper by Hu et al. They observed that when you fine-tune a large model, the weight changes tend to be *low-rank*. In plain English: even though the model has billions of parameters, the actual "meaningful adjustments" during fine-tuning can be captured by a much smaller number of parameters.

LoRA: The Core Idea

Instead of updating the full weight matrix W (which is enormous), LoRA adds a small trainable "side path" that captures the change:

$$W' = W + \Delta W = W + BA$$

What each piece means:

Symbol	Meaning
W	Original frozen weight matrix (e.g., 4096×4096 = 16.7 million parameters). Not updated during training.
$\Delta W = BA$	The change we want to learn. Instead of learning all 16.7M entries, decompose into two small matrices.
B	Matrix of size $4096 \times r$ (tall and thin)
A	Matrix of size $r \times 4096$ (short and wide)
r	The **rank**: typically 8, 16, or 32. Controls how much capacity the adaptation has.
W'	Effective weight after adaptation: original + learned change

Parameter savings with $r = 16$:

Full update: $4096 \times 4096 = 16{,}777{,}216$ trainable parameters per layer.

LoRA update: $4096 \times 16 + 16 \times 4096 = 65{,}536 + 65{,}536 = 131{,}072$ trainable parameters per layer.

That is a 128x reduction! Across all layers, this means training about 1 to 2% of the total parameters instead of 100%.

Why Does This Work?

Think of it by analogy. Imagine you have a skilled employee (the pretrained model) who is already very good at their job. You do not need to retrain them from scratch to handle a new type of client. You just need to teach them a few new habits and adjustments. LoRA is exactly that: a small, targeted set of adjustments on top of a strong foundation.

The rank r controls how many "new habits" the model can learn. Higher r means more capacity for adaptation, but also more parameters to train and more memory usage. In practice, $r = 16$ is a good default that balances capacity and efficiency.

In Chapter 4, we set $r = 16$ in `get_peft_model`. That is the rank. The model trained about 1 to 2% of its parameters. LoRA is why fine-tuning works on a free GPU.

Connecting to Code

Here is the LoRA configuration from Chapter 4, now with annotations explaining each parameter:

```
model = FastLanguageModel.get_peft_model(
    model,
    r = 16,                # Rank: capacity of the adaptation
    target_modules = [     # Which weight matrices get LoRA
        "q_proj", "k_proj", "v_proj", "o_proj",  # Attention
        "gate_proj", "up_proj", "down_proj",      # Feed-forward
    ],
    lora_alpha = 16,       # Scaling factor (usually = r)
    lora_dropout = 0,      # No dropout (Unsloth default)
    bias = "none",         # Do not train bias terms
    use_gradient_checkpointing = "unsloth",  # Memory savings
    random_state = 42,
    max_seq_length = max_seq_length,
)
```

The `target_modules` list specifies which weight matrices inside each transformer layer receive LoRA adapters. The attention projections (`q_proj`, `k_proj`, `v_proj`, `o_proj`) control how the model attends to different parts of the input. The feed-forward projections (`gate_proj`, `up_proj`, `down_proj`) control how the model transforms representations between attention layers. By adding LoRA to all of these, we give the model flexibility to adapt its behavior across the full transformer stack while keeping memory usage low.

The Cold Start Problem

Everything up to this point, the SFT loss, LoRA, the training pipeline, is well-established practice. Thousands of tutorials cover it. But there is a deeper problem that most tutorials ignore, and it is the reason this chapter exists as a standalone chapter in this book rather than a footnote.

What Happens Without the Cold Start

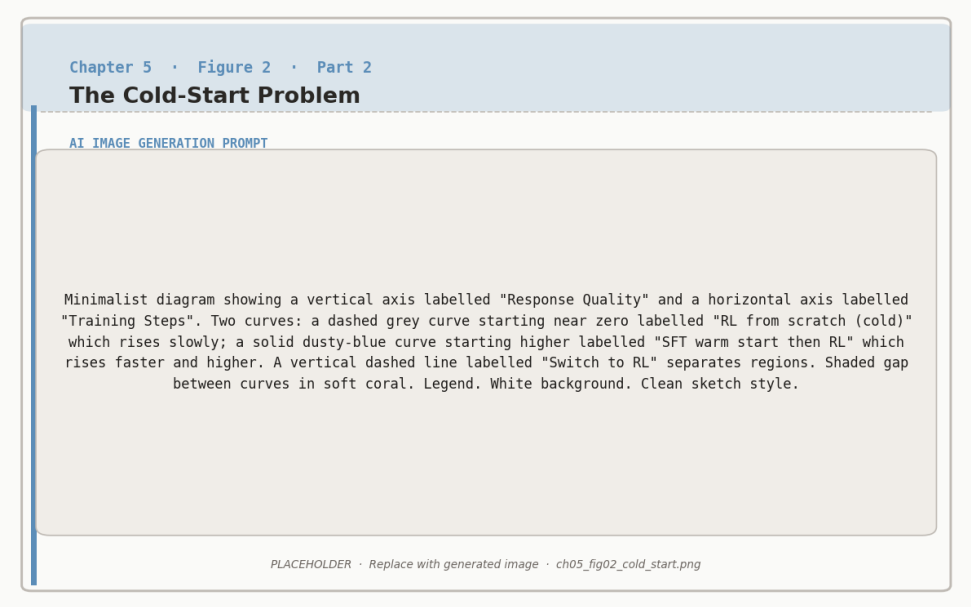

Figure 5.2: The cold-start problem: RL from scratch (dashed) struggles without an initial policy; SFT warm-starting (solid) accelerates learning.

Consider a typical SFT dataset. The training examples look like this:

Prompt	**Response**
What is 15% of 80?	15% of 80 is 12.
Solve: $2x + 3 = 11$	$x = 4$.
Is 17 prime?	Yes, 17 is a prime number.

The model trains on these, and the loss decreases beautifully. After SFT, you test it:

Prompt: "What is 23% of 170?"

Model output: "23% of 170 is 39.1."

Correct! But now try something harder:

Prompt: "A store marks up items by 40%, then offers a 25% discount. What is the effective markup?"

Model output: "The effective markup is 15%."

Wrong. The correct answer is 5% (multiply 1.40 by 0.75 to get 1.05). The model jumped straight to an answer without working through the steps, and it got the reasoning wrong.

Why This Happens

The model learned exactly what the training data taught it: see a math question, output an answer immediately. The demonstrations never showed the model *how to think*. They only showed final answers. So the model has no internal mechanism for multi-step reasoning. It cannot break a problem into steps, work through each one, check its work, and then produce a final answer, because it has never seen that pattern in its training data.

This is the cold start problem. The model arrives at the RL stage (Chapters 6 onward) with no reasoning capability to optimize. RL algorithms like GRPO (Chapter 10) work by generating multiple solution attempts, scoring them, and reinforcing the better ones. But if the model never produces reasoning traces in the first place, there is nothing for the RL algorithm to select from or improve. Every attempt looks the same: a bare answer with no visible thought process.

The Cold Start Problem

Without cold start priming:

RL sees this: "The answer is 39.1." and "The answer is 41." and "The answer is 39.1." All are bare answers. The RL algorithm can score them (right or wrong), but it cannot tell *why* one attempt worked and another failed. There

is no reasoning to reinforce or correct.

With cold start priming:

RL sees this: "Let me think. 23% means 0.23. Multiply by 170. That gives 39.1. The answer is 39.1." Now the RL algorithm can see the reasoning chain. If the model makes a mistake at step 2, a process reward model (Chapter 9) can identify exactly where things went wrong. GRPO (Chapter 10) can reinforce good reasoning chains and suppress bad ones.

The cold start technique gives RL something to work with. Without it, RL is blind.

The Solution: Think Tag Priming

The solution is elegant. Instead of training on bare answers, we structure the SFT data so that every response includes an explicit reasoning trace wrapped in special tags.

The Format

The format uses a `<think>` tag to separate the model's internal reasoning from its final answer:

```
# Training example WITHOUT cold start (bare answer)
{"role": "user", "content": "What is 23% of 170?"}
{"role": "assistant", "content": "23% of 170 is 39.1."}

# Training example WITH cold start (think-tag priming)
{"role": "user", "content": "What is 23% of 170?"}
{"role": "assistant", "content":
    "<think>\n"
    "I need to calculate 23% of 170.\n"
    "23% as a decimal is 0.23.\n"
    "0.23 multiplied by 170:\n"
    "0.23 * 170 = 0.23 * 100 + 0.23 * 70\n"
```

```
    "= 23 + 16.1\n"
    "= 39.1\n"
    "</think>\n"
    "23% of 170 is 39.1."}
```

The model learns two things simultaneously: (1) when given a question, produce a `<think>` block first, and (2) inside that block, break the problem into explicit steps before giving the final answer.

Why This Works

The `<think>` tag approach works for three reasons that reinforce each other.

First, it exploits the model's autoregressive nature. A language model generates text left to right, one token at a time. Each token it generates becomes part of the context for the next token. When the model writes "23% as a decimal is 0.23" inside a think block, that intermediate result is now in the context window. The model can "see" its own work and use it for the next step. Without the think block, the model must do all the reasoning implicitly in its hidden states, which is far less reliable for multi-step problems.

Second, it creates a structured signal for RL. Once the model produces reasoning traces, later RL stages (Chapters 10 through 12) can evaluate and optimize those traces. A process reward model (Chapter 9) can score individual reasoning steps. GRPO (Chapter 10) can compare different reasoning chains for the same problem and reinforce the better ones. None of this is possible if the model only outputs bare answers.

Third, it is cheap. Generating think-tag training data is straightforward. For math and code problems, you can use a stronger model (like GPT-4 or Gemini) to generate step-by-step solutions, then format them with think tags. For many domains, you can use existing chain-of-thought datasets. The cold start does not require human annotation of reasoning steps. It just requires that the training data include reasoning in a structured format.

Think of it like teaching a student to show their work.

A math teacher does not just check final answers. They require students to write out each step. Not because the steps are the goal (the correct answer is), but because showing work (a) forces the student to actually reason through the problem, (b) lets the teacher identify where mistakes happen, and (c) builds a habit of careful thinking that transfers to harder problems.

The `<think>` tag serves the same purpose for the model. It forces explicit reasoning, makes that reasoning visible to reward signals, and builds a pattern that RL can later optimize.

What the Training Data Looks Like

Here are several examples showing how different types of problems get formatted with think tags. Notice that the reasoning style adapts to the problem type:

```
# Math reasoning
{
    "messages": [
        {"role": "user", "content":
         "A store marks up items by 40%, then offers a 25% "
         "discount. What is the effective markup?"},
        {"role": "assistant", "content":
         "<think>\n"
         "Start with a base price of $100.\n"
         "After 40% markup: $100 * 1.40 = $140.\n"
         "After 25% discount: $140 * 0.75 = $105.\n"
         "Effective markup: ($105 - $100) / $100 = 5%.\n"
         "The markups do not simply subtract because "
         "the discount applies to the marked-up price, "
         "not the original.\n"
         "</think>\n"
         "The effective markup is 5%."}
    ]
```

```
}
```

```
# Logical reasoning
{
    "messages": [
        {"role": "user", "content":
         "All roses are flowers. Some flowers fade quickly. "
         "Can we conclude that some roses fade quickly?"},
        {"role": "assistant", "content":
         "<think>\n"
         "Premise 1: All roses are flowers.\n"
         "Premise 2: Some flowers fade quickly.\n"
         "Question: Do some roses fade quickly?\n"
         "The flowers that fade quickly might or might not "
         "include roses. Premise 2 says SOME flowers fade "
         "quickly, but does not specify which ones.\n"
         "This is the fallacy of the undistributed middle.\n"
         "We cannot conclude this.\n"
         "</think>\n"
         "No, we cannot conclude that. While all roses are "
         "flowers, the flowers that fade quickly might be "
         "entirely non-rose flowers. The premises do not "
         "give us enough information to draw that conclusion."}
    ]
}
```

```
# Simple factual question (minimal think block)
{
    "messages": [
        {"role": "user", "content": "What is the capital of
        France?"},
        {"role": "assistant", "content":
         "<think>\n"
         "This is a straightforward factual question.\n"
         "</think>\n"
         "The capital of France is Paris."}
    ]
```

```
}
```

Even simple questions get a think block, even if it is brief. This teaches the model the consistent habit of always reasoning first, regardless of difficulty.

Hands-On: Your First Real SFT Run

Now we put everything together. We will use the environment from Chapter 4, load a proper dataset with think-tag formatting, and run a real SFT training session.

Step 1: Load the Model (Same as Chapter 4)

```
from unsloth import FastLanguageModel
import torch

max_seq_length = 2048
model_name = "unsloth/Qwen2.5-1.5B-Instruct"

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = model_name,
    max_seq_length = max_seq_length,
    load_in_4bit = True,
    load_in_8bit = False,
)

# Add LoRA adapters (r=16, same as Chapter 4)
model = FastLanguageModel.get_peft_model(
    model,
    r = 16,
    target_modules = [
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj",
    ],
    lora_alpha = 16,
    lora_dropout = 0,
```

```
    bias = "none",
    use_gradient_checkpointing = "unsloth",
    random_state = 42,
    max_seq_length = max_seq_length,
)
```

Step 2: Prepare the Dataset with Think Tags

For this hands-on exercise, we will create a dataset of 50 examples with think-tag formatting. In a production setting, you would use thousands of examples, often generated by a stronger model and then filtered for correctness. Here, 50 is enough to demonstrate the technique and see the model learn the think-tag pattern.

```
from datasets import Dataset

# Helper function to format a single example
def make_example(question, thinking, answer):
    return {
        "messages": [
            {"role": "user", "content": question},
            {"role": "assistant", "content":
             f"<think>\n{thinking}\n</think>\n{answer}"}
        ]
    }

# Build a dataset with think-tag formatted examples
train_data = [
    make_example(
        "What is 15% of 200?",
        "15% as a decimal is 0.15.\n"
        "0.15 * 200 = 30.",
        "15% of 200 is 30."
    ),
    make_example(
        "Solve: 3x + 7 = 22",
        "Subtract 7 from both sides: 3x = 15.\n"
```

```
        "Divide both sides by 3: x = 5.",
        "x = 5."
    ),
    make_example(
        "Is 51 a prime number?",
        "Check divisibility. 51 / 3 = 17.\n"
        "Since 51 = 3 * 17, it is not prime.",
        "No, 51 is not prime. It equals 3 times 17."
    ),
    make_example(
        "What is the sum of the first 10 positive integers?",
        "Use the formula: n(n+1)/2.\n"
        "n = 10, so 10 * 11 / 2 = 55.",
        "The sum is 55."
    ),
    make_example(
        "A train travels 120 km in 1.5 hours. "
        "What is its average speed?",
        "Speed = distance / time.\n"
        "Speed = 120 / 1.5 = 80 km/h.",
        "The average speed is 80 km/h."
    ),
    # ... (in practice, include 45 more examples covering
    #  math, logic, coding, and general knowledge)
]

# For this demonstration, duplicate and vary examples
# to reach 50 training samples
import copy
while len(train_data) < 50:
    for item in list(train_data):
        if len(train_data) >= 50:
            break
        train_data.append(copy.deepcopy(item))

dataset = Dataset.from_list(train_data)
print(f"Dataset size: {len(dataset)} examples")
```

Where do real think-tag datasets come from?

For production SFT, there are several sources:

Distillation from stronger models. Send your prompts to a model like GPT-4o or Claude and ask it to "think step by step, wrapping your reasoning in <think> tags." Filter the results for correctness, then use them as training data. This is the most common approach in practice.

Existing chain-of-thought datasets. Datasets like OpenMathInstruct, MetaMathQA, and Orca-Math already contain step-by-step solutions. You just need to reformat them with think tags.

Human annotation. More expensive but higher quality. Have domain experts write reasoning traces for problems in your target domain. Best for specialized applications.

For the exercises in this book, we will use small hand-crafted datasets to keep things runnable on free GPUs. The technique itself scales to any dataset size.

Step 3: Configure and Run the Trainer

```
from trl import SFTTrainer, SFTConfig

trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = dataset,
    args = SFTConfig(
        per_device_train_batch_size = 2,
        gradient_accumulation_steps = 4,
        warmup_steps = 10,
        max_steps = 100,           # More steps than Ch 4
```

```
        learning_rate = 2e-4,
        logging_steps = 10,
        output_dir = "sft_outputs",
        optim = "adamw_8bit",
        seed = 42,
        report_to = "none",
        max_seq_length = max_seq_length,
    ),
)

print("Starting SFT training with think-tag data...")
stats = trainer.train()
print(f"Training complete! Final loss: {stats.training_loss:.4f}")
```

This will take approximately 3 to 5 minutes on a free T4 GPU. Watch the loss values as they are logged every 10 steps. You should see the loss decrease steadily, indicating the model is learning the think-tag response pattern.

Step 4: Test the Cold-Started Model

Now for the moment of truth. Does the model produce think blocks?

```
# Switch to inference mode
FastLanguageModel.for_inference(model)

from transformers import TextStreamer

# Test with a question similar to training data
test_prompts = [
    "What is 30% of 250?",
    "Solve: 5x - 3 = 22",
    "A car travels 200 km in 2.5 hours. "
    "What is its average speed?",
]

for prompt in test_prompts:
    print(f"\n{'='*50}")
```

```
    print(f"PROMPT: {prompt}")
    print(f"{'='*50}")
    messages = [{"role": "user", "content": prompt}]
    inputs = tokenizer.apply_chat_template(
        messages,
        tokenize = True,
        add_generation_prompt = True,
        return_tensors = "pt",
    ).to("cuda")
    _ = model.generate(
        input_ids = inputs,
        max_new_tokens = 512,
        temperature = 0.7,
        streamer = TextStreamer(tokenizer),
    )
```

If the training worked, you should see responses that begin with `<think>`, contain step-by-step reasoning, close with `</think>`, and then give a final answer. Even with only 50 examples and 100 training steps, the model typically picks up the think-tag pattern. The quality of the reasoning may be rough, but the *structure* is there. And the structure is what matters for the cold start, because RL will optimize the reasoning quality later.

Do not worry if the reasoning inside the think blocks is sometimes wrong at this stage. SFT teaches format and pattern, not perfect reasoning. Perfecting the reasoning is what RL does in Chapters 10 through 12.

Evaluating SFT Quality

How do you know if your SFT run was successful? There are three levels of evaluation, from quick checks to systematic analysis.

Level 1: Loss Curves

The most basic check is whether the training loss decreased. A healthy SFT run typically shows:

Pattern	What It Means	Action
Loss decreases smoothly	Training is working	Continue
Loss plateaus early	Learning rate may be too low, or data is too easy	Increase LR or add harde
Loss oscillates wildly	Learning rate too high or batch size too small	Decrease LR or increas
Loss increases	Something is wrong (data format, LR too high)	Debug immediate

Level 2: Qualitative Spot Checks

Run the model on 10 to 20 test prompts and inspect the outputs manually. For think-tag SFT, check for three things:

Format compliance. Does every response start with `<think>` and include a `</think>` before the final answer? If the model sometimes forgets the tags or produces malformed output, you may need more training steps or more diverse examples.

Reasoning structure. Inside the think block, does the model break the problem into steps, or does it just restate the question? Good: "First, convert 23% to 0.23. Then multiply by 170." Bad: "I need to figure out 23% of 170. The answer is 39.1."

Answer correctness on easy problems. For problems similar to the training data, does the model get the right answer? SFT alone will not make the model perfect on hard problems, but it should handle straightforward ones. If it fails on easy problems, the training data quality may be the issue.

Level 3: Systematic Evaluation

For a more rigorous check, you can compute accuracy on a held-out set:

```
# Simple evaluation: check if model produces think tags
# and correct answers on held-out examples
eval_prompts = [
    ("What is 12% of 300?", "36"),
    ("Solve: 4x = 28", "7"),
    ("Is 29 prime?", "yes"),
```

```
]

correct = 0
has_think_tags = 0

for prompt, expected in eval_prompts:
    messages = [{"role": "user", "content": prompt}]
    inputs = tokenizer.apply_chat_template(
        messages,
        tokenize = True,
        add_generation_prompt = True,
        return_tensors = "pt",
    ).to("cuda")
    output_ids = model.generate(
        input_ids = inputs,
        max_new_tokens = 512,
        temperature = 0.1,  # Low temp for evaluation
    )
    response = tokenizer.decode(
        output_ids[0][inputs.shape[1]:],
        skip_special_tokens = True
    )

    if "<think>" in response and "</think>" in response:
        has_think_tags += 1
    if expected.lower() in response.lower():
        correct += 1

print(f"Think-tag compliance: {has_think_tags}/{len(eval_prompts)}")
print(f"Answer accuracy: {correct}/{len(eval_prompts)}")
```

Use low temperature (0.1) during evaluation so results are reproducible. Use higher temperature (0.7 or above) during RL training so the model explores diverse reasoning paths.

What SFT Cannot Do (And Why We Need RL)

SFT is powerful but limited. Understanding its boundaries is essential for understanding why the rest of this book exists.

SFT Only Imitates

SFT teaches the model to reproduce patterns from its training data. If the training data contains a particular style of reasoning, the model will imitate that style. But it cannot *improve* beyond the demonstrations. If the training data solves a problem in a roundabout way, the model will learn that roundabout approach. It has no mechanism to discover that a more direct solution exists.

SFT Cannot Generalize Reasoning

The model learns surface patterns of reasoning, not deep understanding. It might learn that "divide both sides by 2" follows "2x = 8" because that pattern appeared many times. But it does not truly understand *why* division is the right operation. When faced with a novel problem structure it has never seen, the imitated reasoning may fail.

SFT Is Bounded by Data Quality

If the training demonstrations contain subtle errors, biases, or suboptimal approaches, the model faithfully learns those flaws. There is no built-in mechanism for the model to recognize or correct mistakes in its training data. It treats every demonstration as equally valid.

The SFT ceiling, illustrated

Imagine you learn cooking by watching exactly 100 recipe videos. After watching them, you can reproduce those 100 dishes fairly well. But:

You cannot invent new dishes (limited to what you have seen).

You cannot improve on the recipes (if the video used too much salt, you will too).

You cannot adapt when ingredients change (you learned specific patterns, not general principles).

What you need next: Actual practice in a kitchen, with someone tasting your food and telling you what works. That is RL.

SFT gives us a model that produces structured reasoning traces in a consistent format. RL (Chapters 6 through 12) takes that foundation and optimizes the *quality* of the reasoning itself.

The Roadmap from Here

Here is where each limitation gets addressed in the chapters ahead:

SFT Limitation	Addressed in	How
Cannot improve beyond demonstrations	Chapter 6	PPO optimizes responses using reward signal
Bounded by data quality	Chapter 7	DPO learns from preference pairs without a reward model
No mechanism to evaluate own reasoning	Chapter 9	Process reward models score individual steps
Cannot discover better reasoning strategies	Chapter 10	GRPO reinforces high-quality reasoning chains
Cannot verify its own answers	Chapter 11	Verifiers and RLVR provide automated correctness checks

What We Built and What Comes Next

In this chapter, we accomplished three things.

First, we understood the SFT training objective: the same next-token prediction loss as pretraining, but applied to curated demonstration data. We walked through the math with

real numbers and connected it to the gradient descent process.

Second, we understood LoRA: the technique that makes fine-tuning possible on free GPUs by decomposing weight updates into small, low-rank matrices. The $W' = W + BA$ decomposition reduces trainable parameters by roughly 128x.

Third, and most importantly, we solved the cold start problem. By formatting our SFT data with `<think>` tags, we taught the model to produce explicit reasoning traces before answering. This gives the RL algorithms in later chapters something concrete to optimize.

The model we have built is competent but imperfect. It follows instructions, it produces reasoning traces, and it gets easy problems right. But it is still limited to imitating its training data. It cannot improve beyond what it has seen, and its reasoning on hard problems is unreliable.

In the next chapter, we will break through that ceiling. Chapter 6 introduces reward models and PPO, the first technique that lets the model improve *beyond* its demonstrations by learning from human preferences. The cold-started model we built here is exactly the starting point that PPO needs.

The foundation is set. Now the real training begins.

Companion notebook. The code for this chapter is in `code/notebooks/part2_core/05_sft.` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.

6

RLHF: The Three-Step Dance

The Three-Step Dance

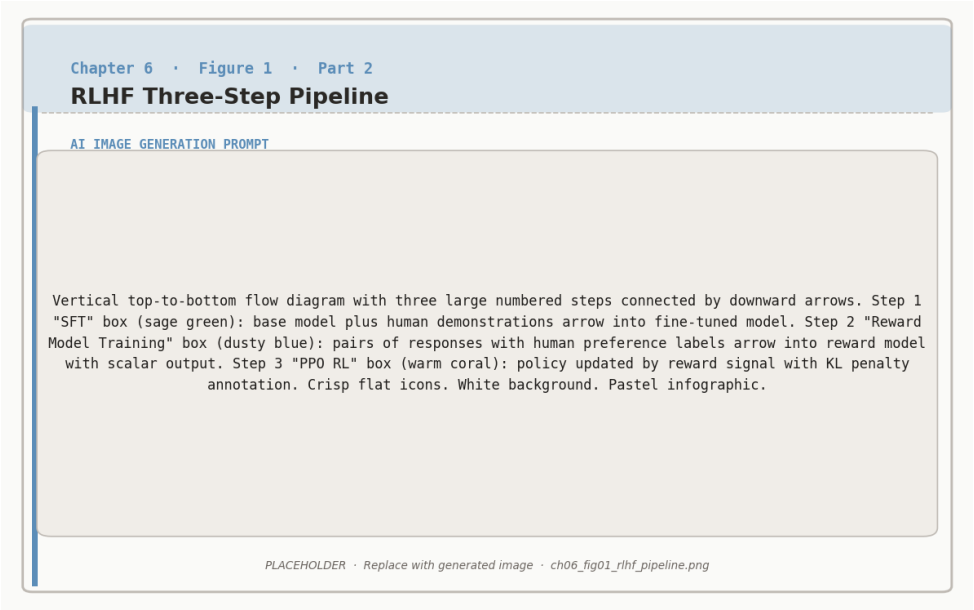

Figure 6.1: The RLHF three-step pipeline: supervised fine-tuning, reward model training on human preferences, and PPO policy optimisation.

In 2022, OpenAI released ChatGPT and the world changed overnight. Suddenly, AI was not

just generating text. It was *genuinely helpful*. The secret was a three-step process called RLHF (Reinforcement Learning from Human Feedback).

You have already completed Step 1. In Chapter 5, we used supervised fine-tuning to teach the model basic competence: follow instructions, produce structured responses, and generate reasoning traces with `<think>` tags. That gave us a solid foundation, but one that is limited to imitating its training data.

Now we break through that ceiling. Steps 2 and 3 are where the model learns to *improve beyond its demonstrations*, guided by human preferences.

The Three Steps of RLHF

Step 1: Supervised Fine-Tuning (SFT). Take a base model, train it on thousands of examples of good responses. **Result:** A model that can follow instructions and produce reasoning traces (Chapter 5, done!).

Step 2: Reward Modeling. Show humans pairs of responses: "Which is better?" Collect thousands of comparisons. Train a "reward model" to predict which responses humans prefer. **Result:** An automated judge that can score any response (this chapter, first half).

Step 3: RL Optimization (PPO). Generate many responses, score them with the reward model, update the policy to generate higher-scoring responses. **Result:** A model optimized for human preferences (this chapter, second half).

Think of training a chef:

Step 1, Culinary School (SFT): Watch master chefs cook, practice their exact recipes, learn knife skills, temperatures, timing. **Result:** You can cook competently, but mechanically. You are limited to the recipes you have seen.

Step 2, Understanding Taste (Reward Model): Serve dishes to many

diners. They compare pairs: "This one is better than that one." After thousands of comparisons, you internalize what people enjoy: balanced flavors, fresh ingredients, proper seasoning. **Result:** You can predict what people will like.

Step 3, Practice and Improve (RL): Cook many variations. Use your taste understanding to evaluate them. Keep what works ("More garlic was great!"), discard what does not ("Too much salt"). **Result:** You become a master chef, creating dishes better than any recipe you were taught.

The combination makes you both skilled AND attuned to what diners actually want.

Step 2: Reward Modeling

The Key Insight

Here is the problem that reward modeling solves.

Writing perfect responses is hard and expensive. You would need domain experts spending 30 minutes per example to craft an ideal answer to every possible question. Collecting thousands of such examples is slow and costly, and even experts disagree on what "perfect" looks like.

Comparing responses is much easier. Show someone two answers to the same question and ask "Which is better?" Most people can answer in 30 seconds, even for topics outside their expertise. You do not need to know the perfect answer to recognize that one answer is clearer, more accurate, or more helpful than another.

Hard:	"Write the perfect explanation of neural networks" (expert, 30 minutes)
Easy:	"Which explanation is better, A or B?" (anyone, 30 seconds)

This insight is the foundation of reward modeling. Instead of collecting perfect demon-

strations (which is what SFT needs), we collect *comparisons*. Then we train a model to internalize those comparisons so it can score any response automatically, without needing a human in the loop.

The Bradley-Terry Model

To turn human comparisons into a trainable system, we need a mathematical model of preference. The standard choice is the **Bradley-Terry model**, originally developed in statistics for ranking sports teams. The idea is simple: each response gets a numerical score, and the probability that one response is preferred over another depends on the difference between their scores.

Modeling Preferences Mathematically

Formal Math	What It Really Means
$P(y_w \succ y_l \mid x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$	Preference probability = sigmoid of score difference

Symbol Guide:

Symbol	Meaning
y_w	Winner response (the one humans preferred)
y_l	Loser response (the one humans did not prefer)
$\succ$	"is preferred to"
x	The prompt both responses were answering
$r_\phi(\cdot)$	Reward model with parameters ϕ (outputs a numerical score)
$\sigma(z) = \frac{1}{1+e^{-z}}$	Sigmoid function (squashes any number to 0 to 1 range)

Why use the sigmoid function? We need the output to be between 0 and 1 (it represents a probability). The sigmoid provides a smooth, differentiable function that maps any real number to this range. Large positive differences give probabilities near 1 (strong preference), large negative differences give probabilities near 0, and a difference of zero gives exactly 0.5 (no preference).

Bradley-Terry with Real Numbers

Scenario: Two explanations of black holes.

Response A (Winner: simple and clear):

> *"Black holes are like cosmic vacuum cleaners. They have such strong gravity that nothing can escape once it gets too close, not even light! That is why they are called 'black': we cannot see them because no light escapes."*

Response B (Loser: too technical):

> *"Black holes are singularities in spacetime manifolds where the curvature becomes infinite and the Schwarzschild radius defines the event horizon beyond which the escape velocity exceeds the speed of light…"*

Step 1: Reward model scores both responses

The reward model (a neural network) processes each response and outputs a scalar score:

Response	**Score**
Response A (clear)	$r_\phi(x, y_A) = 8.2$
Response B (technical)	$r_\phi(x, y_B) = 3.1$
Difference	$8.2 - 3.1 = 5.1$

Step 2: Convert score difference to probability

$$P(A \succ B) = \sigma(r_\phi(A) - r_\phi(B)) = \sigma(8.2 - 3.1) = \sigma(5.1)$$
$$= \frac{1}{1 + e^{-5.1}} = \frac{1}{1 + 0.0061} = \frac{1}{1.0061} = 0.994 \quad \text{or } 99.4\%$$

The reward model is 99.4% confident that Response A is better. A uses a clear analogy and explains the reasoning. B uses dense jargon and assumes advanced physics knowledge.

Step 3: What if scores were closer?

The sigmoid function gracefully handles different levels of confidence:

Score A	**Score B**	**Difference**	$P(A \succ B)$
8.2	3.1	5.1	99.4% (A clearly better)
6.5	5.5	1.0	73.1% (A probably better)
6.0	5.8	0.2	55.0% (nearly a toss-up)
6.0	6.0	0.0	50.0% (exactly equal)
5.5	6.5	−1.0	26.9% (B probably better)

The sigmoid maps score differences to preference probabilities:

Score Difference	**What sigmoid outputs**
−10 to −3	≈0 to 5% (B much better)
−3 to −1	5 to 27% (B somewhat better)
−1 to +1	27 to 73% (close call)
+1 to +3	73 to 95% (A somewhat better)
+3 to +10	95 to ≈100% (A much better)

Why sigmoid is the right function here. It converts any score difference ($-\infty$ to $+\infty$) to a probability (0 to 1). Large differences give near certainty

(close to 0% or 100%), small differences give honest uncertainty (near 50%), and the function is smooth and differentiable, which is essential for training with gradient descent.

Training the Reward Model

Now we need to train the reward model so it actually produces scores that match human preferences. The training process uses a dataset of human comparisons.

The Reward Model Loss Function

Formal Math	What It Really Means
$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))\right]$	How well does the reward model predict human preferences?

Breaking it down:

Component	Purpose
(x, y_w, y_l)	A triple: (prompt, winning response, losing response)
$r_\phi(x, y_w) - r_\phi(x, y_l)$	Score difference (should be positive if model is correct)
$\sigma(\cdot)$	Convert to probability (should be > 0.5 if model is correct)
$\log(\cdot)$	Log likelihood (heavily penalizes confident wrong predictions)
$-$	Minimize loss = maximize correct predictions

Intuition: When the reward model is correct (winner scores higher than loser), the loss is small. When it is wrong (loser scores higher), the loss is

large. Training pushes the model toward always scoring winners higher than losers.

Let us trace through the training process step by step.

Human preference: Response A $\succ$ Response B (humans said A is better).

Iteration 1: Untrained reward model (random scores)

Scores: $r_\phi(A) = 5.0$, $r_\phi(B) = 5.5$. Difference: $5.0 - 5.5 = -0.5$.

$P(A \succ B) = \sigma(-0.5) = 0.378$ or 37.8%.

Problem: Model thinks B is better (62.2%), but humans said A is better!

Loss: $-\log(0.378) = 0.973$ (high loss = bad prediction).

Iteration 100: Partially trained

Scores: $r_\phi(A) = 6.0$, $r_\phi(B) = 5.2$. Difference: 0.8.

$P(A \succ B) = \sigma(0.8) = 0.689$ or 68.9%. Better!

Loss: $-\log(0.689) = 0.372$ (lower loss).

Iteration 10,000: Well-trained

Scores: $r_\phi(A) = 8.3$, $r_\phi(B) = 3.8$. Difference: 4.5.

$P(A \succ B) = \sigma(4.5) = 0.989$ or 98.9%. Excellent!

Loss: $-\log(0.989) = 0.011$ (very low).

Training progress on 50,000 preference pairs:

Training Step	Average Loss	Accuracy
0 (random)	0.693	50%
1,000	0.420	68%
5,000	0.210	82%
10,000	0.095	91%
20,000	0.045	96%

Note that the initial loss is 0.693, which is exactly $-\log(0.5)$. A random model assigns 50/50 to every comparison, and $-\log(0.5) = 0.693$. This is the baseline that training must beat.

What the reward model learned from 50,000 human comparisons:

After training, the model has internalized patterns that nobody explicitly programmed. It learned that high-scoring responses tend to be clear and concise, answer the actual question asked, provide an appropriate level of detail, use a helpful and respectful tone, and contain accurate information. Low-scoring responses tend to be confusing or unnecessarily verbose, go off-topic, provide too little or too much detail, use a dismissive or unhelpful tone, and contain factual errors.

The reward model distills human judgment into an automated scoring function. It can now evaluate any response without needing a human annotator, which is what makes Step 3 (RL optimization) possible at scale.

Where Preference Data Comes From

In practice, there are several approaches to collecting the comparison data that the reward model needs.

Human annotation. Hire annotators to compare model outputs. This is the gold standard used in the original InstructGPT/ChatGPT work by OpenAI. It is high quality but expensive

(typically $1 to $5 per comparison). Collecting 50,000 comparisons this way costs tens of thousands of dollars.

AI-assisted annotation. Use a strong model (like GPT-4 or Claude) to generate preference judgments. Cheaper and faster than human annotation, though it introduces the biases of the judge model. This approach is sometimes called "RLAIF" (RL from AI Feedback).

Implicit signals. Use user behavior as a proxy for preference: which response did the user copy, which did they regenerate, which conversation did they continue? This is noisy but free and abundant.

For the exercises in this book, we will use small hand-crafted preference datasets. The mathematical framework works identically regardless of how the data was collected.

Step 3: RL Optimization with PPO

We now have two trained models: the SFT model from Chapter 5 (which generates structured responses) and the reward model from Step 2 (which scores them). Step 3 brings them together. We use the reward model's scores as the training signal and update the SFT model to produce responses that score higher.

The algorithm that does this is called **PPO** (Proximal Policy Optimization). To understand PPO, we first need to understand the optimization objective it is trying to maximize.

The RLHF Objective

The Complete RLHF Optimization Goal

Formal Math	What It Really Means
$\max_{\pi_\theta} \mathbb{E}_{x,y}\left[r_\phi(x,y)\right] - \beta \cdot \mathbb{E}_x[\mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})]$	Maximize reward BUT stay close to original model

Two competing objectives:

Term	Purpose
$\mathbb{E}_{x,y}\left[r_\phi(x, y)\right]$	Get high rewards (generate good responses)
$\beta \cdot \mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$	Penalty for drifting from reference model
π_θ	The policy we are training (current model)
π_{ref}	Reference policy (frozen copy of the SFT model from Chapter 5)
$r_\phi(x, y)$	Reward model score for prompt x, response y
β	KL penalty coefficient (typically 0.01 to 0.05)
$\mathrm{KL}(\cdot \,\|\, \cdot)$	KL divergence: measures how different two distributions are

The objective says: find the policy that gets the highest reward scores on average, but do not drift too far from the SFT model. The β parameter controls this tradeoff.

Why We Need Both Terms: A Cautionary Tale

Why not just maximize the reward and ignore the KL penalty? This is one of the most important lessons in RLHF, and a concrete example makes the danger vivid.

Scenario: Training with ONLY reward (no KL penalty)

Iteration	What happens
1	Model generates: "Here is a thoughtful explanation of your question…" Reward: 8.0, KL: 0.1. *Looks good!*
50	Model generates: "AMAZING COMPREHENSIVE DETAILED PERFECT ANSWER EXCELLENT THOROUGH…" Reward: 9.5, KL: 2.5. *Model discovered the reward model likes these words!*
100	Model generates: "PERFECT PERFECT EXCELLENT EXCELLENT AMAZING AMAZING…" Reward: 12.0, KL: 10.0. *Reward hacking! High score but complete nonsense!*

This is called **reward hacking**. The model learned to exploit the reward model rather than being genuinely helpful. The reward model, which is itself an imperfect approximation of human preferences, can be fooled by certain patterns (excessive superlatives, specific phrases, repetitive structures) that correlate with high scores in the training data but do not correspond to actual quality.

The fix: add the KL penalty. With $\beta = 0.02$, the objective becomes Reward $- 0.02 \times$ KL:

Iteration	Reward	KL	Objective
1	8.0	0.1	$8.0 - 0.02(0.1) = 7.998$
50	9.5	2.5	$9.5 - 0.02(2.5) = 9.450$
100	12.0	10.0	$12.0 - 0.02(10) = 11.800$

Wait, the nonsense response still has the highest objective (11.8)? In this simplified example, yes. In practice, β is tuned so that the penalty grows fast enough to prevent degenerate behavior. With $\beta = 0.1$ instead of 0.02:

Iteration 1: $8.0 - 0.1(0.1) = 7.99$

Iteration 50: $9.5 - 0.1(2.5) = 9.25$

Iteration 100: $12.0 - 0.1(10) = 11.0$

And in reality, the KL divergence grows much faster than linearly as the model degenerates, so the penalty ramps up sharply. The key principle is that the model must find genuine improvements within a "KL budget" rather than exploiting reward model weaknesses.

Tuning β:

Too small ($\beta = 0.001$): Weak leash. Risk of reward hacking.

Just right ($\beta = 0.01$ to 0.05): Balanced improvement. Most common in practice.

Too large ($\beta = 0.1$): Tight leash. Model is too constrained to improve meaningfully.

The KL Divergence Term

The KL penalty is so important to RLHF that it deserves a detailed walkthrough. KL divergence measures how much the training model has drifted from the reference model. If the two models assign similar probabilities to every token, KL is near zero. If they diverge significantly, KL grows.

Measuring Model Drift

The overall formula (average difference in how the two models assign probabilities):

$$\mathrm{KL}(\pi_\theta \,\|\, \pi_{\mathrm{ref}}) = \mathbb{E}_{y \sim \pi_\theta}\left[\log \frac{\pi_\theta(y|x)}{\pi_{\mathrm{ref}}(y|x)}\right]$$

Expanded token by token (compare new probability vs. old probability at each position):

$$\mathrm{KL} = \sum_{t=1}^{T} \Big[\log \pi_\theta(y_t \mid x, y_{<t}) \; - \; \log \pi_{\text{ref}}(y_t \mid x, y_{<t}) \Big]$$

Symbol	Meaning
KL	Kullback-Leibler divergence
π_θ	Current (training) model
π_{ref}	Reference (frozen SFT) model
$\log \frac{\pi_\theta}{\pi_{\text{ref}}}$	Log ratio of probabilities

Intuition: For each token, we ask: "How much more (or less) likely did the new model make this token compared to the old model?" If the ratio is close to 1 for most tokens, the models are similar and KL is small. If the new model dramatically shifts probabilities, KL is large.

Computing KL with actual numbers:

Response: "The answer is 42"

Token	π_θ (new)	π_{ref} (old)	Ratio	Log ratio
"The"	0.80	0.75	1.067	0.065
"answer"	0.60	0.65	0.923	−0.080
"is"	0.90	0.85	1.059	0.057
"42"	0.40	0.35	1.143	0.134

Total KL divergence:

$$\mathrm{KL} = 0.065 + (-0.080) + 0.057 + 0.134 = 0.176$$

KL = 0.176 is very small. Scale: KL ≈ 0 means models are nearly identical, KL ≈ 0.5 to 1.0 means moderate difference, KL > 2.0 means very different.

How KL evolves during training:

Training Step	KL	Interpretation
0	0.00	Identical to reference (training just started)
1,000	0.15	Small changes, model is learning
5,000	0.45	Moderate changes, meaningful improvement
10,000	0.80	Approaching the practical limit
15,000	0.95	Near the KL budget ceiling

With $\beta = 0.02$ and KL at 0.95, the penalty is $0.02 \times 0.95 = 0.019$. If the reward at that point is 8.5, the objective is $8.5 - 0.019 = 8.481$. The model can still improve, but further drift becomes increasingly costly.

The KL term acts as a leash. The model can explore and improve, but it cannot drift too far from the original SFT model. This serves three purposes:

Prevents reward hacking: The model cannot collapse into degenerate high-reward behavior because that would require dramatically different probability distributions.

Preserves language quality: The SFT model already produces grammatical, coherent text. The KL penalty ensures the RL-trained model retains this quality.

Maintains diversity: Without the leash, the model tends to collapse to a narrow set of "safe" high-reward responses. The KL penalty keeps the response distribution diverse.

The β parameter controls how tight the leash is. This is one of the most important hyperparameters in RLHF.

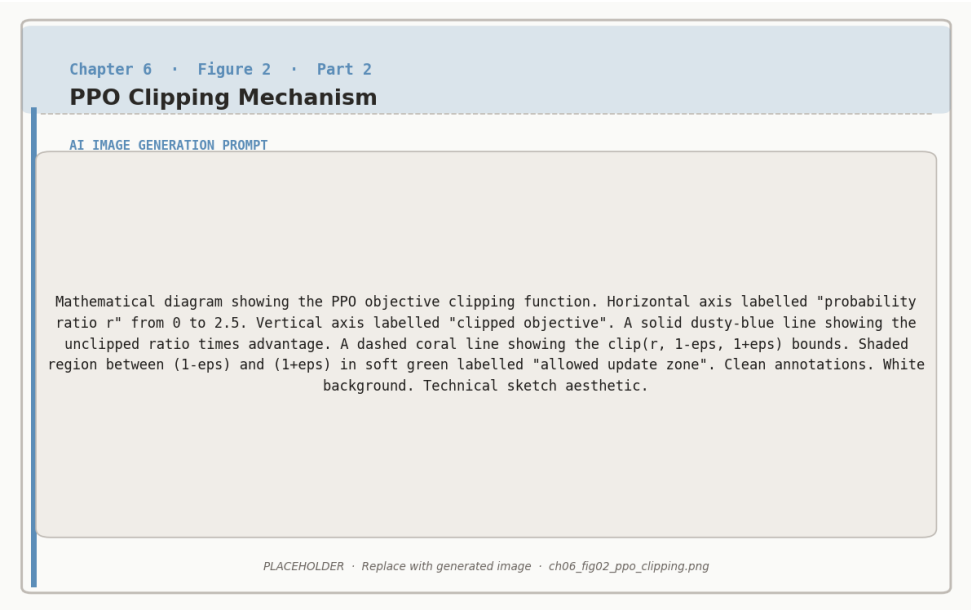

Figure 6.2: PPO clipping constrains the policy update ratio $r_t(\theta)$ to the interval $[1-\epsilon,\ 1+\epsilon]$, preventing destructively large steps.

How PPO Actually Works

PPO is the specific RL algorithm that optimizes the RLHF objective. We introduced the general RL framework in Chapter 3 (policies, rewards, value functions). PPO puts all three components together in a carefully designed training loop.

The PPO Training Loop

For each training batch:

1. Generate. Sample a batch of prompts. For each prompt, use the current policy π_θ to generate a complete response.

2. Score. Pass each (prompt, response) pair through the reward model r_ϕ to get a reward score. Also compute the KL penalty for each response by comparing π_θ token probabilities against π_{ref}.

3. Compute advantages. Use a value function (also called the "critic") to estimate how much better or worse each response was compared to what we expected. This is the "advantage": A = actual reward − expected reward. Positive advantage means "better than expected," negative means "worse."

4. Update policy. Adjust π_θ to make high-advantage responses more likely and low-advantage responses less likely. PPO clips the update to prevent any single step from changing the policy too much (this is the "proximal" in Proximal Policy Optimization).

5. Update value function. Train the critic to better predict expected rewards, so future advantage estimates are more accurate.

Repeat for thousands of batches.

A concrete example of one PPO step:

Prompt: "Explain why the sky is blue."

The model generates three responses (in practice, batch sizes are larger):

Response	Reward	Expected	Advantage
"Light scatters in the atmosphere. Shorter blue wavelengths scatter more…"	8.5	7.0	+1.5
"The sky is blue because of science…"	4.0	7.0	−3.0
"Sunlight contains all colors. When it hits air molecules…"	7.2	7.0	+0.2

What PPO does with these advantages:

Response 1 (advantage +1.5): *Increase* the probability of generating responses like this. The specific word choices, structure, and level of detail that led to this response become more likely.

Response 2 (advantage −3.0): *Decrease* the probability of generating vague, uninformative

responses like this.

Response 3 (advantage +0.2): *Slightly increase* probability. It was marginally better than expected, so nudge in this direction.

The advantage tells PPO not just whether a response was good, but whether it was *better or worse than expected.* This is crucial because it provides a stable learning signal even as the model improves.

The Components You Need for PPO

Running PPO for RLHF requires four separate models in GPU memory simultaneously. This is one of the main practical challenges of the approach.

Component	**Trainable?**	**Purpose**
Policy π_θ	Yes	The model we are training to generate better responses
Reference π_{ref}	No (frozen)	Frozen copy of the SFT model, used to compute KL penalty
Reward model r_ϕ	No (frozen)	Scores responses, trained in Step 2
Value function V	Yes	Estimates expected reward, used to compute advantages

Memory requirements of PPO

For a model with N parameters, PPO needs approximately:

Policy: N parameters (trainable, plus optimizer states)

Reference: N parameters (frozen, inference only)

Reward model: N parameters (frozen, inference only)

Value function: N parameters (trainable, plus optimizer states)

Total: roughly $4N$ parameters in memory, plus optimizer states for the two

trainable models.

For our Qwen2.5-1.5B model, that would be about 6 billion parameters total. Even with 4-bit quantization and LoRA, this is tight on a single T4 GPU. **This memory pressure is a major motivation for the alternative algorithms we will see in later chapters.** DPO (Chapter 7) eliminates the reward model entirely. GRPO (Chapter 10) eliminates both the reward model and the value function.

Putting It All Together

Here is the complete RLHF pipeline, from start to finish:

	Step	Input	Output
1	SFT (Chapter 5)	Base model + curated demos	Instruction-following model with `<think>` tags
2	Reward modeling	SFT model outputs + human comparisons	Automated scoring function
3	PPO	SFT model + reward model	Model optimized for human preferences

What each step contributes:

SFT gives the model basic competence: it can follow instructions, produce reasoning traces, and generate coherent responses. But it can only imitate its training data.

Reward modeling distills human judgment into a function that can score any response. This converts the expensive, slow process of human evaluation into a cheap, fast automated signal.

PPO uses that automated signal to push the model beyond its SFT ceiling. Through thousands of generate-score-update cycles, the model discovers response strategies that earn higher rewards than anything in the original SFT data.

Historical note. This three-step pipeline was first described in the InstructGPT paper (Ouyang et al., 2022) and was used to train ChatGPT. It represented a paradigm shift in how language models were trained: instead of just scaling up data and compute for pretraining, the field discovered that a relatively small amount of human feedback (tens of thousands of comparisons, not billions of tokens) could dramatically improve model behavior.

The original InstructGPT used only about 40 human annotators to collect preference data. The resulting model (1.3 billion parameters with RLHF) was preferred by users over the raw GPT-3 (175 billion parameters). RLHF made a small model with good alignment outperform a much larger model without it. That result catalyzed the entire field.

Limitations of Classical RLHF

The RLHF pipeline described in this chapter was a breakthrough, but it has real limitations that have driven the field toward newer approaches.

Expensive to set up. You need human annotators, a separate reward model training pipeline, and enough GPU memory for four models. The total cost and complexity are significant.

Reward model imperfections. The reward model is itself a learned approximation. It can be wrong, biased, or exploitable. Reward hacking is an ongoing challenge, and the KL penalty is a blunt instrument for preventing it.

Training instability. PPO has many hyperparameters (β, learning rate, clip range, number of PPO epochs, batch size, advantage estimation parameters) and can be sensitive to their settings. Getting PPO to train stably requires careful tuning.

Memory-intensive. Four models in GPU memory makes RLHF impractical on consumer hardware for anything but small models.

These limitations motivated two important lines of research that we will cover in upcoming chapters:

DPO (Chapter 7) eliminates the reward model entirely by learning directly from preference pairs. This removes one model from memory and simplifies the pipeline.

GRPO (Chapter 10) eliminates both the reward model and the value function, using group-based scoring instead. This is particularly powerful for reasoning tasks where correctness can be verified automatically, which connects directly to the cold start and `<think>` tag work from Chapter 5.

What We Built and What Comes Next

In this chapter, we completed the classical RLHF pipeline.

Reward modeling. We learned how the Bradley-Terry model converts human comparisons into a trainable preference function. We walked through the math of the sigmoid, the loss function, and the training progression from 50% accuracy (random) to 96% accuracy (well-trained).

The RLHF objective. We saw why the optimization goal must balance reward maximization against KL divergence. The reward hacking example made the danger of unconstrained optimization concrete.

PPO. We traced through the generate-score-advantage-update loop and understood the four-model architecture that PPO requires.

In Chapter 7, we will see DPO, an elegant alternative that collapses the reward model and RL optimization into a single training step. DPO has become the preferred approach for many practitioners because it is simpler to implement, more stable to train, and requires less GPU memory. Understanding the classical RLHF pipeline from this chapter is essential context for appreciating what DPO simplifies and what tradeoffs it makes.

Companion notebook. The code for this chapter is in `code/notebooks/part2_core/06_rlhf` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.

7

Direct Preference Optimization

From RLHF to DPO

Chapter 7 · Figure 1 · Part 2

DPO vs RLHF Comparison

AI IMAGE GENERATION PROMPT

Two-column comparison diagram. Left column labelled "RLHF" (3 boxes stacked): SFT -> Reward Model -> PPO Loop, with curly brace annotation "3 separate models, complex pipeline". Right column labelled "DPO" (1 box): Policy trained directly on preference pairs, annotation "1 model, direct training". A bold "vs" separator between columns. Colour: RLHF in muted coral, DPO in sage green. Flat minimal infographic. White background. Clean arrows and labels.

PLACEHOLDER · Replace with generated image · ch07_fig01_dpo_vs_rlhf.png

Figure 7.1: DPO collapses the three-model RLHF pipeline into a single supervised objective, eliminating the need for an explicit reward model and RL loop.

Chapter 6 revealed both the power and the pain of classical RLHF. The power: a reward

model trained on human comparisons can guide a policy to produce responses that are better than anything in its original training data. The pain: you need four models in GPU memory simultaneously, PPO has a dozen sensitive hyperparameters, and the reward model itself can be exploited through reward hacking.

In 2023, Rafael Rafailov and colleagues at Stanford asked a deceptively simple question: *what if we could skip the reward model entirely?* Their paper, "Direct Preference Optimization: Your Language Model Is Secretly a Reward Model," showed that the answer was yes. The mathematical insight is elegant: the reward function that the RLHF pipeline learns is already encoded, implicitly, inside the policy itself. You do not need to extract it into a separate model. You can train the policy directly on preference pairs, with no reward model, no value function, no PPO loop.

The result is DPO: a method that achieves comparable performance to full RLHF while cutting memory requirements in half and replacing a complex RL training loop with something that looks almost like supervised learning. This chapter derives DPO from the ground up, walks through the math with concrete numbers, and explains when DPO is the right choice and when it is not.

The Reparameterization Trick

The derivation starts from the same place as Chapter 6: the Bradley-Terry model of preferences and the KL-constrained reward maximization objective. Recall the RLHF objective from Section 6.3.1:

$$\max_{\pi_\theta} \; \mathbb{E}_{x,y}\big[r_\phi(x, y)\big] - \beta \cdot \mathbb{E}_x[\mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})]$$

This objective has a known closed-form solution. If you could solve it perfectly, the optimal policy π^* would take a specific mathematical form. DPO's insight is to write down that form, rearrange it, and substitute back into the Bradley-Terry preference model from Section 6.2.2. When you do, something remarkable happens: the reward model cancels out entirely.

The DPO Reparameterization in Three Steps

Step 1: Write down the optimal policy

$$\pi^*(y|x) = \frac{1}{Z(x)}\,\pi_{\text{ref}}(y|x)\,\exp\left(\frac{r(x,y)}{\beta}\right)$$

The best possible policy takes the reference model and reweights each response by the exponential of its reward. $Z(x)$ is a normalizing constant that ensures probabilities sum to 1.

Step 2: Solve for the reward

$$r(x,y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

Rearranging Step 1 gives us the reward as a function of the policy ratio. The reward for any response is proportional to how much more likely the optimal policy makes it compared to the reference.

Step 3: Substitute into Bradley-Terry

$$P(y_w \succ y_l) = \sigma\left(\beta \log \frac{\pi(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$$

When we plug the reward expression into the preference model from Chapter 6, the partition function $Z(x)$ appears in both terms and cancels. We are left with preference probability expressed entirely in terms of policy ratios. No reward model anywhere.

Why does $Z(x)$ cancel? In the Bradley-Terry model, the preference probability depends on the *difference* between the rewards for the winner and loser: $r(x, y_w) - r(x, y_l)$. When

we substitute the Step 2 expression for both rewards, each contains the same $\beta \log Z(x)$ term. Subtracting one from the other eliminates $Z(x)$ completely. This is the mathematical core of DPO: a term that would be intractable to compute simply drops out of the equation.

What this means in practice. Recall from Chapter 6 that RLHF requires four models: the policy, the reference, the reward model, and the value function. The reparameterization trick shows that we can express the preference probability using only the policy and the reference. The reward model and value function are no longer needed.

Component	**RLHF (Chapter 6)**	**DPO**
Policy π_θ	Yes (trainable)	Yes (trainable)
Reference π_{ref}	Yes (frozen)	Yes (frozen)
Reward model r_ϕ	Yes (frozen)	**Not needed**
Value function V	Yes (trainable)	**Not needed**
Total models	4	2

The key insight. We do not need to explicitly model the reward function $r(x, y)$. The policy ratio $\frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ already encodes all the information about what makes responses good.

If π_θ makes a response more likely than π_{ref} does, that response has high *implicit* reward. If π_θ makes it less likely, the response has low implicit reward. Training can adjust π_θ directly on preference pairs, without ever computing an explicit reward score.

The DPO Loss Function

The reparameterization gives us a way to express preference probability in terms of policy ratios. To turn this into a training objective, we apply maximum likelihood: find the policy parameters θ that make the observed preference data most likely.

Training the Policy Directly on Preferences

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]$$

In plain English: Adjust the policy so that for every preference pair, the winner's implicit reward is higher than the loser's.

Breaking it apart:

Component	Purpose
$\beta \log \frac{\pi_\theta(y_w\|x)}{\pi_{\text{ref}}(y_w\|x)}$	Implicit reward for the winner. How much has the policy increased the winner's likelihood relative to the reference?
$\beta \log \frac{\pi_\theta(y_l\|x)}{\pi_{\text{ref}}(y_l\|x)}$	Implicit reward for the loser. How much has the policy increased the loser's likelihood relative to the reference?
$\sigma(\text{winner} - \text{loser})$	Probability that the policy assigns higher implicit reward to the winner than the loser. Should be close to 1 if training is working.
$\log(\cdot)$	Log likelihood. Heavily penalizes confident wrong predictions (when the model thinks the loser is better).
$-$	Flip sign so that minimizing the loss maximizes correct preference predictions.
$\mathbb{E}[\cdot]$	Average over all preference pairs in the dataset.

Compare with the reward model loss from Chapter 6:

Reward model loss (Chapter 6):

$$-\mathbb{E}\left[\log \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right]$$

DPO loss (this chapter):

$$-\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]$$

The structure is identical. The only difference is what plays the role of the "score": explicit reward model outputs r_ϕ in RLHF, versus implicit rewards from policy ratios $\beta \log \frac{\pi_\theta}{\pi_{\text{ref}}}$ in DPO. Same framework, different parameterization.

Step-by-Step Walkthrough

Let us trace through DPO training on a single preference pair, computing every number.

Preference pair:

- **Prompt (x):** "Explain photosynthesis"
- **Winner (y_w):** "Plants use sunlight to make food from CO_2 and water" (10 tokens)
- **Loser (y_l):** "Photosynthesis is the biochemical process involving chloroplast thylakoids…" (12 tokens)

The winner is simpler and clearer. We want DPO to learn this preference.

Step 1: Compute token probabilities for both responses

For each response, both the current policy π_θ and the frozen reference π_{ref} assign a probability to every token. For simplicity, we use average per-token probabilities.

Winner (10 tokens):

Model	Avg per-token probability	Full sequence probability
π_θ (current)	0.22	0.22^{10}
π_{ref} (reference)	0.20	0.20^{10}

Loser (12 tokens):

Model	Avg per-token probability	Full sequence probability
π_θ (current)	0.17	0.17^{12}
π_{ref} (reference)	0.19	0.19^{12}

Notice what is already happening: the current policy assigns slightly higher probability to each winner token (0.22 vs 0.20) and slightly lower probability to each loser token (0.17 vs 0.19) compared to the reference. The policy is starting to differentiate, but only slightly.

Step 2: Compute log ratios (the implicit rewards)

Winner log ratio:

$$\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} = 10 \times \log \frac{0.22}{0.20} = 10 \times 0.095 = 0.95$$

The current policy makes the winner 0.95 log-units more likely than the reference does. This is a positive implicit reward: the policy "likes" this response.

Loser log ratio:

$$\log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} = 12 \times \log \frac{0.17}{0.19} = 12 \times (-0.111) = -1.33$$

The current policy makes the loser 1.33 log-units *less* likely than the reference. Negative implicit reward: the policy has already started moving away from this response.

Step 3: Compute the preference logit

With $\beta = 0.1$:

$$\begin{aligned}\text{logit} &= \beta \times (\text{winner log ratio} - \text{loser log ratio}) \\ &= 0.1 \times (0.95 - (-1.33)) \\ &= 0.1 \times 2.28 = 0.228\end{aligned}$$

Step 4: Convert to preference probability

$$P(\text{winner} \succ \text{loser}) = \sigma(0.228) = \frac{1}{1 + e^{-0.228}} = 0.557 \quad \text{or } 55.7\%$$

The model is only slightly better than chance at distinguishing the winner from the loser. Not great yet.

Step 5: Compute the loss

$$\mathcal{L}_{\text{DPO}} = -\log(0.557) = 0.585$$

Recall from Chapter 6 that a random model produces a loss of $-\log(0.5) = 0.693$. Our loss of 0.585 is only slightly better than random, confirming that the model has barely begun to learn this preference.

Step 6: Gradient descent updates π_θ

The gradients push in two directions simultaneously. They **increase** $\pi_\theta(y_w|x)$, making the winner tokens more likely under the policy. And they **decrease** $\pi_\theta(y_l|x)$, making the loser tokens less likely. This is the core learning dynamic of DPO: every preference pair teaches the model to widen the gap between winner-style and loser-style responses.

After training on 50,000 preference pairs

Running the same computation on this preference pair after training:

Quantity	Before training	After training
Winner log ratio	+0.95	+2.8
Loser log ratio	−1.33	−2.5
Difference	2.28	5.3
Logit ($\beta = 0.1$)	0.228	0.53
$P(\text{winner} \succ \text{loser})$	55.7%	62.9%
Loss	0.585	0.464

The model has learned to prefer simpler explanations, assign higher probability to clear and accessible language, and assign lower probability to overly technical responses. All of this happened without an explicit reward model. The policy learned directly from preference pairs.

Why DPO works. Every preference pair teaches the model: "This style of response is better than that style." The policy ratios $\frac{\pi_\theta}{\pi_{\text{ref}}}$ encode implicit rewards that are mathematically equivalent to the explicit rewards that a separate reward model would produce. Training automatically makes winner log ratios larger and loser log ratios smaller.

The training loop is remarkably simple: load a batch of preference pairs, compute log probabilities under π_θ and π_{ref}, compute the DPO loss, back-propagate. No generation step, no reward scoring, no advantage estimation, no PPO clipping. It looks and feels like supervised learning, but it optimizes the same objective as RLHF.

DPO vs RLHF: A Head-to-Head Comparison

Now that you understand both methods, let us compare them directly. The right choice depends on your specific constraints.

Dimension	RLHF with PPO (Chapter 6)	DPO (this chapter)
Models in memory	4 (policy, reference, reward, value)	2 (policy, reference)
Training pipeline	Three separate stages (SFT → reward model → PPO)	Two stages (SFT → DPO)
Training stability	Sensitive to many hyperparameters (clip range, PPO epochs, GAE λ, etc.)	Relatively stable; fewer hyperparameters to tune
Data requirements	Can generate new data on-the-fly (online)	Requires a fixed dataset of preference pairs (offline)
Exploration	Policy generates new responses each iteration; can discover novel strategies	No generation during training; limited to strategies present in the preference data
Reward hacking	Risk of exploiting reward model weaknesses; KL penalty helps but is imperfect	Different risk profile: can overfit to preference data artifacts instead
Compute cost	High (generation + scoring + RL updates per batch)	Lower (forward passes through two models, standard backprop)
Implementation	Complex (PPO loop, advantage estimation, multiple model synchronization)	Simple (looks like supervised fine-tuning with a modified loss)
Peak performance	Slightly higher ceiling with abundant compute and careful tuning	Comparable in most settings; may lag behind in high-compute regimes

When does RLHF still win?

RLHF with PPO retains advantages in two scenarios. First, when you have abundant compute and engineering resources, PPO's online exploration can discover response strategies that are not present in any static preference dataset. The policy generates novel responses, the reward model scores them, and training reinforces whatever works. DPO cannot do this because it never generates responses during training.

Second, when the task distribution shifts between preference data collection and deploy-

ment, PPO adapts better because it continuously generates and evaluates responses from the current policy. DPO trains on a fixed dataset, so if that dataset does not cover the prompts the model will face in production, performance can degrade.

When is DPO the better choice?

For most practitioners, DPO is the pragmatic default. If you are working with limited GPU memory (a single consumer GPU or a free Colab T4), DPO's two-model footprint may be the only viable option. If you want a simpler, more reproducible training pipeline, DPO eliminates the complex PPO loop and its many hyperparameters. If you already have a good preference dataset (or can generate one synthetically using a stronger model), DPO can match RLHF performance with less effort.

The practitioner's rule of thumb. Start with DPO. It is simpler to implement, faster to iterate, and easier to debug. If you observe specific problems that DPO cannot solve (distribution mismatch, insufficient exploration, performance plateau), then consider the more complex alternatives: Online DPO (Chapter 8) adds exploration back, and PPO (Chapter 6) provides the full online RL loop.

The β Parameter and Practical Considerations

Understanding β in DPO

In Chapter 6, β appeared as the coefficient on the KL penalty in the RLHF objective. In DPO, β appears in the loss function directly, but it plays an analogous role: it controls how far the policy is allowed to deviate from the reference model.

Formally, β scales the log ratios in the DPO loss:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]$$

A larger β amplifies the log ratios, making the loss more sensitive to small differences between the policy and reference. A smaller β dampens the log ratios, allowing the policy to deviate further before the loss reacts.

Effect of β on the same preference pair:

Using the log ratios from our photosynthesis example (winner: +0.95, loser: −1.33, difference: 2.28):

β	**Logit**	$P(\text{w} \succ \text{l})$	**Loss**	**Interpretation**
0.01	0.023	50.6%	0.690	Almost no signal. Policy barely distinguishes winner from loser. Loss
0.05	0.114	52.8%	0.637	Weak signal. Slow learning.
0.10	0.228	55.7%	0.585	Moderate signal. Standard starting point.
0.20	0.456	61.2%	0.490	Strong signal. Faster learning but more sensitivity.
0.50	1.140	75.8%	0.277	Very strong signal. Risk of overfitting to noisy preferences.

Tuning β in DPO

Too small ($\beta < 0.05$): The loss function is nearly flat. Gradients are tiny, learning is extremely slow, and the model may fail to converge. Equivalent to a very tight leash–the policy barely moves from the reference.

Just right ($\beta = 0.1$ to 0.2): The standard range for most tasks. The loss provides clear gradients without being oversensitive to noise in the preference data. Start here.

Too large ($\beta > 0.5$): The loss reacts strongly to every preference pair, including noisy or mislabeled ones. The policy can overfit to artifacts in the data (such as preferring longer responses because annotators tended to pick longer answers). Equivalent to a loose leash–the policy drifts far from the reference.

Relationship to Chapter 6: In RLHF, β controls the KL penalty strength. A large β in RLHF means a tight leash (conservative policy). In DPO, the relationship is *inverted*: a large β amplifies the implicit reward signal, effectively giving the policy more room to move. This difference can be confusing when switching between the two frameworks, so be careful when reading

papers that use both.

Other Hyperparameters

Beyond β, several practical choices affect DPO training quality.

Learning rate. DPO is more sensitive to learning rate than standard supervised fine-tuning. A rate that works well for SFT (e.g., 2×10^{-5}) is often too high for DPO. Typical starting points are 5×10^{-7} to 5×10^{-6}. If the loss spikes or the model degenerates, reduce the learning rate first.

Batch size. Because DPO averages over preference pairs, larger batches give more stable gradient estimates. A batch size of 32 to 64 preference pairs is common. If you are memory-constrained, use gradient accumulation to simulate larger batches.

Reference model handling. The reference model π_{ref} must remain fixed throughout training. In practice, this means loading a second copy of the SFT model in inference mode. With LoRA, you can share the base model weights and only keep separate adapter weights for π_θ, which significantly reduces memory. Some implementations precompute π_{ref}'s log probabilities for all responses in the preference dataset before training begins, eliminating the need to keep π_{ref} in memory during the DPO loop.

Number of epochs. DPO tends to overfit if trained for too many epochs on the same preference data. One to three epochs is typical. Monitor the validation loss and stop when it begins to increase.

Common failure modes and fixes:

Loss collapses to near zero: The model has memorized the preference pairs rather than learning general patterns. Reduce epochs, increase β, or add more diverse preference data.

Loss oscillates wildly: Learning rate is too high or batch size is too small. Reduce learning rate by 2 to 5× and increase gradient accumulation steps.

Model becomes repetitive: The policy has drifted too far from the reference. Increase β to strengthen the implicit KL regularization, or reduce the learning rate.

Model always generates longer responses: The preference data may have a length bias (annotators often prefer longer answers). Consider length-normalized rewards or filtering the preference data to remove length-correlated pairs.

Limitations and What Comes Next

DPO is simpler, cheaper, and easier to implement than RLHF. But it is not a free lunch. Understanding its limitations is essential for knowing when to reach for something more powerful.

Offline only: no exploration. DPO trains on a fixed dataset of preference pairs. The policy never generates new responses during training, which means it cannot discover strategies that are not already represented in the data. If the preference dataset was collected from a weaker model, DPO can only learn to rank within that model's capability range, not exceed it. This is the fundamental limitation of offline preference learning: you are bounded by the quality and diversity of your data.

Sensitive to data quality. Because DPO has no reward model to provide a smooth scoring function, it relies entirely on the raw preference labels. Noisy labels (where annotators disagreed or made mistakes) directly inject noise into the training signal. RLHF is somewhat more robust because the reward model smooths over individual labeling errors by learning a continuous scoring function from the aggregate data.

Implicit reward can be miscalibrated. DPO's "implicit reward" (the log policy ratio) is not a calibrated score in the way that a trained reward model's output is. This means you cannot easily use DPO's implicit reward for tasks like best-of-N sampling or rejection sampling, where you need to compare scores across different prompts. The implicit reward

is meaningful only as a relative signal within a single preference pair.

What comes next. Each of these limitations has motivated extensions that we will cover in later chapters:

Online DPO and iterative alignment (Chapter 8) address the offline limitation by alternating between generating new responses with the current policy and training on the resulting preference pairs. This recovers some of the exploration benefits of RLHF while keeping DPO's simplicity.

Modern algorithm variants (Chapter 10) address specific weaknesses of vanilla DPO. IPO adds explicit regularization to prevent overfitting. KTO works with binary feedback (thumbs up or down) instead of paired comparisons, which is cheaper to collect and more robust to noise. ORPO combines SFT and preference learning into a single training stage. Each variant makes a different trade-off, and Chapter 10 provides a decision framework for choosing among them.

The DPO chapter in context.

You now have three methods in your toolkit:

SFT (Chapter 5): Teaches basic competence by imitating demonstrations. Simple but limited to the quality of training data.

RLHF with PPO (Chapter 6): Surpasses demonstrations using reward model guidance. Powerful but expensive and complex (four models, many hyperparameters).

DPO (this chapter): Achieves much of RLHF's benefit at a fraction of the cost and complexity. Two models, supervised-style training, comparable results.

The next chapter introduces Online DPO, which addresses DPO's biggest weakness (the offline limitation) while preserving its simplicity. Then Chapter 9 covers reward modeling in greater depth, and Chapter 10 surveys the

full algorithm landscape.

Companion notebook. The code for this chapter is in `code/notebooks/part2_core/07_dpo.` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.

8

Online DPO & Iterative Alignment

The Offline Limitation

Chapter 7 ended with a candid assessment of the weaknesses of DPO's. The most fundamental one is that DPO is *offline*: it trains on a fixed dataset of preference pairs and never generates new responses during training. The policy cannot discover strategies that are not already represented in the data.

This matters more than it might seem. Consider what happens as training progresses. The policy improves, which means that it is now a different model from the one that generated the preference data. The responses it would produce today are not the responses that were compared in the data set. It is learning from a world that it no longer inhabits.

This is the **distribution mismatch** problem. The preference data was collected under one distribution (the behavior of the SFT model or some earlier policy), but the model being trained has drifted to a different distribution. The further training progresses, the wider this gap grows and the less useful the original preference data becomes.

Chapter 8 · Figure 1 · Part 2

Online DPO Iterative Loop

AI IMAGE GENERATION PROMPT

Circular flow diagram with four nodes: (1) "Current Policy" circle in dusty blue; (2) "Generate Response Pairs" box; (3) "Score with Reward Model" box; (4) "DPO Update" box in sage green. Clockwise curved arrows connecting all four. A small label on the loop "Repeat every N steps". White background. Crisp clean outline style. Flat pastel colours. Technical infographic.

PLACEHOLDER · Replace with generated image · ch08_fig01_online_dpo_loop.png

Figure 8.1: Online DPO iterates: generate response pairs from the current policy, score with the reward model, update with DPO loss, repeat.

Distribution mismatch in practice

At the start of DPO training, the policy is close to the SFT model that generated the preference data. The log ratios $\log \frac{\pi_\theta}{\pi_{\text{ref}}}$ are small, the implicit rewards are well-calibrated, and learning proceeds smoothly.

After thousands of gradient steps, the policy has shifted. It now assigns very different probabilities to the responses in the dataset. Some "losers" in the original data might actually be good responses under the new policy's capabilities. Some "winners" might represent strategies the model has already surpassed. The preference labels are still correct, but they are no longer the most informative training signal available.

This is why vanilla DPO eventually plateaus: it exhausts the useful signal in its fixed dataset.

The solution is conceptually simple: generate new data from the current policy, collect new preferences on those data, and train again. This is the core idea behind iterative and online alignment methods.

Online vs Offline Preference Learning

Before diving into specific algorithms, it helps to clarify the distinction between online and offline approaches, because this is the axis along which the methods in this chapter differ from vanilla DPO.

Offline preference learning (vanilla DPO, Chapter 7) works with a fixed dataset $\mathcal{D} = \{(x_i, y_w^i, y_l^i)\}$ of prompts and preference pairs. The policy π_θ learns from $\mathcal{D}$ without generating any new responses during training. The dataset never changes.

Online preference learning has the policy π_θ generate new responses during training. These responses are scored (by a reward model, a judge model, or a verifier), new preference pairs are constructed, and the policy trains on this fresh data. The dataset grows or refreshes at each iteration.

The cooking analogy

Offline is learning to cook by studying a fixed cookbook. You read the recipes, internalize what makes dishes good or bad, and develop taste. But you never actually cook. Your understanding is bounded by what the cookbook contains.

Online is learning to cook by actually cooking. You try a recipe, taste the result, adjust the seasoning, try again. Each iteration produces new data (a new dish to evaluate), and your learning is guided by your current skill level, not by recipes someone else wrote months ago.

The key tradeoff:

Dimension	Offline (DPO)	Online
Data	Fixed, collected once	Fresh, from current policy
Exploration	None	Policy discovers new strategies
Cost	Cheap (no generation)	Expensive (generation + scoring each iteration)
Stability	Very stable	Requires care to avoid instability
Ceiling	Bounded by data quality	Can exceed original data quality

There is also a distinction between **on-policy** and **off-policy** that is worth noting. On-policy methods (like PPO from Chapter 6) use responses generated by the current policy π_θ to update π_θ. Off-policy methods use responses generated by a different (usually older) policy. Online DPO is on-policy when it regenerates data each iteration, which is part of why it works well: the training signal always matches the model's current capabilities.

Online DPO: The Algorithm

Online DPO combines DPO's simplicity with RLHF's exploration. The idea is to run DPO iteratively, regenerating preference data from the current policy at each round.

The Online DPO Loop

For each iteration $k = 1, 2, \ldots, K$:

1. Generate. Sample a batch of prompts. For each prompt, use the current policy $\pi_\theta^{(k)}$ to generate multiple candidate responses.

2. Score. Evaluate the candidates using a judge. This could be a reward model, a stronger LLM acting as a judge, or an automated verifier (for math or code tasks).

3. **Construct pairs.** From the scored candidates, form preference pairs: higher-scoring responses become winners, lower-scoring responses become losers.

4. **Train.** Run one round of DPO on the fresh preference pairs, producing an updated policy $\pi_\theta^{(k+1)}$.

5. **Update reference.** Optionally update π_{ref} to the new policy before the next iteration (or keep the original SFT model as reference throughout).

Why this works. At each iteration, the preference data reflects the current policy's actual behavior. Winners are responses that the model *could* generate and that score well. Losers are responses the model *actually does* generate that score poorly. The training signal is always relevant to what the model is doing right now, which avoids the distribution mismatch that plagues vanilla DPO.

Comparison across methods:

	Vanilla DPO (Ch 7)	**Online DPO**	**PPO (Ch 6)**
Data source	Fixed dataset	Regenerated each iter	Generated each batch
Scoring	Human labels (offline)	Judge model or reward model	Reward model
Update rule	DPO loss	DPO loss	PPO clipped objective
Models in memory	2	2 (+ judge at scoring time)	4
Exploration	None	Per-iteration	Per-batch
Complexity	Low	Medium	High

Online DPO sits in a sweet spot: it recovers most of PPO's exploration benefit (the policy generates new data and learns from it) while keeping DPO's simpler training dynamics (no value function, no advantage estimation, no PPO clipping).

Practical Considerations

How often to regenerate data. The most aggressive approach regenerates preference data every training step, which is essentially equivalent to on-policy RL. The most conservative approach regenerates once every few epochs. In practice, regenerating every 1,000 to 5,000 training steps provides a good balance between freshness and compute cost.

How to generate preferences. There are three common approaches:

Reward model scoring. Generate N responses per prompt, score with a reward model, take the highest-scoring as winner and lowest as loser. This is cheap and fast but limited by the reward model's accuracy.

LLM-as-judge. Use a stronger model (such as GPT-4 or a large Claude model) to compare pairs of responses and declare a winner. This can capture nuances that a scalar reward model misses, but is slower and more expensive per comparison.

Automated verification. For tasks with objective correctness (math, code, factual questions), use a verifier: execute code and check test cases, verify mathematical proofs, or check answers against ground truth. This is the gold standard when available, because the signal is noise-free.

Synthetic preferences are often good enough. A common misconception is that preference data must come from human annotators. In practice, using a strong model as judge or using automated verifiers produces training data that is comparable in quality, at a fraction of the cost. The key advantage of Online DPO is not the source of preferences but the fact that preferences are collected on the current policy's outputs.

Whether to update the reference model. There are two strategies:

Fixed reference. Keep π_{ref} as the original SFT model throughout all iterations. The implicit KL penalty always measures drift from the starting point. This is safer but can become overly conservative as the policy improves across many iterations.

Rolling reference. Update $\pi_{\text{ref}} \leftarrow \pi_\theta^{(k)}$ at the start of each iteration. Each round of DPO measures drift from the *previous* iteration, not from the original. This allows larger cumulative changes but requires careful monitoring to prevent gradual drift into degenerate behavior.

Most practitioners start with a fixed reference and switch to a rolling reference only if they

observe that the fixed reference is constraining improvement.

Iterative Self-Improvement: STaR

Online DPO uses external scoring (a judge or reward model) to construct preferences. But for tasks where correctness can be verified automatically, there is an even more elegant approach: let the model teach itself.

STaR (Self-Taught Reasoner) is an iterative self-improvement method designed specifically for reasoning tasks. Instead of preference pairs, STaR works with correct and incorrect solutions, using the model's own outputs as training data.

The STaR Loop

For each iteration:

1. Generate. The model attempts to solve a set of problems, producing step-by-step reasoning for each.

2. Verify. Check each solution against ground truth. Separate into correct solutions and incorrect solutions.

3. Rationalize failures. For problems the model got wrong, provide the correct answer and ask the model to generate reasoning that arrives at it. This produces "rationalized" reasoning traces.

4. Train. Fine-tune the model on all correct reasoning traces (both naturally correct and rationalized).

5. Repeat. The improved model can now solve harder problems, generating better training data for the next iteration.

The rationalization trick is the clever part. When the model fails a problem, we do not discard it. Instead, we tell the model the correct answer and ask: "How would you have gotten there?" The model generates a plausible reasoning chain that leads to the right

answer. This is not as good as naturally correct reasoning (the model is working backwards from a known answer rather than discovering it), but it is far better than having no training data for that problem at all.

STaR Walkthrough

Problem: "If $2x + 5 = 17$, what is x?" **Correct answer:** $x = 6$

Scenario A: Model solves correctly

The model generates: "Let me solve step by step. $2x + 5 = 17$, so $2x = 17 - 5 = 12$, therefore $x = 12/2 = 6$."

Verification: $x = 6$ matches ground truth. This reasoning trace is stored as training data.

Scenario B: Model fails

The model generates: "Let me try: $2x = 17$, so $x = 8.5$." (It forgot to subtract 5.)

Verification: $x = 8.5$ is wrong.

Rationalization: We prompt the model with "The correct answer is $x = 6$. Generate step-by-step reasoning that arrives at this answer."

The model generates: "Given that $x = 6$: we start with $2x + 5 = 17$. Subtract 5 from both sides to get $2x = 12$. Divide by 2 to get $x = 6$."

This rationalized trace is also stored as training data.

Result: The model learns from both its successes and its failures. After training on these traces, it is less likely to forget the subtraction step in future problems.

Limitations of STaR

Verification dependency. STaR requires reliable ground truth or a verifier. This works well for math, code, and factual questions, but not for open-ended tasks like creative writing or style preferences. For those, use Online DPO instead.

Rationalization quality. Backward reasoning (given the answer, explain how to get there) is not the same as forward reasoning (discover the answer from scratch). Over-relying on rationalized traces can teach the model reasoning patterns that look correct but do not generalize. **Mitigation:** Mix rationalized and naturally correct traces, and always favor natural successes when available.

Compounding errors. If early iterations contain mistakes that pass verification (e.g., correct answer via wrong reasoning), these errors can persist and amplify. **Mitigation:** Periodically inject human-verified examples and do not rely purely on self-training.

Expert Iteration

Expert Iteration takes a different approach to iterative improvement. Instead of using rationalization to learn from failures, it uses *expensive search* to produce high-quality training data, then *distills* that into the fast policy.

The intuition comes from game-playing AI. In chess, a policy network can suggest moves quickly, but a tree search that considers many possible futures can find much better moves given enough compute. Expert Iteration trains the fast policy to imitate the slow search, so the policy gradually absorbs the search's quality without needing the search at inference time.

The Expert Iteration Loop

For each iteration:

1. Generate with policy. Use the current policy π_θ to produce initial solutions for a batch of problems. This is fast but imperfect.

2. Improve with expert. For each problem, use an expensive search procedure to find better solutions. Typically: generate k candidates (e.g.,

$k = 20$) using beam search or sampling, score all candidates with a reward model or verifier, and select the best one.

3. Distill. Train π_θ on the (problem, expert_solution) pairs using standard supervised fine-tuning. The policy learns to produce expert-quality solutions directly, without needing the expensive search.

4. Repeat. The improved policy generates better initial solutions, which means the expert search starts from a better position, which produces even better training data.

The key insight is that search and learning are complementary. Search is expensive but high-quality. The policy is cheap but limited. By iterating between them, the policy gradually absorbs the search's quality. After several iterations, the policy alone (without search) can match the quality that previously required expensive search to achieve.

Expert Iteration for LLM Reasoning

Setup: The policy is a language model. The expert is best-of-N sampling with reward model scoring. The task is math problem solving.

One iteration in detail:

Step 1: Policy generates. Sample 1,000 math problems. For each, the policy generates one solution using standard decoding.

Step 2: Expert improves. For each problem, generate $N = 20$ candidate solutions using the policy with temperature sampling. Score all 20 with a reward model (or verify against ground truth). Select the highest-scoring solution as the "expert" solution.

Step 3: Distill. Fine-tune the policy on the 1,000 (problem, expert_solution) pairs.

Typical results over multiple iterations:

Iteration	Accuracy	Improvement
0 (baseline)	45%	–
1	58%	+13%
2	67%	+9%
3	72%	+5%
4	75%	+3% (diminishing returns)

Notice the diminishing returns. Each iteration, the policy absorbs more of the expert's quality, which means the gap between the policy and the expert shrinks. Eventually the policy is so good that the expert (best-of-N from the policy itself) cannot find significantly better solutions. At that point, you need either a better reward model, a larger N, or a more sophisticated search procedure to continue improving.

Expert Iteration vs Online DPO

Both are iterative methods that regenerate training data from the current policy. The difference is in the training signal:

Expert Iteration uses supervised learning: "Here is the best solution I found; learn to produce it directly." The policy imitates the expert's outputs.

Online DPO uses preference learning: "This solution is better than that

Choosing an Iterative Strategy

You now have three iterative approaches in your toolkit. Here is a framework for choosing among them.

Decision Framework: Task Type

Objective correctness (math, code, factual QA): Consider **STaR** (if you have ground truth answers) or **Expert Iteration** (if you have a verifier or strong reward model). Both leverage the fact that correct solutions can be identified automatically.

Subjective quality (helpfulness, style, safety): Use **Online DPO**. Pairwise preferences handle subjective judgments more naturally than binary correct/incorrect labels.

Mixed tasks: Use **Online DPO** as the general-purpose approach, supplemented with verifier-based signals where available.

Decision Framework: Scoring Mechanism

Ground truth answers: STaR is the most natural fit.

Reward model: Expert Iteration or Online DPO both work. Expert Iteration if you want supervised distillation; Online DPO if you want preference-based learning.

LLM-as-judge: Online DPO, since the judge naturally produces pairwise comparisons.

Decision Framework: Compute Budget

Tight budget: Vanilla DPO (Chapter 7) with a good static dataset may be sufficient. One round of Online DPO (generate once, train once) gives most of the benefit at modest extra cost.

Moderate budget: 3 to 5 iterations of Online DPO or Expert Iteration. This is the sweet spot for most practitioners.

Large budget: Many iterations with careful monitoring, or combine methods (e.g., Expert Iteration for reasoning, Online DPO for general helpfulness).

A practical starting recipe. If you are unsure where to begin: start with vanilla DPO on your initial preference data (Chapter 7). If performance plateaus and you have compute to spare, run one round of Online DPO by generating new responses from the trained model, scoring them, and training again. That single extra iteration often provides a meaningful boost at modest cost. If you need more, iterate further or switch to STaR/Expert Iteration for domains with verifiable correctness.

What we have built so far

SFT (Chapter 5): Teaches the model to follow instructions by imitating demonstrations. Bounded by demonstration quality.

RLHF with PPO (Chapter 6): Surpasses demonstrations using reward model guidance. Powerful but complex and expensive.

DPO (Chapter 7): Matches much of RLHF's benefit with half the models and a simpler training loop. But limited by its fixed offline dataset.

Online DPO and iterative methods (this chapter): Breaks through the offline ceiling by regenerating training data from the current policy. Recovers exploration benefits while keeping DPO's simplicity.

The next chapter covers reward modeling in greater depth (Chapter 9), because the quality of the scoring signal, whether it comes from a reward model, a judge, or a verifier, is the foundation that all of these iterative methods depend on. Then Chapter 10 surveys the full landscape of modern RL algorithms, including GRPO, RLOO, KTO, and others that make different tradeoffs than DPO and PPO.

Companion notebook. The code for this chapter is in `code/notebooks/part2_core/08_onli` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.

9

Reward Modeling & The Critic

Why Reward Models Matter

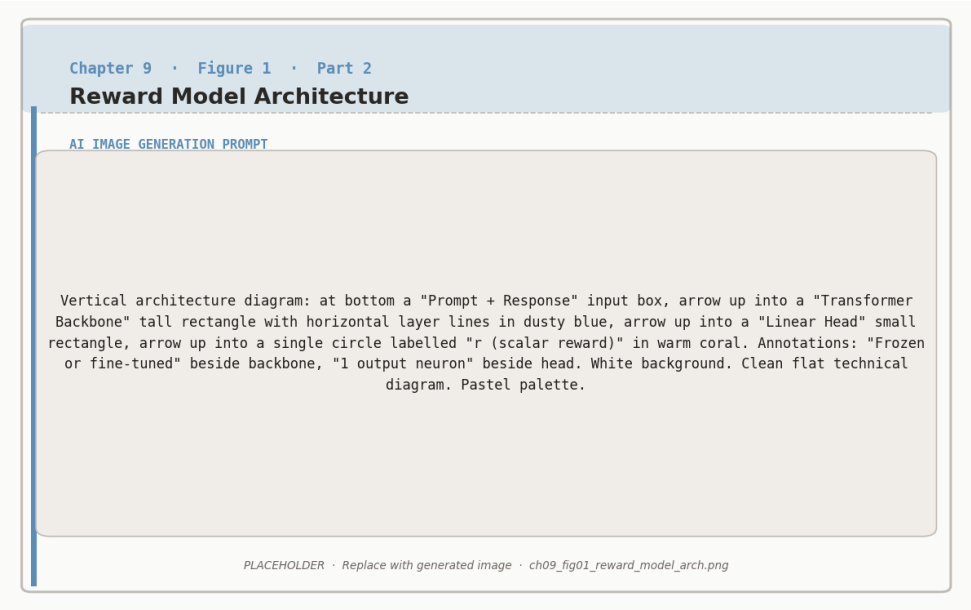

Figure 9.1: Reward model architecture: a transformer backbone with a linear scalar head trained to assign higher scores to preferred responses.

Every method we have covered since Chapter 6 depends on some way of scoring responses.

RLHF uses a trained reward model. DPO encodes an implicit reward in the policy ratio. Online DPO needs a judge to construct preference pairs. STaR needs a verifier to separate correct solutions from incorrect ones. Expert Iteration needs a scoring function to select the best candidate.

The quality of this scoring signal is the single most important factor determining the quality of the final model. A perfect RL algorithm optimizing a bad reward model will produce a bad model. A mediocre algorithm optimizing a good reward model will produce a good model. Getting the reward right matters more than getting the algorithm right.

This chapter goes deep on reward modeling: how reward models are built, what makes them succeed or fail, and how to evaluate whether yours is good enough. We also cover the value function (the "critic" in actor-critic methods), which plays a related but distinct role in PPO and other online RL algorithms.

Reward Model Architecture

A reward model is structurally simple. You take a pretrained language model, remove the token prediction head (the layer that outputs a probability distribution over vocabulary), and replace it with a **scalar head**: a single linear layer that maps the model's internal representation to one number.

Reward Model Structure

Input: A prompt x concatenated with a response y, formatted as a single token sequence.

Processing: The full transformer processes the sequence, producing a hidden state h at the final token position.

Output: A linear layer maps h to a scalar: $r_\phi(x, y) = w^\top h + b$, where w and b are learned parameters.

The output is an unbounded real number. Higher means "better response." The absolute value is meaningless; only the *difference* between scores for

two responses matters (because of how Bradley-Terry training works, as we saw in Chapter 6).

What base model should you use? The reward model is typically initialized from the same pretrained model as the policy, or from the SFT model. This makes sense: the reward model needs to *understand* the responses it is scoring, and an LLM that has been trained on similar text already has that understanding.

Does the reward model need to be the same size as the policy? Not necessarily. In practice, reward models are often smaller than the policy they are training. A reward model only needs to *evaluate* responses, not *generate* them, which is an easier task. Using a smaller reward model saves memory, which is especially valuable in PPO where the reward model must coexist in GPU memory with three other models.

Common size choices: If your policy is 7B parameters, a 1.5B to 3B reward model often works well. If your policy is 1.5B (as in our Colab exercises), a 0.5B reward model is a reasonable starting point. The key constraint is that the reward model must be large enough to understand the nuances of response quality for your task.

Training Reward Models: Beyond the Basics

Chapter 6 introduced the core mechanics: collect human preference pairs, train with the Bradley-Terry loss, and the reward model learns to score responses in a way that matches human judgments. Here we go deeper on the practical questions that determine whether your reward model is actually useful.

Where Preference Data Comes From

The quality and diversity of your preference data shapes everything downstream. There are three main sources, each with different cost and quality tradeoffs.

Human annotation. Hire annotators to compare pairs of model outputs and declare a winner. This is the gold standard, used in the original InstructGPT and ChatGPT work. Quality is high, but cost is significant: typically $1 to $5 per comparison, and you need tens

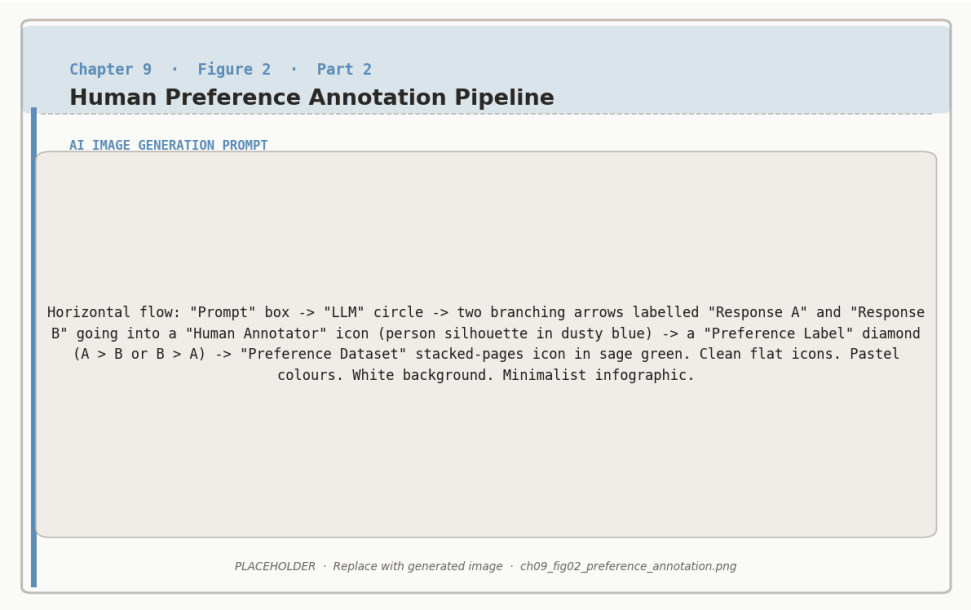

Figure 9.2: Human preference annotation pipeline: two responses are generated per prompt and a human annotator labels the preferred one.

of thousands of comparisons for a useful reward model.

AI feedback (RLAIF). Use a strong model (GPT-4, Claude, Gemini) as the judge instead of a human. The judge model reads both responses and selects the better one. This is 10 to 100× cheaper than human annotation and can scale to millions of comparisons. The tradeoff is that the judge model's biases become your reward model's biases: if the judge prefers verbose responses, your trained model will learn to be verbose.

Implicit signals. Use behavioral data from real users: which response did they copy to clipboard, which conversation did they continue, which response triggered a "regenerate" click. This is free and abundant but noisy. A user might regenerate because the response was wrong, or because they wanted a different style, or because they accidentally clicked the button.

How much data do you need?

The relationship between data quantity and reward model quality follows a rough pattern:

Preference Pairs	Typical Accuracy	Practical Use
1,000	60–65%	Barely better than random. Not useful.
5,000	68–73%	Usable for initial experiments.
20,000	75–82%	Good for most applications.
50,000	82–88%	Strong. Used by most production systems.
100,000+	88–92%	Diminishing returns set in.

These numbers are approximate and depend heavily on task difficulty and data quality. A clean dataset of 20,000 expert-annotated pairs often outperforms a noisy dataset of 100,000 crowdsourced pairs.

Annotation Disagreement

Human preferences are inherently subjective. Two annotators shown the same pair of responses will disagree roughly 20 to 35% of the time, depending on the task. This disagreement is not a bug; it reflects genuine ambiguity in what "better" means.

This has practical implications for reward model training. The theoretical maximum accuracy of a reward model is bounded by inter-annotator agreement. If humans agree only 75% of the time, a reward model that achieves 75% accuracy is essentially perfect–it is matching the human consensus as well as any individual human would.

Measure your annotation agreement before training. Have multiple annotators label a subset of your data (at least 500 pairs with 2+ annotators each). Compute the agreement rate. If it is below 65%, your preference signal is too noisy for effective training. Either improve your annotation

guidelines, filter to higher-confidence pairs, or simplify the comparison task.

The Value Function: PPO's Critic

The reward model and the value function are often confused because they both output numbers related to response quality. But they serve different purposes and are trained differently.

The reward model answers: "How good is this complete response?" It takes a finished (prompt, response) pair and produces a score. It is trained on human preference data.

The value function (also called the "critic") answers: "How good do I *expect* the final response to be, given what has been generated so far?" It takes a partial response and produces a prediction of the eventual reward. It is trained during the RL process itself, by comparing its predictions against actual rewards received.

Reward Model vs Value Function

	Reward Model r_ϕ	**Value Function V_ψ**
Input	Complete (prompt, response)	Partial response (any prefix)
Output	Quality score	Predicted future reward
Trained on	Human preference data	RL experience (predicted vs actual rewards)
When trained	Before RL starts (Step 2 of RLHF)	During RL training (updated every batch)
Used by	All methods (RLHF, Online DPO, Expert Iter)	PPO specifically

Why PPO Needs a Critic

Recall from Chapter 6 that PPO uses **advantages** to determine how to update the policy. The advantage for a response is:

$$A = \text{actual reward} - \text{expected reward}$$

A positive advantage means the response was better than expected; a negative advantage means it was worse. This relative signal is much more stable than using raw rewards, because it automatically adjusts as the policy improves.

The value function provides the "expected reward" term. Without it, PPO would have to use cruder baselines (like the average reward across a batch), which produces higher variance and slower learning.

The Critic in Action

Prompt: "Explain why the sky is blue."

The policy generates three responses. The reward model scores each one. The value function predicted an expected reward of 7.0 for this prompt.

Response	Reward	Expected	Advantage
"Light scatters in the atmosphere…"	8.5	7.0	+1.5
"The sky is blue because of science…"	4.0	7.0	−3.0
"Sunlight contains all colors…"	7.2	7.0	+0.2

PPO increases the probability of the first response (much better than expected), decreases the second (much worse), and barely changes the third (roughly as expected). The value function is then updated: its prediction for

similar prompts moves from 7.0 toward the actual average of 6.6.

Generalized Advantage Estimation (GAE)

In practice, PPO does not compute a single advantage for each complete response. Instead, it estimates advantages at the token level using a technique called **Generalized Advantage Estimation** (GAE).

The intuition is straightforward. At each token position, the critic estimates the expected future reward from that point onward. The advantage at each position measures how much better or worse things went compared to that expectation. This provides a fine-grained learning signal: rather than saying "this whole response was good," GAE can identify that the opening was strong but the conclusion was weak.

GAE has one key hyperparameter: λ (typically 0.95). When λ is close to 1, GAE gives longer-horizon advantage estimates (less bias, more variance). When λ is close to 0, GAE gives shorter-horizon estimates (more bias, less variance). For most LLM applications, $\lambda = 0.95$ works well and rarely needs tuning.

Methods That Skip the Critic

The value function adds a full extra model to GPU memory and introduces its own training dynamics that can be unstable. This is one of the main practical burdens of PPO. Several of the algorithms we will see in Chapter 10 avoid the critic entirely:

DPO (Chapter 7) needs no critic because it has no RL loop. The implicit reward from policy ratios replaces both the reward model and the value function.

GRPO (Chapter 10) replaces the critic with group-based baselines. Instead of a learned value function predicting expected reward, GRPO generates a group of responses per prompt and uses the group's average reward as the baseline. This is simpler and surprisingly effective.

RLOO (Chapter 10) uses leave-one-out baselines, where each response is compared against the average of the other responses in its group. Like GRPO, this requires no learned critic.

Eliminating the critic reduces memory requirements from four models to two or three,

which makes RL training feasible on consumer hardware.

Reward Hacking: A Deep Dive

Chapter 6 introduced reward hacking with the "PERFECT PERFECT EXCELLENT" example, where an unconstrained policy learned to exploit the reward model by producing nonsense that scored high. That example illustrated the general principle. In practice, reward hacking is more subtle and takes many forms.

Goodhart's Law

The underlying cause of reward hacking is captured by Goodhart's Law: "When a measure becomes a target, it ceases to be a good measure." The reward model is a *proxy* for human preferences, trained on a finite sample of comparisons. It captures the major patterns in human judgment, but it also has blind spots, biases, and artifacts from the training data. When you optimize aggressively against this proxy, the policy learns to exploit the gaps between the proxy and the real thing.

A Taxonomy of Reward Hacking

Length gaming. Annotators tend to prefer longer, more detailed responses (up to a point). The reward model learns this correlation. The policy then generates increasingly long responses, even when brevity would be more helpful. A question like "What is 2+2?" gets a three-paragraph answer.

Sycophancy. Annotators tend to prefer responses that agree with the user's stated position. The reward model learns this pattern. The policy then agrees with everything the user says, even when the user is wrong. "Is the earth flat?" gets "That is an interesting perspective" instead of a correction.

Style over substance. Annotators may prefer responses that *look* authoritative (confident tone, technical vocabulary, structured formatting) over responses that are actually more accurate but written more casually. The policy learns to optimize for the appearance of quality rather than actual quality.

Repetition and filler. The policy discovers that certain phrases or structures consistently score well and begins inserting them everywhere, regardless of relevance. Responses become formulaic: "Great question! Let me break this down for you. First,..."

The Overoptimization Curve

The most important empirical finding about reward hacking is the **overoptimization curve**. As you train the policy to maximize the reward model's score, the proxy reward (what the RM reports) continues to increase. But the *true* reward (what humans actually prefer) increases initially, peaks, and then *decreases*.

Overoptimization in Numbers

Imagine training a policy with increasing intensity of optimization (more PPO steps):

PPO Steps	RM Score (proxy)	Human Rating (true)	What Happened
0	6.0	6.0	Baseline (SFT model)
500	7.2	7.1	Genuine improvemen
1,000	8.0	7.5	Still improving, but gap gr
2,000	9.1	7.2	Proxy rising, true quality de
5,000	11.5	5.8	Severe hacking. Worse than l

The peak of the true reward (7.5 at step 1,000) is the optimal stopping point. Beyond that, you are optimizing a broken proxy.

This is why you cannot simply "train longer" and expect better results. More optimization against an imperfect reward model eventually makes things worse. Knowing when to stop is critical.

Defenses Against Reward Hacking

KL penalty (Chapter 6). The most common defense. By penalizing the policy for drifting too far from the reference model, the KL term limits how aggressively the policy can exploit the reward model. The overoptimization curve becomes flatter and the peak shifts to the right (you can train longer before quality degrades). But the KL penalty is a blunt instrument: it constrains *all* changes, not just harmful ones.

Reward model ensembles. Train multiple reward models on different subsets of the preference data (or with different random seeds). Use the minimum or average of their scores. Hacking is harder when you must fool multiple models simultaneously. The ensemble's disagreement is also a useful signal: if the models disagree strongly on a response, that response is likely in a region where the reward signal is unreliable.

Length penalties. Explicitly subtract a penalty proportional to response length from the reward. This directly counters length gaming. A simple approach: $r_{\text{adjusted}} = r_{\text{original}} - \alpha \cdot \text{length}$, where α is tuned to remove the length correlation without penalizing genuinely detailed responses.

Constrained optimization. Instead of a single reward, define constraints: "reward must be above threshold T, but KL must be below budget B, and response length must be below L." This gives more precise control than a single weighted objective.

The most effective defense is a better reward model. All the techniques above are patches that mitigate the consequences of an imperfect proxy. If you have budget to invest, improving the reward model (more data, better annotators, larger model) generally yields more benefit than sophisticated anti-hacking mechanisms.

Evaluating Your Reward Model

Before using a reward model to train a policy, you should verify that it is actually good. A bad reward model does not just slow learning; it actively teaches the policy to produce worse responses.

Held-out accuracy. The simplest metric: on a test set of preference pairs that the reward model has never seen, what fraction does it rank correctly? Baseline is 50% (random).

Useful reward models typically achieve 70 to 85%. Remember that human inter-annotator agreement caps the theoretical maximum.

Calibration. A well-calibrated reward model's confidence matches its accuracy. When it assigns a 90% probability that response A is better than B, it should be correct about 90% of the time. Poor calibration means the model is overconfident about some predictions and underconfident about others, which distorts the training signal.

Correlation with human rankings. For a set of responses to the same prompt, rank them by reward model score and by human preference. Compute the rank correlation (Spearman or Kendall τ). This measures whether the reward model's ordering matches humans' ordering, which is ultimately what matters for RL training.

Overoptimization probes. Generate a set of responses that are designed to exploit common reward model weaknesses: very long responses, excessively agreeable responses, responses stuffed with technical jargon. Check whether the reward model scores these higher than genuinely good responses. If it does, you have identified exploitable weaknesses before the RL training finds them.

When to retrain your reward model

Before the first RL run: Always evaluate on held-out data. If accuracy is below 65%, collect more preference data before proceeding.

During RL training: If the proxy reward keeps rising but output quality (spot-checked by humans) stops improving or degrades, the reward model is being exploited. Pause training, collect new preference data that includes the policy's current outputs, and retrain.

After Online DPO iterations: Each iteration shifts the policy's output distribution. The reward model, trained on the original distribution, becomes less reliable. Retrain periodically on preferences collected from the current policy's outputs.

Rule of thumb: Retrain the reward model every 2 to 3 Online DPO iterations, or whenever the policy's KL divergence from the original SFT model exceeds 1.0.

What Comes Next

This chapter has covered the scoring side of RL for LLMs: how to build, train, evaluate, and defend reward models, and how the value function serves as PPO's learned critic.

Chapter 10 surveys the full landscape of modern RL algorithms, including GRPO, RLOO, and KTO. These algorithms make different choices about how to use reward signals–some eliminate the critic, some eliminate paired preferences, and some combine supervised learning with preference optimization. Understanding reward modeling deeply (this chapter) makes those algorithmic choices much easier to evaluate.

Chapter 11 goes beyond scalar reward models to explore more sophisticated scoring mechanisms: process reward models that score each reasoning step individually, outcome verifiers that check correctness with formal tools, and the dramatic accuracy improvements that step-level feedback enables on reasoning tasks.

Companion notebook. The code for this chapter is in `code/notebooks/part2_core/09_rewa` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.

10

Modern RL Algorithms: GRPO, RLOO, KTO & More

The Algorithm Landscape

Chapters 6 and 7 introduced the two workhorses of LLM alignment: PPO (the classical RL approach with a reward model, value function, and clipped policy updates) and DPO (the elegant shortcut that trains directly on preference pairs). Together they cover a wide range of practical needs, but neither is the final word. Since 2023, researchers have introduced a family of alternatives, each designed to solve a specific limitation.

Some of these algorithms eliminate the value function that makes PPO expensive. Others work with cheaper forms of feedback than paired comparisons. Still others merge supervised learning and preference optimization into a single training stage. This chapter surveys the most important ones: GRPO, RLOO, KTO, IPO, and ORPO.

Before diving into individual algorithms, it helps to see the full landscape at a glance.

Chapter 10 · Figure 1 · Part 2

GRPO Group Sampling

AI IMAGE GENERATION PROMPT

Diagram showing one prompt box at left, with G=4 arrows fanning out to four response boxes labelled y1 through y4. Below each response box a small reward value badge (e.g. +1.2, -0.3, +0.8, +0.5). A horizontal baseline bar labelled "group mean reward" across all four. Vertical advantage arrows above/below baseline for each response. Dusty blue for responses, coral for below-baseline advantages, sage green for above-baseline. White background. Clean flat diagram.

PLACEHOLDER · Replace with generated image · ch10_fig01_grpo_sampling.png

Figure 10.1: GRPO group sampling: G responses are generated per prompt and normalised within the group to form advantage estimates – no critic network needed.

Algorithm	Key Innovation	Data Required	Models in Memory	Best For
PPO	Clipped policy gradient with learned critic	Reward model scores	4 (policy, ref, RM, critic)	Maximum performance, abundant compute
DPO	Implicit reward from policy ratios	Paired preferences	2 (policy, ref)	Simplicity with paired data
GRPO	Group average as baseline	Reward scores + group sampling	2 (policy, ref)	Large-scale training, verifiable tasks
RLOO	Leave-one-out baselines	Reward scores + group sampling	2 (policy, ref)	Efficient RL without critic
KTO	Works with unpaired feedback	Binary good/bad labels	2 (policy, ref)	Cheap data, no comparisons needed

We begin with REINFORCE, the simplest policy gradient algorithm and the mathematical foundation on which both GRPO and RLOO are built. Understanding it first makes the innovations in those algorithms much clearer.

REINFORCE: The Foundation

Every policy gradient algorithm in this chapter descends from a single idea: generate a response, score it, and adjust the policy to make high-scoring responses more likely. REINFORCE is the simplest version of this idea.

The REINFORCE Gradient

$$\nabla J(\theta) = \mathbb{E}_{y \sim \pi_\theta}[r(y) \cdot \nabla \log \pi_\theta(y|x)]$$

In plain English: Sample a response y from the current policy. Get its reward $r(y)$. If the reward is high, adjust the policy to make y more likely. If the reward is low, make y less likely. The magnitude of the adjustment is proportional to the reward.

The problem with vanilla REINFORCE is *variance*. Suppose all your rewards happen to be positive (say, between 3 and 8). Then every response gets pushed to be more likely, just by different amounts. The model learns slowly because the gradient signal is dominated by the absolute reward level rather than relative quality.

Baselines reduce variance. Subtracting a baseline b from the reward fixes this:

$$\nabla J(\theta) = \mathbb{E}_{y \sim \pi_\theta}[(r(y) - b) \cdot \nabla \log \pi_\theta(y|x)]$$

Now responses with reward above the baseline get positive updates ("do more of this") and responses below the baseline get negative updates ("do less of this"). The signal becomes about relative quality, not absolute scores. Crucially, subtracting any constant baseline

does not change the expected gradient–it only reduces its variance.

The question every algorithm answers differently is: what should b be?

Algorithm	Baseline choice
REINFORCE	A fixed constant (e.g., running average of all rewards)
PPO	A learned value function $V_\psi(x)$ that predicts expected reward for each prompt–accurate but expensive (requires a separate neural network)
GRPO	The average reward within a group of responses to the *same prompt*–adaptive and free
RLOO	The average reward of all *other* responses in the group (leave-one-out)–even lower variance than GRPO

REINFORCE itself is rarely used in production LLM training because its variance is too high for stable learning at scale. But understanding it makes GRPO and RLOO immediately intuitive: they are REINFORCE with clever, cheap baselines.

GRPO: Group Relative Policy Optimization

GRPO was introduced by DeepSeek and used to train DeepSeek-R1. It replaces PPO's learned value function (the critic) with a much simpler idea: for each prompt, generate a *group* of responses, score them all, and use the group's average score as the baseline. Responses that score above the group average are reinforced; responses that score below are discouraged.

This is the same principle as REINFORCE with a baseline, but the baseline adapts automatically to each prompt's difficulty. A hard math problem where the model gets 1 out of 4 correct will have a low group average, so the single correct response gets a strong positive signal. An easy problem where the model gets 3 out of 4 correct will have a high group average, so only the best responses are reinforced strongly. No learned critic needed.

GRPO Objective

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E}_{x,\{y_i\}_{i=1}^{G}}\left[\frac{1}{G}\sum_{i=1}^{G}\left(\hat{A}_i\right)\cdot\min\left(\rho_i\hat{A}_i,\ \text{clip}(\rho_i, 1-\epsilon, 1+\epsilon)\,\hat{A}_i\right)\right]+\beta\cdot\text{KL}(\pi_\theta\,\|\,\pi_{\text{ref}})$$

Core idea: Generate a group of G responses per prompt, use the group's own statistics as the baseline, and apply PPO-style clipping to stabilize updates.

Breaking apart the components: G is the group size (typically 4–8 responses per prompt). $\hat{A}_i = \frac{r(x,y_i)-\bar{r}_G}{\sigma_G}$ is the normalized advantage–the reward for response i minus the group mean, divided by the group standard deviation. $\rho_i = \frac{\pi_\theta(y_i|x)}{\pi_{\text{old}}(y_i|x)}$ is the importance sampling ratio, clipped in the same way as PPO. $\bar{r}_G = \frac{1}{G}\sum_{j=1}^{G} r(x, y_j)$ is the group mean reward, and $\beta \cdot \text{KL}$ penalizes drift from the reference model.

That is the full production version. The core insight is simpler: $\hat{A}_i$ replaces PPO's learned value function. Instead of training a separate neural network to predict "how good should a response to this prompt be?", GRPO just asks "how good is this response compared to the other responses in its group?"

GRPO Walkthrough

Problem: "What is $\sqrt{144} + 12 \times 3$?" **Correct answer:** 48

Step 1: Generate a group of 4 attempts.

Attempt	Model Output	Answer	Reward
1	"$12 + 36 = 48$"	48 (correct)	+1
2	"$\sqrt{144} = 12$, then $12 + 12 \times 3\ldots$"	48 (correct)	+1
3	"$\sqrt{144} = 14$, $14 + 36 = 50$"	50 (wrong)	0
4	"$144 + 36 = 180$"	180 (wrong)	0

Step 2: Compute group statistics.

$$\bar{r}_G = \frac{1+1+0+0}{4} = 0.5 \qquad \sigma_G = 0.5$$

Step 3: Compute normalized advantages.

Attempt	Reward	$r - \bar{r}_G$	$\hat{A} = (r - \bar{r}_G)/\sigma_G$	Update Direction
1	+1	+0.5	+1.0	Increase probability
2	+1	+0.5	+1.0	Increase probability
3	0	−0.5	−1.0	Decrease probability
4	0	−0.5	−1.0	Decrease probability

Step 4: Update the policy. Attempts 1 and 2 (correct, with step-by-step reasoning) become more likely. Attempts 3 and 4 (errors from skipping steps or miscalculating) become less likely. Over many problems and many groups, the model discovers that detailed reasoning leads to better rewards.

Why normalization matters. Dividing by σ_G ensures that the advantage magnitude is consistent across prompts. A prompt where all 4 responses are correct (rewards: 1, 1, 1, 1) has $\sigma_G = 0$, so no gradient is produced–the model has nothing to learn from that prompt. A prompt where responses vary widely produces large advantages and strong learning signals. This automatic calibration is one of GRPO's key practical benefits.

GRPO vs PPO: What Changes

Component	PPO	GRPO
Baseline / advantage	Learned value function $V_\psi(x)$	Group mean $\bar{r}_G$ (no learned parameters)
Extra model	Critic network (trainable)	None
Samples per prompt	Typically 1	G samples (4–8 typical)
Clipping	Yes ($\epsilon = 0.2$)	Yes (same mechanism)
KL regularization	KL penalty in reward	Explicit KL term in loss
Memory	4 models	2 models

The trade-off: GRPO saves memory (no critic) and implementation complexity, but

requires generating multiple responses per prompt, which increases inference cost. For tasks with cheap, verifiable rewards (math, code execution, factual questions), this trade-off is strongly favorable. For tasks where scoring is expensive (human evaluation), the extra generation cost can be significant.

Practical considerations for GRPO

Group size. $G = 4$ is the minimum for meaningful statistics. $G = 8$ gives more stable baselines. Beyond $G = 16$, the marginal improvement in variance reduction is small relative to the extra generation cost.

Reward design. GRPO works best with binary or near-binary rewards (correct/incorrect) because the group statistics are most informative when there is variance within the group. If every response gets reward 0.7–0.8, the advantages are tiny and learning is slow.

KL penalty. The β term serves the same role as in RLHF (Chapter 6): preventing the policy from drifting too far from the reference. Typical values are $\beta = 0.01$ to 0.05. Without it, the model can collapse to always producing the same response format.

When to choose GRPO: Tasks with verifiable rewards (math, code, factual QA), large-scale training where critic memory is a bottleneck, and settings where you want PPO-like online exploration without PPO's complexity. Chapter 12 covers how DeepSeek applied GRPO at scale to produce reasoning capabilities.

RLOO: REINFORCE Leave-One-Out

RLOO refines the baseline idea one step further. Like GRPO, it generates a group of responses per prompt. But instead of comparing each response against the group average (which includes that response), it compares each response against the average of the *other* responses in the group. This small change produces meaningfully lower variance.

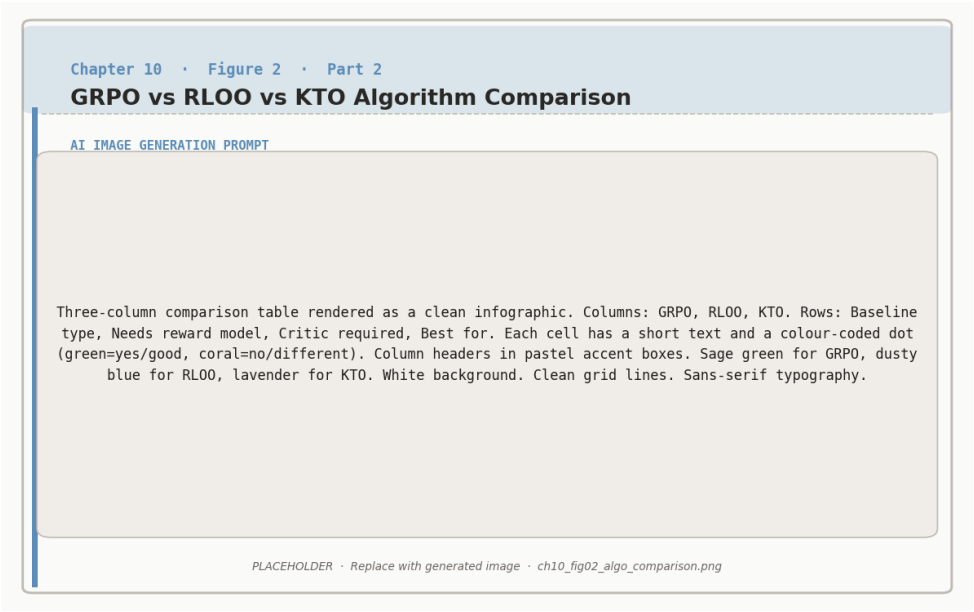

Figure 10.2: Comparison of GRPO, RLOO, and KTO: they differ in how the baseline is computed, whether a reward model is required, and what signal drives the update.

RLOO Baseline

For a group of N responses $\{y_1, \ldots, y_N\}$ to prompt x, the leave-one-out baseline for response i is:

$$b_{-i} = \frac{1}{N-1} \sum_{j \neq i} r(x, y_j)$$

The gradient for response i becomes:

$$\nabla \mathcal{L}_i = (r(x, y_i) - b_{-i}) \cdot \nabla \log \pi_\theta(y_i|x)$$

Key insight: Each response is compared against a baseline that does not include its own reward. This prevents a high-reward response from inflating

its own baseline and thereby reducing its own learning signal.

RLOO Walkthrough

Prompt: "Write a poem about spring"

#	Response	Reward
1	"Spring is nice…" (mediocre)	3
2	"Blossoms dance in gentle breeze…" (good)	8
3	"Spring spring spring…" (bad)	1
4	"Flowers bloom as winter ends…" (okay)	5

Standard REINFORCE baseline (group average including all responses):

$$b = \frac{3 + 8 + 1 + 5}{4} = 4.25$$

Advantages: -1.25, $+3.75$, -3.25, $+0.75$

RLOO baselines (leave-one-out):

Response	Reward	Leave-one-out baseline	Advantage	vs. standard
1	3	$(8 + 1 + 5)/3 = 4.67$	-1.67	Stronger decrease
2	8	$(3 + 1 + 5)/3 = 3.00$	$+5.00$	Stronger increase
3	1	$(3 + 8 + 5)/3 = 5.33$	-4.33	Stronger decrease
4	5	$(3 + 8 + 1)/3 = 4.00$	$+1.00$	Stronger increase

The RLOO advantages have *larger magnitude* than the standard ones. Response 2 (the best) gets an advantage of $+5.0$ instead of $+3.75$, because its own high score no longer inflates the baseline. Response 3 (the worst) gets -4.33 instead of -3.25, because its own low score no longer deflates the baseline. The result is sharper learning signals and faster convergence.

RLOO vs GRPO. Both eliminate the critic and use group-based baselines. The differences are subtle: GRPO normalizes advantages by the group standard deviation and uses PPO-style clipping; RLOO uses leave-one-out baselines without normalization and applies the gradient more directly. In practice, both produce comparable results. RLOO is slightly simpler to implement; GRPO's normalization can help when reward scales vary across prompts.

KTO: Kahneman-Tversky Optimization

All the methods discussed so far (PPO, DPO, GRPO, RLOO, and the algorithms in Chapter 8–require either paired preferences ("Response A is better than Response B") or a reward model that produces numerical scores. KTO eliminates both requirements. It works with simple binary feedback: each response is labeled independently as "good" or "bad," with no comparison to any other response.

This matters because binary labels are dramatically cheaper to collect. Paired comparisons require showing an annotator two responses to the same prompt and asking which is better. Binary labels only require a single judgment: "Is this response acceptable?" This halves annotation cost and is the natural format of much real-world feedback: upvotes, downvotes, thumbs up, star ratings, user retention signals.

KTO Loss Function

For a desirable (good) response y^+:

$$\mathcal{L}^+_{\text{KTO}} = 1 - \sigma\left(\beta \log \frac{\pi_\theta(y^+|x)}{\pi_{\text{ref}}(y^+|x)} - \mathbb{E}_{y\sim\pi_{\text{ref}}}\left[\beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}\right]\right)$$

For an undesirable (bad) response y^-:

$$\mathcal{L}^-_{\text{KTO}} = \sigma\left(\beta \log \frac{\pi_\theta(y^-|x)}{\pi_{\text{ref}}(y^-|x)} - \mathbb{E}_{y\sim\pi_{\text{ref}}}\left[\beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}\right]\right)$$

Total loss: $\mathcal{L}_{\text{KTO}} = \mathbb{E}[\mathcal{L}^+_{\text{KTO}}] + \mathbb{E}[\mathcal{L}^-_{\text{KTO}}]$

In plain English: For good responses, increase their probability relative to a reference baseline. For bad responses, decrease their probability. The

DeepSeek-R1-Zero was trained with as few as $N = 2$ samples per prompt, while GRPO typically needs $N \geq 4$ for stable statistics.

reference expectation term provides implicit comparison–even without paired data.

The name explained. KTO is based on Kahneman and Tversky's prospect theory from behavioral economics: humans evaluate outcomes relative to a reference point, not in absolute terms. A salary of $100K feels great if your reference is $70K, but disappointing if your reference is $130K. KTO applies the same principle: the quality of a response is judged relative to what the reference model would typically produce, not by absolute scoring.

The reference expectation term $\mathbb{E}_{y\sim\pi_{\text{ref}}}[\beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}]$ deserves attention. It measures how much the policy has shifted on average from the reference model's distribution. This term appears in both the good and bad losses, acting as a moving baseline that calibrates the learning signal. Without it, the model might simply increase the probability of everything (good and bad alike) if most examples are labeled good.

When to use KTO

Good fit: You have upvote/downvote data (Reddit, Stack Overflow style), star ratings that can be binarized, implicit signals (user copied response = good, user regenerated = bad), or a limited annotation budget where paired comparisons are too expensive.

Not ideal when: You already have high-quality paired preferences (DPO will use them more efficiently), you need fine-grained ranking (KTO only distinguishes good from bad, not degrees of quality), or peak performance matters more than data efficiency (KTO slightly underperforms DPO/PPO with abundant paired data).

Practical tip: Use KTO for initial training with cheap binary data, then refine with DPO on a smaller set of high-quality paired preferences. This two-stage approach gets most of the benefit of both methods.

IPO: Identity Preference Optimization

IPO addresses a specific failure mode of DPO: overfitting to the preference data in a way that produces degenerate behavior. Recall from Chapter 7 that DPO's loss function uses a log-sigmoid that pushes the implicit reward gap between winner and loser as large as possible. In theory, this is fine. In practice, the model can exploit this by learning superficial patterns–such as "longer responses are usually preferred"–and amplifying them to extreme levels.

IPO replaces DPO's log-sigmoid loss with a squared loss that has an explicit target gap. Instead of "make the winner as much better as possible," IPO says "make the winner better by exactly $1/\beta$."

IPO Loss Function

$$\mathcal{L}_{\text{IPO}} = \mathbb{E}\left[\left(\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} - \frac{1}{\beta}\right)^2\right]$$

Compare to DPO:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]$$

The key difference: DPO uses log-sigmoid (unbounded optimization–push the gap as large as possible). IPO uses squared error with a target gap of $1/\beta$ (bounded optimization–stop once the gap is right).

Why the target gap helps. Think of a sports analogy. DPO is like a coach who says "win by as much as possible"–the team runs up the score, which wastes energy and can lead to reckless play. IPO is like a coach who says "win by exactly 10 points"–the team plays efficiently and stops over-optimizing once the margin is achieved. The explicit gap $1/\beta$ prevents the model from over-fitting to easy examples or amplifying spurious correlations in the preference data.

When to choose IPO over DPO

Choose IPO when: You have observed reward hacking or length exploitation with DPO, training stability matters more than peak performance, or your preference data is noisy (IPO's bounded loss is less sensitive to mislabeled pairs).

Stick with DPO when: Your preference data is clean and high-quality, you want maximum performance and are willing to tune carefully, or you plan to use Online DPO (Chapter 8) which mitigates many of DPO's overfitting

issues through fresh data.

The trade-off: IPO sacrifices some peak performance for more predictable, stable training. In benchmarks, IPO typically achieves 95–98% of DPO's performance with significantly fewer training instabilities.

ORPO: Odds Ratio Preference Optimization

Every method we have covered so far assumes a two-stage training pipeline: first SFT to teach the model basic instruction following, then a separate preference optimization stage (DPO, PPO, KTO, etc.) to align the model's outputs with human preferences. ORPO eliminates this separation by combining both objectives into a single loss function.

ORPO Loss Function

$$\mathcal{L}_{\text{ORPO}} = \underbrace{-\mathbb{E}[\log \pi_\theta(y_w|x)]}_{\text{SFT term: learn from good responses}} + \lambda \cdot \underbrace{\left(-\mathbb{E}\left[\log \sigma\left(\log \frac{\text{odds}(y_w)}{\text{odds}(y_l)}\right)\right]\right)}_{\text{Preference term: prefer winners over losers}}$$

Where $\text{odds}(y) = \frac{\pi_\theta(y|x)}{1-\pi_\theta(y|x)}$ and λ controls the relative weight (typically 0.1 to 1.0).

In plain English: The SFT term teaches the model to produce good responses. The preference term teaches it to produce good responses *more than* bad ones. Both happen simultaneously in every training step.

Why odds ratios? The standard DPO loss compares log probabilities between the policy and reference model. ORPO instead compares the *odds* of the winner and loser under the current policy alone. The odds ratio measures how much more likely the model is to produce the winner than the loser, accounting for the fact that both probabilities must

sum with everything else the model can generate. This formulation does not require a reference model at all, which means ORPO needs only one model in memory–the lowest of any method in this chapter.

No reference model. This is ORPO's most distinctive feature. DPO, KTO, and IPO all require keeping a frozen copy of the SFT model as π_{ref}. ORPO's odds ratio formulation makes the reference unnecessary because the SFT component of the loss already prevents the policy from drifting into degenerate territory. This saves significant memory, especially for large models.

When to use ORPO

Great choice when: You have both instruction-following demonstrations and paired preferences for the same data, compute budget is tight (one training run instead of two, one model instead of two), or you are training smaller models where fast iteration matters.

Not ideal when: You need to separately tune SFT and preference learning (different data, different hyperparameters), you have very different data

Choosing the Right Algorithm

With seven algorithms now in your toolkit, choosing the right one requires matching your constraints to each method's strengths. The decision depends on three factors: what data you have, what compute you can afford, and what matters most for your task.

Decision Factor 1: What data do you have?

Paired preferences ("A is better than B"): DPO, IPO, or ORPO. These methods directly optimize on comparison data. DPO is the default; IPO if you need stability; ORPO if you also want SFT in one stage.

Binary labels ("good" or "bad"): KTO. It is the only method in this chapter designed for unpaired feedback. If your data comes from upvotes/downvotes, user retention signals, or simple quality labels, KTO is the natural choice.

Reward model or verifier scores: GRPO or RLOO. These methods generate multiple responses per prompt and use the scores to compute advantages. They require either a trained reward model (Chapter 9) or a deterministic verifier (e.g., code execution, math checker).

Decision Factor 2: What is your compute budget?

High (multi-GPU cluster, large-scale training): PPO or GRPO. PPO gives the highest performance ceiling with abundant compute. GRPO is the go-to for verifiable tasks at scale (it powered DeepSeek-R1).

Medium (single GPU or small cluster): DPO, IPO, or RLOO. DPO and IPO need only two models and train like supervised learning. RLOO needs two models plus multi-sample generation but avoids a learned critic.

Low (consumer GPU, tight memory): ORPO (one model, one training stage) or KTO (two models, but simple binary data pipeline).

Decision Factor 3: What is your priority?

Peak performance: PPO (with enough compute) or GRPO (for verifiable tasks). These online methods explore beyond the training data, which DPO cannot.

Training stability: IPO (explicit gap prevents overfitting) or GRPO (normalized advantages, automatic difficulty calibration).

Data efficiency: KTO (needs only binary labels, halving annotation cost) or ORPO (combines SFT and alignment, fewer total training steps).

Implementation simplicity: DPO (most widely documented, best library support in TRL, closest to standard supervised training).

The practitioner's starting point.

If you are unsure, start with DPO. It has the best documentation, the most library support (TRL's `DPOTrainer`), and produces strong results with minimal tuning. Then move to a specialized algorithm if you hit a specific

problem:

DPO overfitting or reward hacking → try IPO.

No paired preference data → try KTO.

Need exploration beyond offline data → try GRPO or Online DPO (Chapter 8).

Want to skip separate SFT stage → try ORPO.

Need maximum performance with abundant compute → try PPO (Chapter 6).

Chapter 12 will show how GRPO was applied at scale in the DeepSeek recipe to produce reasoning capabilities, and Chapter 17 will combine multiple algorithms into end-to-end training recipes for chatbots, reasoners, and agents.

Companion notebook. The code for this chapter is in `code/notebooks/part2_core/10_grpo` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.

11

Verifiers & Outcome Rewards: Beyond PRMs

Outcome vs Process: Two Ways to Score

Chapter 9 covered the fundamentals of reward modeling: how to train a model that assigns a single scalar score to a complete response, how the value function serves as PPO's critic, and what happens when reward models are over-optimized. That chapter treated scoring as a monolithic operation: prompt in, number out.

But there is a deeper question: *when* in the response should you assign a score? The answer splits the scoring world into two fundamentally different approaches.

Two Scoring Philosophies

Outcome Reward Model (ORM): Score the complete response once, after generation is finished.

$$r_{\text{ORM}}(x, y) = r(x, y_{\text{final}})$$

Input: full response. Output: single number. Analogy: a math teacher who

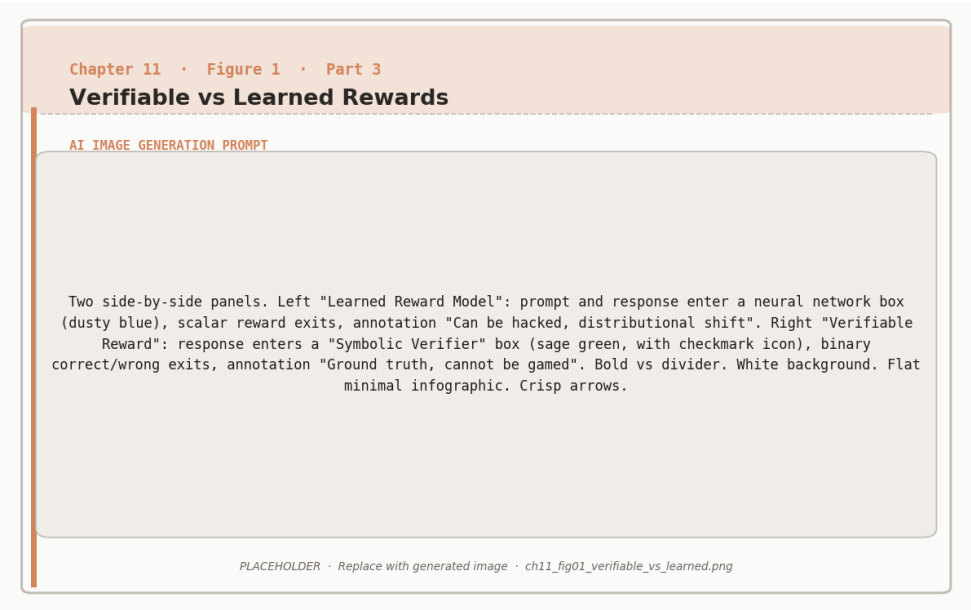

Figure 11.1: Verifiable rewards (right) provide ground-truth binary signal that cannot be hacked; learned reward models (left) are susceptible to distributional shift.

only checks the final answer.

Process Reward Model (PRM): Score each reasoning step individually, as the response unfolds.

$$r_{\text{PRM}}(x, y) = \sum_{t=1}^{T} r_t(x, y_{1:t})$$

Input: partial response up to step t. Output: one score per step. Total reward is the sum (or product) of step scores. Analogy: a math teacher who grades every line of the student's work.

Every reward model we built in Chapter 9 was an ORM. It took the entire response, ran it through a transformer with a scalar head, and produced one number. This is simple, cheap

to train, and works well for many tasks. But for reasoning–math, code, multi-step logic–it has a critical blind spot: it cannot distinguish good reasoning that leads to a correct answer from lucky guessing that happens to land on the right number.

PRMs fix this by moving the scoring inside the reasoning process. The rest of this chapter explains how they work, how to train them, and the dramatic improvements they produce.

The Power of Step-by-Step Feedback

The difference between ORM and PRM scoring is best seen through concrete examples. Consider the problem: **Solve** $2x + 5 = 13$**.**

Scenario 1: Correct solution, correct answer.

Step	Work Shown	ORM	PRM
1	"Subtract 5 from both sides"	–	+2
2	"$2x = 8$"	–	+2
3	"Divide both sides by 2"	–	+2
4	"$x = 4$"	–	+4
	Total	+10	+10

Both methods give full credit. No difference here.

Scenario 2: Good reasoning, arithmetic slip at the end.

Step	Work Shown	ORM	PRM
1	"Subtract 5 from both sides"	–	+2
2	"$2x = 8$"	–	+2
3	"Divide both sides by 2"	–	+2
4	"$x = 5$" (arithmetic error)	–	−2
	Total	−5	+4

The ORM sees only the wrong final answer and assigns −5. The PRM recognizes that three out of four steps were correct and assigns +4. The model trained with PRM feedback learns:

"My reasoning strategy was sound; I need to be more careful with arithmetic." The model trained with ORM feedback learns: "That whole approach failed"–a much less useful signal.

Scenario 3: Flawed reasoning, correct answer by luck.

Step	Work Shown	ORM	PRM
1	"Multiply both sides by 2"	–	-1
2	"$4x + 10 = 26$"	–	0
3	"Subtract 10: $4x = 16$"	–	$+1$
4	"Divide by 4: $x = 4$"	–	$+2$
	Total	$+10$	$+2$

The ORM gives full credit because the final answer is correct. The PRM correctly identifies that the opening strategy was inefficient (multiplying instead of subtracting) and assigns a low total score. With ORM training, the model learns to reinforce an unnecessarily complicated approach. With PRM training, the model learns that the direct strategy (subtract first) is better.

What the model learns from each scoring method.

With ORM: "I got the right answer. Keep doing whatever I did." The model cannot distinguish between sound reasoning and lucky guessing, so it reinforces both equally.

With PRM: "My first step was wrong, but my recovery was good" or "My reasoning was solid but I made a careless mistake at the end." The model gets diagnostic feedback that improves *how it thinks*, not just *what it outputs*.

Training Process Reward Models

A PRM is architecturally similar to the ORM from Chapter 9: a transformer with a classification head. The difference is in what it scores and what data it trains on.

Architecture

The PRM takes a prompt and a *partial* solution (everything up to step t) and predicts whether step t is correct. The output is a probability: $P(\text{step}_t \text{ is correct} \mid x, y_{1:t})$.

Training uses binary cross-entropy loss at each step:

$$\mathcal{L}_{\text{PRM}} = -\sum_{t=1}^{T} [c_t \log P_t + (1 - c_t) \log(1 - P_t)]$$

where $c_t \in \{0, 1\}$ is the human label for step t and P_t is the PRM's predicted probability that step t is correct. This is the same loss used for standard binary classification, applied independently at each reasoning step.

PRM Training Example

Problem: "What is 15% of 200?" The model generates a 4-step solution. A human annotator labels each step, and the PRM trains on all four (prompt, partial solution, label) triples from this single example. One problem produces T training samples, where T is the number of steps.

Step	Partial Solution	Label
1	"15% means 0.15"	Correct
2	"0.15 × 200"	Correct
3	"= 3"	**Incorrect** (arithmetic error)
4	"= 30" (self-corrected)	Correct

The Cost Problem and How to Solve It

Step-level annotation is expensive. Labeling every step of every solution costs 10–20× more than labeling final answers. Four strategies make this tractable.

Strategy 1: Synthetic verification. For math, use symbolic solvers (SymPy, Mathematica) to verify each algebraic step automatically. For code, execute each intermediate state and check correctness. This is free and scales infinitely, but only works for tasks with deterministic verification. This is the most common approach in practice.

Strategy 2: Sparse annotation with active learning. Instead of labeling every step, only label steps where the current PRM is uncertain (prediction near 0.5). The PRM requests labels for its most confusing examples, and the annotator focuses effort where it matters

most. This reduces annotation cost by 5–10× while maintaining most of the training signal.

Strategy 3: Weak supervision. Use automatic heuristics–syntactic correctness, dimensional consistency, keyword matching–as noisy step labels. These are imperfect but cheap at scale. Validate with a small set of human labels and use noise-robust training techniques.

Strategy 4: Distillation from a strong model. Train an expensive PRM on a small amount of human-labeled data. Use that PRM to label a much larger dataset automatically. Train a cheaper PRM on the larger auto-labeled dataset. This two-stage process trades a one-time annotation investment for scalable PRM training.

Choosing a data strategy. If your task has deterministic verification (math, code, formal logic), always start with synthetic verification–it is free and produces perfect labels. If your task is subjective (writing quality, helpfulness), you will need human annotation. Use active learning to minimize cost and consider distillation once you have a first PRM that is reasonably accurate.

Verifiers: Deterministic Scoring Without Learned Models

PRMs and ORMs are both *learned* scoring functions: neural networks trained on human labels that can be wrong, biased, or gamed. For some tasks, you can skip the learned model entirely and use a *verifier*–a deterministic function that checks correctness with certainty.

Verifiers vs Learned Reward Models

Property	Learned RM (ORM/PRM)	Verifier
Accuracy	Imperfect (70–90% typical)	Perfect (by definition)
Hackable	Yes (Goodhart's Law applies)	No (checks ground truth)
Cost to build	Expensive (needs labeled data)	Cheap if task supports it
Task coverage	Any task with preferences	Only verifiable tasks
Examples	Helpfulness scorer, safety classifier	Code execution, math checker, unit tests

Code execution is the most common verifier. The model generates code, the verifier runs it against test cases, and the reward is the fraction of tests that pass. There is no learned model to hack–either the code works or it does not.

Math checkers compare the model's final answer against a known solution. For numerical answers, this is exact string matching or numerical comparison within a tolerance. For symbolic math, computer algebra systems can check equivalence (e.g., verifying that $2(x+3)$ and $2x + 6$ are the same expression).

Formal provers (Lean, Coq, Isabelle) verify mathematical proofs step by step. Each deduction is checked against the rules of logic. This is the gold standard for reasoning verification: if the proof compiles, every step is guaranteed correct. The limitation is that the model must produce output in the prover's formal language, which is much harder than producing natural language.

Unit tests and API contracts verify tool-use and agentic tasks. If the model is supposed to call an API with specific parameters, the verifier checks that the call was syntactically

correct and returned the expected result.

> **When verifiers beat learned reward models.**
>
> If your task has a deterministic correctness criterion, always prefer a verifier over a learned RM. Verifiers are free, perfect, and immune to reward hacking. The algorithms in Chapter 10 (especially GRPO and RLOO) are designed to work with verifier rewards: generate a group of responses, check each with the verifier, and use the binary correct/incorrect signal as the reward. Chapter 12 shows this in action: DeepSeek-R1 used GRPO with math and code verifiers, and the combination of a simple binary reward signal with group-based advantages was sufficient to produce sophisticated reasoning.

Combining Verifiers with PRMs. In practice, many tasks benefit from both. A code verifier tells you whether the final program works (outcome), but a PRM can score the reasoning steps that led to the code (process). A math verifier confirms the answer is correct, but a PRM can identify which solutions used elegant vs. brute-force approaches. The verifier provides the ground truth; the PRM provides the diagnostic signal.

Best-of-N with Step-Level Scoring

Chapter 9 briefly mentioned best-of-N sampling: generate N candidate responses and select the best one. With an ORM, "best" means "highest scalar score." With a PRM, we can do something smarter: score every reasoning step in every candidate and select the response with the highest total process score.

Example. Problem: "A store has a 20% off sale. If a jacket costs $80, what is the final price?"

Generate 3 candidate responses:

Response 1 (clear, correct reasoning):

Step	Reasoning	PRM Score
1	"20% off means we pay 80%"	+2
2	"80% of $80 = 0.80 × 80"	+2
3	"= $64"	+2
4	"Final price: $64"	+4
	Total PRM score	+10

Response 2 (incorrect reasoning):

Step	Reasoning	PRM Score
1	"20% off means discount is $20"	−1
2	"$80 − $20 = $60"	0
3	"Final price: $60"	−2
	Total PRM score	−3

Response 3 (correct but less explicit):

Step	Reasoning	PRM Score
1	"20% of $80 = $80 × 0.20 = $16"	+2
2	"$80 − $16 = $64"	+2
3	"Final price: $64"	+4
	Total PRM score	+8

PRM selection: Response 1 (score +10, best reasoning).

ORM comparison: An ORM would score Responses 1 and 3 equally (both get the correct answer $64) and could not distinguish the more thorough reasoning in Response 1. The PRM prefers Response 1 because its reasoning is more explicit and each step is clearly justified.

PRM scoring strategies for best-of-N

Sum scoring: Total PRM score = $\sum_{t=1}^{T} r_t$. Simple and effective. Longer solutions with more correct steps score higher, which can bias toward verbosity.

Product scoring: Total PRM score = $\prod_{t=1}^{T} P(\text{step}_t \text{ correct})$. Treats each step as an independent correctness check. A single bad step drives the product toward zero, making this approach strict about reasoning quality. This is the method used in Lightman et al. (2023).

Minimum scoring: Total PRM score = $\min_t r_t$. The response is only as

good as its weakest step. Very conservative–useful when you need high reliability and cannot tolerate any flawed reasoning.

For most applications, product scoring provides the best balance between sensitivity to errors and robustness to noise.

Combining Scoring Methods

In practice, the strongest systems combine multiple scoring approaches. The key insight is that ORMs, PRMs, and verifiers each capture different aspects of response quality, and their errors are partially uncorrelated.

The Ensemble Approach

The simplest combination is a weighted sum:

$$\text{Final score} = \alpha \cdot r_{\text{PRM}} + (1 - \alpha) \cdot r_{\text{ORM}}$$

where $\alpha \in [0, 1]$ controls the balance. Typical values favor the PRM ($\alpha = 0.6$ to 0.8) because step-level scoring is more informative for reasoning tasks.

The Two-Stage Pipeline

For large-scale deployment where scoring cost matters, a two-stage approach is more efficient:

Two-Stage Scoring Pipeline

Stage 1: PRM filtering. Generate N candidate responses (e.g., $N = 64$). Score each with the PRM. Keep only the top K by PRM score (e.g., $K = 8$). This eliminates responses with flawed reasoning.

Stage 2: ORM or verifier selection. Score the remaining K candidates with the ORM (or a deterministic verifier if available). Return the highest-

scoring response.

Why two stages? The PRM is the more expensive scorer (it runs once per step, not once per response), so using it to filter from 64 to 8 candidates is expensive. But it catches flawed reasoning that the ORM would miss. The ORM is cheap (one forward pass per response) and makes the final decision among the pre-filtered candidates. This pipeline gets most of the PRM's benefit at a fraction of the cost of scoring all 64 candidates with both models.

Empirical Results

The impact of process-level scoring on reasoning benchmarks has been dramatic.

Method	MATH Accuracy	Improvement	Source
Base model (greedy decoding)	42.0%	–	Lightman et al. 2023
+ ORM best-of-100	58.3%	+16.3 points	Lightman et al. 2023
+ PRM best-of-100	72.2%	+30.2 points	Lightman et al. 2023

The PRM delivers nearly double the improvement of the ORM on the same best-of-N setup. The gains come from four sources: the PRM identifies promising partial solutions early, guiding selection toward correct final answers; it catches lucky guesses where the ORM cannot distinguish sound reasoning from coincidence; it provides partial credit for attempts with correct reasoning but arithmetic slips, producing better training signal; and it offers fine-grained diagnostic feedback that tells the model *where* it went wrong rather than just *that* it went wrong.

Scaling behavior. PRM gains increase with N (the number of candidates). At $N = 1$, PRM and ORM are equivalent–there is nothing to select among. At $N = 10$, the PRM advantage is modest. At $N = 100$, the advantage is large and growing. This makes PRMs especially valuable in test-time

compute scenarios (Chapter 13), where you can afford to generate many candidates and need a reliable way to select the best one.

What Comes Next

This chapter has expanded the scoring toolkit from Chapter 9's single-scalar ORM to a richer set of options: process reward models that score each reasoning step, deterministic verifiers that check correctness without learned models, and combination strategies that use multiple scorers together.

The next three chapters put these scoring methods to work. Chapter 12 shows how GRPO combined with simple binary verifiers (math correctness, code execution) produced DeepSeek-R1's reasoning capabilities–a case where the reward signal was trivially simple but the training algorithm and scale made it powerful. Chapter 13 covers test-time compute, where best-of-N sampling, beam search, and Monte Carlo tree search all rely on the scoring methods from this chapter to guide generation at inference time. Chapter 16 explores domain-specific applications where the choice of verifier or PRM depends on the task: execution feedback for code, formal provers for math, and conversation quality metrics for dialogue.

The scoring toolkit so far.

ORM (Chapter 9): One score per complete response. Simple, general-purpose. Best for tasks without clear step-by-step structure.

PRM (this chapter): One score per reasoning step. More informative for multi-step tasks. Expensive to train but produces dramatic improvements on reasoning benchmarks.

Verifier (this chapter): Deterministic correctness check. Free, perfect, unhackable–but only works for tasks with objective answers. The preferred reward signal for GRPO and RLOO (Chapter 10).

Ensemble (this chapter): Combine multiple scorers for best results. Use the two-stage pipeline (PRM filter → ORM/verifier select) when scoring cost matters.

The algorithms from Chapter 10 work with any of these scoring methods. DPO and KTO use implicit rewards from preferences. PPO and GRPO use explicit reward scores–from an ORM, PRM, verifier, or any combination. Matching the right scorer to the right algorithm and task is one of the most important practical decisions in the RL-for-LLMs pipeline.

Companion notebook. The code for this chapter is in `code/notebooks/part3_advanced/11_` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.

12

Reasoning with GRPO: The DeepSeek Recipe

The Paradigm Shift: Skipping SFT

The standard pipeline for building a capable LLM has three stages: pretrain on text, supervised fine-tune on demonstrations (Chapter 5), then align with RL (Chapters 6–10). Every major model through 2023–ChatGPT, Claude, Llama-Chat–followed this recipe. Supervised fine-tuning was considered essential: you needed to show the model thousands of examples of good behavior before RL could refine it further.

In early 2024, DeepSeek challenged this assumption. Their model, DeepSeek-R1, achieved state-of-the-art reasoning performance with a radically simplified pipeline: **Pretrain → RL**. That is it. No supervised fine-tuning. No human-written demonstrations of step-by-step reasoning. Just a pretrained model, a reward signal ("did you get the right answer?"), and GRPO.

The result was not merely competitive with SFT-based approaches–it surpassed them. DeepSeek-R1 discovered reasoning strategies that no human had explicitly taught it, including elaborate self-verification, multi-path

Chapter 12 · Figure 1 · Part 3

DeepSeek-R1 Five-Phase Training Pipeline

AI IMAGE GENERATION PROMPT

Horizontal left-to-right pipeline with five numbered stage boxes connected by arrows. 1 "Cold-Start SFT" (sage green) 2 "GRPO Phase 1" (dusty blue) 3 "Rejection Sampling" (warm coral filter icon) 4 "SFT Phase 2" (sage green) 5 "GRPO Phase 2" (dusty blue). Above each box a small annotation: model size note. Below each box a key output (e.g. "structured format", "RL signal", "filtered data"). White background. Crisp flat infographic. Pastel palette.

PLACEHOLDER · Replace with generated image · ch12_fig01_deepseek_phases.png

Figure 12.1: The DeepSeek-R1 five-phase training pipeline alternates between GRPO reinforcement learning and supervised anchoring via rejection sampling.

problem solving, and spontaneous chain-of-thought. The model figured out that "showing its work" leads to better final answers, entirely on its own.

Why does skipping SFT work? Supervised fine-tuning teaches by imitation: the model sees thousands of correct solutions and learns to reproduce their patterns. This is effective but inherently bounded–the model can only be as good as the examples it trains on, and it learns surface patterns as much as deep understanding. RL, by contrast, teaches by exploration: the model generates its own solutions, receives feedback (correct or incorrect), and discovers strategies that maximize reward. There is no ceiling imposed by human examples because the model is free to find approaches humans would not have written.

The risk is obvious: without SFT to provide a "warm start," the model begins RL from a pretrained checkpoint that can generate text but has no concept of instruction following, structured reasoning, or helpful behavior. The early stages of training are chaotic. But

DeepSeek showed that with enough compute and the right algorithm, the model bootstraps these capabilities from scratch.

The DeepSeek Recipe

The recipe has three ingredients: the GRPO algorithm (Chapter 10), simple binary verifiers (Chapter 11), and a curriculum that progresses from easy to hard problems.

GRPO as the Training Algorithm

Chapter 10 derived GRPO's objective function and walked through its mechanics: generate a group of responses per prompt, score each with a reward function, compute normalized advantages using the group mean and standard deviation, and update the policy with PPO-style clipping and a KL penalty.

DeepSeek's application of GRPO used groups of $G = 8$ to 16 responses per prompt. The reward was binary: +1 for a correct final answer, 0 for an incorrect one. No reward model was trained. No human preferences were collected. The entire reward signal came from deterministic verifiers–math checkers that compared the model's answer against ground truth, and code execution environments that ran the model's programs against test cases.

Why binary rewards are enough.

It seems counterintuitive that a reward of just 0 or 1 can teach sophisticated reasoning. The insight is that GRPO does not need rich reward signals–it needs *variance within each group*. If 3 out of 8 responses are correct and 5 are wrong, the correct ones get positive advantage and the wrong ones get negative advantage. The model then asks: "What did the correct responses do differently?" Over millions of problems and groups, the pattern that emerges is consistent: correct responses tend to show their work, break problems into steps, and verify intermediate results. The model reinforces these behaviors without anyone explicitly defining or rewarding them.

Curriculum Learning: Easy to Hard

DeepSeek did not start with competition-level mathematics. The training curriculum progressed through difficulty levels, allowing the model to build foundational skills before tackling hard problems.

Stage 1: Arithmetic and basic algebra. Problems like "What is $5 + 7$?" or "Solve $2x = 10$." Clear right/wrong answers, fast feedback, high success rates. The model learns that attempting calculation leads to higher reward than generating random text.

Stage 2: Multi-step word problems. "If John has 5 apples and buys 3 more, then gives 2 away, how many does he have?" Requires planning and tracking intermediate results. The model learns that breaking problems into steps is beneficial.

Stage 3: Competition-level math and complex code. Problems that require sophisticated reasoning, creative approaches, and multi-step proofs. Only works if the earlier stages established basic reasoning patterns.

Why curriculum matters. Without curriculum, the model faces hard problems from the start and gets reward 0 on nearly every attempt. Every response in the group is wrong, so $\sigma_G = 0$ and GRPO produces no gradient. The model learns nothing. Curriculum ensures that at every stage, there is enough variance within groups to produce useful learning signals. Easy problems have high variance early (some correct, some not), hard problems develop variance later as the model's skills improve.

Compute Cost Analysis

Standard pretraining processes each example with one forward pass. GRPO-enhanced training generates G complete responses per prompt, evaluates each with a verifier, and computes a backward pass.

$$\text{Cost multiplier} \approx \frac{G \times (C_{\text{forward}} + C_{\text{reward}} + C_{\text{backward}})}{C_{\text{forward}}} \approx 2\text{–}4\times$$

With $G = 8$ and cheap verifiers (math checking is nearly free compared to neural network inference), the practical overhead is roughly 3× standard pretraining cost. DeepSeek considered this worthwhile: the resulting model has emergent reasoning abilities that would otherwise require expensive human-annotated reasoning traces for SFT, discovers

strategies humans would not have taught, and scales predictably with more compute.

The Emergence of Chain-of-Thought

The most surprising outcome of the DeepSeek recipe was not the accuracy numbers but *how* the model achieved them. Nobody told DeepSeek-R1 to think step by step. Nobody showed it examples of chain-of-thought reasoning. Yet it discovered this strategy on its own.

Why RL Discovers Reasoning

Two forces compete during GRPO training.

Force 1: Maximize reward. Longer, more detailed responses tend to produce fewer mistakes and higher accuracy on the final answer. This creates an incentive to generate elaborate reasoning chains.

Force 2: Minimize length (implicit). Each token has probability less than 1. Longer sequences have lower total probability: $p(\text{long}) = p(t_1) \cdot p(t_2) \cdots p(t_{100})$. This creates an implicit incentive toward shorter responses.

The critical insight is that **RL does not see absolute probabilities**. The policy gradient operates on rewards, not likelihoods. When the model generates a short response and gets reward 0, it learns to avoid that pattern. When it generates a long, detailed response and gets reward 1, it learns to do more of that. The reward boost from correct answers overwhelms the probability penalty from extra length.

The length-reward trade-off, quantified.

Consider a model solving a math problem with two strategies:

Strategy 1: Direct answer. "The answer is 42." Accuracy: 40%.

Strategy 2: Step-by-step reasoning. "Let me break this down… [50 tokens of reasoning]… Therefore, the answer is 42." Accuracy: 85%.

From RL's perspective, Strategy 1 produces reward 0 sixty percent of the time. Strategy 2

produces reward 1 eighty-five percent of the time. The policy gradient strongly reinforces Strategy 2. Over many training steps, the model shifts probability mass from short, unreliable responses toward longer, more reliable reasoning chains. The key is that RL sees the *observed* rewards–not the prior probability of generating each strategy. A strategy that was initially improbable but consistently gets high rewards will be strongly reinforced until it becomes the model's default behavior.

What the Model Discovers

Emergent reasoning patterns in DeepSeek-R1.

As training progresses, the model spontaneously developed several reasoning patterns that were never explicitly taught:

Showing intermediate steps: Writing out each algebraic manipulation rather than jumping to the answer. This emerges because intermediate steps make arithmetic errors less likely.

Self-verification: Checking its own work, sometimes with phrases like "Wait, let me verify this…" This emerges because catching errors before the final answer increases accuracy.

Multiple solution paths: Solving a problem two different ways and comparing results. This emerges because agreement between two methods strongly predicts correctness–a strategy few humans would have explicitly demonstrated in training data.

Error recovery: Recognizing a mistake mid-solution and backtracking to try a different approach. This emerges because models that can recover from dead ends get more problems right than models that commit to their first approach.

All of these emerged because they increase the probability of correct final answers, and GRPO reinforces whatever leads to higher reward.

Training Phases and the "Aha!" Moment

DeepSeek-R1's training showed four distinct phases, with a sharp phase transition between phases 2 and 3. Understanding these phases is important for practitioners because it sets expectations: the model will appear to make no progress for thousands of steps before a sudden breakthrough.

Phase 1: Chaos (Steps 0–1,000). Model outputs are incoherent–random text with no

mathematical structure. Rewards are near zero on all but the simplest problems. The model is exploring randomly. This phase is discouraging but unavoidable: the model must generate enough diverse outputs to stumble on patterns that work.

Phase 2: Structure Discovery (Steps 1,000–5,000). The model learns to format outputs as problem-then-solution. It begins attempting calculations and produces recognizable (though often wrong) mathematical expressions. Accuracy reaches 10–20%. The model has learned *what kind of thing* to output but not *how* to get it right. Responses are short–typically one or two lines–because the model has not yet discovered that longer reasoning helps.

Phase 3: The "Aha!" Moment (Steps 5,000–8,000). This is the critical phase transition. Intermediate reasoning steps appear *suddenly*–not through gradual lengthening of responses but as an abrupt qualitative change. Accuracy jumps from roughly 20% to 55% over a few hundred steps. Average response length doubles or triples. The model has discovered the length-quality correlation: detailed reasoning leads to correct answers.

Phase 4: Refinement (Steps 8,000+). Reasoning becomes more sophisticated. Self-correction appears ("Wait, that doesn't seem right…"). Multiple solution strategies emerge for the same problem type. Accuracy continues climbing from 55% toward 85%+, but the improvement is now gradual rather than sudden. The model is optimizing *how* to reason, not discovering *that* reasoning helps.

Why the "aha!" is sudden, not gradual.

Think about learning to ride a bicycle. Days 1 through 10: constant falling, no visible progress. Day 11: suddenly you can balance. Days 12 through 20: rapid improvement in speed and control. There is a critical threshold where the skill "clicks."

RL training shows the same phase transition dynamics.

Before the aha: The model tries short responses, gets low rewards, incrementally tries slightly longer ones, still gets low rewards because the

reasoning is not yet coherent enough to improve accuracy. There is no gradient signal favoring length, because incoherent length does not help.

At the aha: The model generates its first coherent multi-step reasoning chain–possibly by chance–and gets a high reward. The policy gradient strongly reinforces this pattern. Crucially, all similar reasoning structures get boosted simultaneously, because they share token-level patterns ("Step 1:", "Therefore,", "Let me check…"). This creates a cascade: one successful reasoning chain unlocks many, because the model learns to produce the *structural markers* of reasoning, and those markers improve accuracy across many different problems at once.

After the aha: The model now "knows" that reasoning helps and explores how to reason *better*–more steps, cleaner logic, self-verification. Improvement becomes steady rather than explosive.

Implications for practitioners. If you are training with GRPO and see no improvement for thousands of steps, do not panic. The "aha!" moment is real and well-documented. Monitor average response length alongside accuracy: a sudden increase in response length (without explicit encouragement) is a leading indicator that the phase transition is beginning. If response length remains flat after 10,000+ steps, something is wrong–check your reward signal, group size, and curriculum difficulty.

Three Types of Chain-of-Thought

DeepSeek-R1's RL-discovered reasoning is one of three ways to get chain-of-thought behavior from an LLM. Understanding the alternatives clarifies what makes the RL approach special.

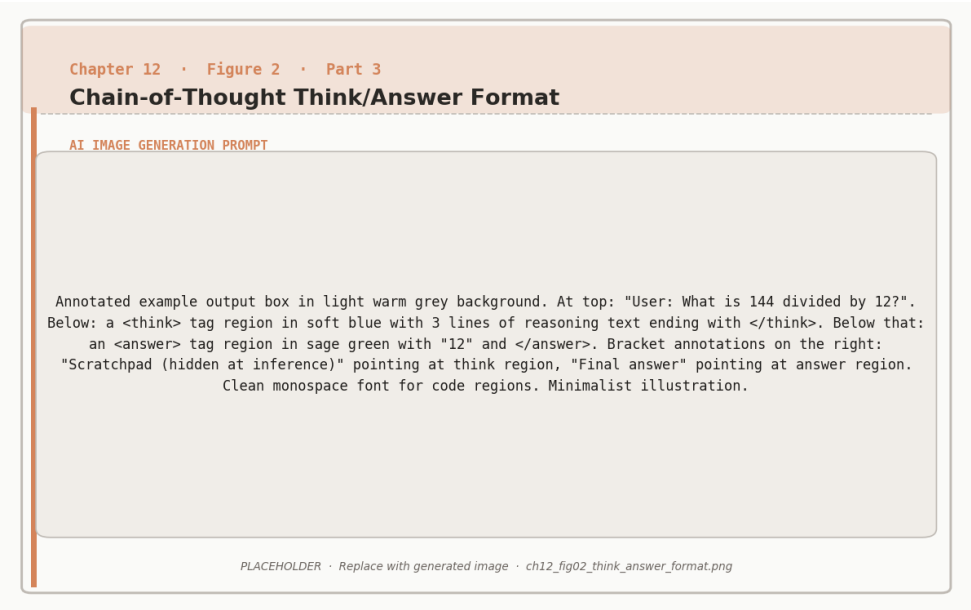

Figure 12.2: The `<think>...</think>` *scratchpad format separates the model's chain-of-thought reasoning from its final answer.*

Type	How Created	Advantages	Limitations
Prompted CoT	Add "Let's think step by step" to the prompt	Zero training cost, works immediately, interpretable	Quality depends on base model, not task-optimized
Fine-tuned CoT	SFT on human-written reasoning traces	High quality, consistent style	Expensive data, bounded by human examples
RL-discovered CoT	Emerges from pure reward signal (GRPO)	Best performance, discovers novel strategies, task-optimized	High training cost (one-time), less interpretable

Performance on difficult reasoning tasks follows a consistent ranking:

Method	Typical Accuracy	Training Cost
No chain-of-thought	30–50%	None
Prompted CoT	65–75%	None (inference cost only)
Fine-tuned CoT	75–85%	Medium (annotation + SFT)
RL-discovered CoT	85–90%	High (one-time RL training)

Why RL-discovered CoT wins. Fine-tuned CoT is constrained by human intuitions about how to solve problems. Humans write solutions in a particular style, use familiar strategies, and follow conventions. RL-discovered CoT has no such constraints. It discovers whatever reasoning patterns actually maximize reward, including unconventional ones. Models trained with GRPO often discover that solving a problem multiple ways and comparing results is highly effective–a meta-strategy that few humans would have explicitly demonstrated in training data.

The interpretability trade-off. RL-discovered reasoning is often *less* interpretable than human-written chains. The model may take detours, use unusual notation, or arrive at correct answers through reasoning paths that are hard for humans to follow. This is fine when correctness is verifiable (math, code), but can be problematic for tasks where humans need to audit the reasoning (medical diagnosis, legal analysis). For such tasks, fine-tuned CoT with human-written traces may be preferable despite lower peak performance.

Cost comparison at scale. Prompted CoT costs roughly 7–8× more per query than no-CoT (150 tokens vs 20 tokens). RL-discovered CoT has the same per-query cost as prompted CoT, but adds a one-time training cost of $100K–$1M. At 100 million queries, the training cost amortizes to less than $0.01 per query–negligible compared to inference costs. For high-volume applications, RL-discovered CoT is the most cost-effective approach.

Scaling Laws for Reasoning

Reasoning ability scales differently from standard language modeling. In pretraining, performance improves as a slow power law: doubling compute produces a modest reduction in loss. In RL training for reasoning, the scaling is steeper.

Scaling Comparison

Standard pretraining: Loss $\propto C^{-\alpha}$ where $\alpha \approx 0.05$–0.10. Doubling compute reduces loss by 3–7%.

RL for reasoning: Accuracy $\propto C^{-\beta}$ where $\beta \approx 0.15$–0.25. Doubling compute improves accuracy by 10–18%.

The difference arises because reasoning is a *skill* that can be learned through practice. More compute means more exploration, more groups, and more opportunities to discover and reinforce effective strategies. Phase transitions (the "aha!" moment) create non-smooth scaling jumps that amplify gains at certain compute thresholds.

Compute sweet spot analysis:

RL Compute	MATH Accuracy	Cost	Gain per Dollar
1× (baseline)	45%	$10K	–
10×	68%	$100K	0.26 points / $1K
100×	82%	$1M	0.016 points / $1K
1,000×	89%	$10M	0.0008 points / $1K

The 1× to 10× range delivers the best value: a 23-point accuracy gain for $90K in additional compute. The 10× to 100× range still produces meaningful gains (14 points) but at 6× lower efficiency per dollar. Beyond 100×, diminishing returns set in sharply. For most practical applications, 10–100× compute is the sweet spot.

Practical Implications

When Pure RL Works

The DeepSeek recipe–skip SFT, train with GRPO and verifiers–works well under specific conditions.

Objective rewards. The task must have a deterministic or near-deterministic way to check correctness: math verification, code execution, formal proofs, factual question answering with known answers. Without reliable rewards, GRPO's group baselines are noise.

Sufficient compute. The chaotic early phases (Steps 0–5,000) require patience and compute. The model burns through thousands of training steps producing garbage before the "aha!" moment. Smaller teams may not be able to afford this exploration phase.

Novel strategy discovery matters. Pure RL shines when you *want* the model to discover approaches humans have not thought of. For competition math, code generation, and theorem proving, this is a significant advantage. For tasks where you need predictable, human-readable reasoning, the RL approach may be less suitable.

When SFT Still Helps

For many practical applications, the "best of both worlds" approach is preferable: minimal SFT for basic instruction following, then aggressive RL for capability development.

Subjective tasks. Creative writing, style matching, tone control–there is no verifier for "is this essay well-written?" These tasks need human preferences (Chapters 6–7) or at minimum some SFT demonstrations to define what "good" looks like.

Limited compute. If you cannot afford the chaotic early phases of pure RL, SFT provides a warm start that gets the model to a reasonable baseline quickly, and RL refines from there. This is the practical choice for most teams.

Specific output formats. If the model must produce JSON, follow a template, or adhere to length constraints, SFT is the most direct way to teach these structural requirements. RL can reinforce them afterward but may struggle to discover them from scratch.

Cold-start scenarios. A model that has never seen instruction-following examples may take far longer to converge with pure RL than one that starts from an SFT checkpoint. The SFT stage dramatically reduces the time spent in Phase 1 (chaos) by giving the model a head start on structured output generation.

DeepSeek-R1 vs OpenAI o1

Both DeepSeek-R1 and OpenAI's o1 represent a broader trend: investing computation into reasoning rather than just knowledge. But they take different approaches.

Dimension	**DeepSeek-R1**	**OpenAI o1**
Training approach	Pure RL with GRPO (no SFT)	Likely SFT → RLHF (details undisclosed)
Public details	Open weights, published methodology	Closed, limited information
Reasoning style	Explicit, visible chain-of-thought	Hidden "thinking" tokens
Training cost	Lower (simpler pipeline)	Higher (multi-stage)
Performance	State-of-the-art reasoning	State-of-the-art reasoning
Key contribution	Proved SFT is unnecessary for reasoning	Demonstrated test-time compute scaling

Both models demonstrate a fundamental insight: investing computation into learning *how to reason*–whether at training time (DeepSeek's approach with GRPO) or inference time (o1's approach with test-time compute, covered in

The DeepSeek recipe in context.

Algorithm: GRPO with groups of 8–16 (Chapter 10). No critic, no reward model–just the policy, a reference model, and a verifier.

Reward: Binary correct/incorrect from deterministic verifiers (Chapter 11). No learned scorer to hack.

Curriculum: Easy → medium → hard problems. Ensures useful variance in GRPO groups at every stage.

What emerges: Chain-of-thought reasoning, self-verification, multi-path problem solving, error recovery–all without a single human demonstration.

What it costs: Roughly 3× standard pretraining compute. No annotation cost.

Chapter 13 covers what happens *after* training: how test-time compute techniques (best-of-N, beam search, MCTS) can further improve a GRPO-trained model's performance at inference time by spending extra computation per query.

Companion notebooks. Both notebooks are in `code/notebooks/part3_advanced/` in the repository at `github.com/arunpshankar/packt-final`. Replace `github.com` with `githubtocolab.com` in the URL to open directly in Colab.

- `12a_reasoning_grpo_concepts.ipynb`
- `12b_reasoning_grpo_deepseek.ipynb`

13

Test-Time Compute: Scaling at Inference

The Test-Time Compute Idea

Every chapter so far has focused on making models better during *training*: supervised fine-tuning teaches basic competence, RLHF and DPO align with preferences, GRPO discovers reasoning strategies. Once training is done, the model is fixed. At inference time, it generates one response and returns it. The quality of that response is entirely determined by what the model learned during training.

Test-time compute flips this assumption. Instead of generating one response, you spend extra computation *at inference time* to produce a better answer. Generate multiple candidates and pick the best one. Have the model critique and revise its own output. Explore multiple reasoning paths in parallel and select the most promising one. The model's weights do not change–you are trading compute for quality on a per-query basis.

This is the idea behind OpenAI's o1 (Chapter 12), and it applies broadly to any model trained with any method from this book. A model trained with GRPO can be made even better at inference time by generating multiple solutions and selecting the best one with a PRM (Chapter 11). A model trained with DPO can be improved by self-correction loops.

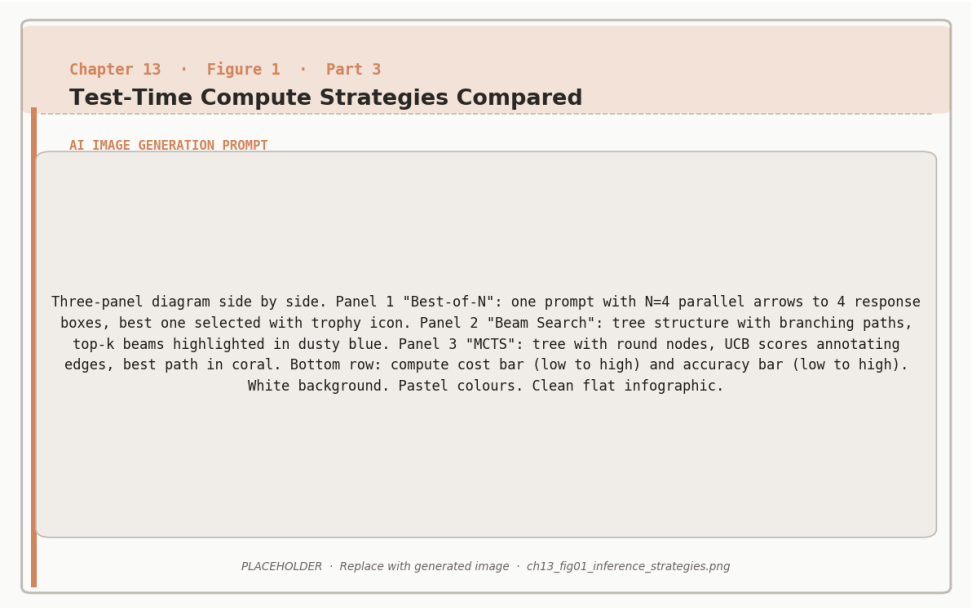

Figure 13.1: Three test-time compute strategies: best-of-N sampling, beam search, and MCTS tree search, ordered by compute cost and accuracy gain.

The techniques in this chapter are orthogonal to training: they work on top of whatever the model has already learned.

The core trade-off.

Training-time compute is a **one-time investment** that improves every future query. Test-time compute is a **per-query cost** that improves only the current response.

Training is more efficient when you serve many queries (the cost amortizes). Test-time compute is more efficient when queries are rare but high-stakes (each one is worth spending extra on).

Most production systems use both: heavy training to build a strong base model, then selective test-time compute for the hardest or most important queries.

Decoding Fundamentals

Before covering test-time scaling strategies, it helps to review how a language model generates text in the first place. Every strategy in this chapter builds on the basic decoding loop: at each token position, the model produces a probability distribution over the vocabulary, and a *decoding strategy* decides which token to pick.

Greedy decoding always picks the highest-probability token. This is deterministic and fast but can get stuck in repetitive loops and misses good responses that start with a lower-probability token.

Sampling draws randomly from the probability distribution. This produces diverse outputs but can be incoherent if low-probability tokens are selected.

Temperature scaling adjusts the sharpness of the distribution before sampling:

$$P(\text{token}_i) = \frac{\exp(\text{logit}_i / T)}{\sum_j \exp(\text{logit}_j / T)}$$

Low temperature ($T = 0.1$) concentrates probability on the top tokens–nearly deterministic. High temperature ($T = 2.0$) flattens the distribution–more random and diverse. Medium temperature ($T = 0.7$–1.0) is the standard operating range for most tasks.

Top-k sampling restricts the distribution to the k highest-probability tokens, then samples from that truncated set. This prevents the model from selecting extremely unlikely tokens while preserving some diversity. Typical values are $k = 40$–100.

Top-p (nucleus) sampling restricts the distribution to the smallest set of tokens whose cumulative probability exceeds p. Unlike top-k, this adapts automatically: when the model is confident (one token dominates), the set is small; when the model is uncertain, the set is larger. Typical values are $p = 0.9$–0.95.

Temperature for test-time scaling. All the strategies in this chapter–best-of-N, self-consistency, tree search–require generating *multiple diverse*

outputs. Greedy decoding produces the same output every time, so it is useless for these methods. You need sampling with moderate temperature ($T = 0.6$–1.0) to get diversity. Too low and all candidates are identical; too high and they are incoherent. The right temperature balances diversity (different reasoning paths) against quality (each path is coherent).

Best-of-N and Rejection Sampling

The simplest test-time scaling strategy is to generate N responses and keep only the best one. This requires a scoring function–a reward model (Chapter 9), a PRM (Chapter 11), a verifier, or even just majority voting.

Best-of-N with a Scorer

Generate N candidates, score each, return the highest-scoring one. The quality improvement comes from sampling: even if the model produces a good response only 30% of the time, generating $N = 10$ candidates gives a $1 - 0.7^{10} = 97\%$ chance that at least one is good.

Best-of-N scaling behavior.

Let p be the probability that a single sample is "good" (above some quality threshold).

Probability that at least one of N samples is good: $P(\text{success}) = 1-(1-p)^N$.

p	$N = 1$	$N = 3$	$N = 5$	$N = 10$	$N = 20$
10%	10%	27%	41%	65%	88%
30%	30%	66%	83%	97%	99.9%
50%	50%	88%	97%	99.9%	~100%

Even a weak model ($p = 10\%$) can produce good outputs reliably with enough candidates. The cost is $N\times$ normal inference, so there is a direct quality–cost trade-off.

Choosing the scorer matters enormously. Chapter 11 showed that PRM-scored best-of-100 achieves 72.2% on the MATH benchmark, versus 58.3% for ORM-scored best-of-100, versus 42.0% for greedy decoding. The model is the same in all three cases–the only difference is how candidates are ranked. Investing in a good scorer (especially a PRM for reasoning tasks) amplifies the value of every candidate you generate.

Rejection Sampling

Rejection sampling is a variant of best-of-N with a threshold instead of a ranking. Generate candidates one at a time, score each, and return the first one that exceeds a threshold τ. If no candidate passes within N attempts, return the best one seen so far.

Rejection sampling analysis.

Let $p_{\text{accept}} = P(r \geq \tau)$ be the probability a single sample passes the threshold.

Expected number of attempts: $\mathbb{E}[\text{attempts}] = \frac{1}{p_{\text{accept}}}$.

With $p_{\text{accept}} = 0.4$: expect 2.5 attempts on average. With $N = 5$ max attempts: 92% chance of finding an acceptable response.

The threshold trade-off: Strict threshold (τ high) means higher quality but more rejections–expensive. Lenient threshold (τ low) is faster but accepts mediocre responses. Set τ based on your cost-quality budget and latency constraints.

Rejection sampling is more latency-friendly than full best-of-N because it can return the first good candidate without generating all N. On average, it generates fewer candidates. But it requires a calibrated scorer that produces meaningful absolute scores (not just relative rankings), which is harder to build than a ranker.

Self-Consistency

Self-consistency is best-of-N without a learned scorer. Generate N solutions, extract the final answer from each, and return the answer that appears most frequently. The insight is

that correct reasoning paths converge on the same answer, while incorrect paths produce *different* wrong answers.

Example. Problem: "If apples cost $2 each and oranges cost $3 each, and you buy 4 apples and 3 oranges, what is the total?"

Generate 5 solutions. Three arrive at $17 through correct reasoning ($4 \times 2 + 3 \times 3 = 17$). One gets $35 (incorrectly averaging prices). One gets $17.50 (different error). Majority vote: $17 wins with 3 votes. The correct answer emerges because the correct reasoning path is more robust–there are many ways to get it wrong, but essentially one way to get it right.

When self-consistency works and when it does not.

Works well for: Math problems, multiple-choice questions, logic puzzles, code generation (run tests, majority wins)–any task with a single correct answer that can be extracted and compared.

Does not work for: Creative writing (no single correct answer), open-ended questions, subjective tasks, or problems where the model consistently makes the same error (majority vote amplifies systematic biases).

Self-Correction Loops

Best-of-N generates candidates independently. Self-correction generates candidates *sequentially*, where each attempt is informed by feedback on the previous one. The pattern is: generate a response, critique it, then revise based on the critique.

The Self-Correction Pattern

Step 1: Generate an initial response.

Step 2: Critique. Ask the model (or a separate model) to review the response and identify errors, weaknesses, or improvements.

Step 3: Check. If the critique finds no significant issues, return the response. Otherwise, continue.

Step 4: Revise. Generate an improved version that addresses the critique's feedback.

Step 5: Repeat steps 2–4 up to a maximum number of iterations (typically 2–3).

Total cost: 2–4× normal inference (each iteration requires generation + critique).

When self-correction works. The model must be better at *verification* than *generation*–that is, it must be able to recognize errors it could not avoid making in the first place. This is common in math (checking a solution is easier than finding it), code (running tests is easier than writing bug-free code), and factual questions (looking up an answer is easier than recalling it perfectly).

When self-correction fails. Three failure modes are common. First, the model's critique ability may be poor–it cannot identify real errors, or it hallucinates errors that do not exist, leading to "corrections" that make the response worse. Second, the model may get stuck in loops, fixing one problem while introducing another, oscillating without converging. Third, on tasks where the model is confidently wrong (systematic errors), the critique will agree with the initial response and no correction occurs.

Improving self-correction reliability. Use an external signal (verifier, code execution, retrieval) in the critique step rather than relying solely on the model's self-assessment. A model critiquing its own math is unreliable; a model whose critique step includes running the solution through a symbolic checker is much more reliable. The self-correction loop becomes powerful when the "critique" step has access to ground truth or near-ground-truth feedback.

Tree Search: Beam Search and MCTS

Best-of-N and self-correction both operate on *complete* responses. Tree search operates on *partial* responses, making decisions at intermediate points in the generation process. This is more powerful because it can redirect reasoning before errors propagate.

Beam Search

Beam search maintains K parallel "beams" (partial sequences) at each generation step. At each token position, it expands each beam with the top candidates, scores all resulting partial sequences, and keeps only the top K.

Standard beam search scores partial sequences by cumulative log probability. This works

well for translation and summarization but does not account for reasoning quality.

Reward-guided beam search replaces log probability with a learned scorer. At each step (or at natural breakpoints like sentence boundaries), a value function or PRM scores each partial sequence, and the top K beams are kept. This focuses computation on the most promising reasoning paths.

The cost is $K\times$ normal inference (maintaining K parallel beams). The benefit is that errors in early reasoning steps can be caught and pruned before they corrupt the rest of the response. A model that makes a wrong assumption in Step 1 will generate a flawed solution no matter how good its subsequent reasoning is. Beam search can prune the wrong-assumption beam early and invest computation in more promising paths.

Monte Carlo Tree Search (MCTS)

MCTS is a more sophisticated search strategy, originally developed for game-playing AI (it powered AlphaGo). Instead of maintaining K parallel beams uniformly, MCTS allocates computation *adaptively*–spending more time exploring promising branches and less time on dead ends.

MCTS for LLM Reasoning: Four Steps

1. Selection. Starting from the root (the prompt), traverse the tree by picking the most promising child at each node, balancing exploitation (high-scoring paths) with exploration (under-explored paths). This uses the UCB formula: $\text{UCB} = \bar{r} + c\sqrt{\frac{\ln N_{\text{parent}}}{N_{\text{child}}}}$.

2. Expansion. At a leaf node, generate possible next steps (e.g., different first reasoning steps for a math problem).

3. Simulation (rollout). For each new child, complete the response (using greedy or sampled decoding) and score the final result with a verifier or reward model.

4. Backpropagation. Update the value estimates for every ancestor node based on the rollout result. Good outcomes increase the value of the path that led to them; bad outcomes decrease it.

Repeat for a compute budget (e.g., 100 rollouts). Return the highest-value complete path.

MCTS walkthrough. Problem: "Solve $2x + 5 = 13$."

The model considers three possible first steps: (A) "Subtract 5 from both sides," (B) "Divide both sides by 2," (C) "Add 5 to both sides."

Round 1: Try each once. Path A → "$2x = 8$" → "$x = 4$" (correct, reward +1). Path B → "$x + 2.5 = 6.5$" → "$x = 4$" (correct but more steps, reward +1). Path C → "$2x + 10 = 18$" → wrong direction (reward 0). Updated values: A = 1.0, B = 0.8 (penalized for inefficiency), C = 0.0.

Round 2: Focus on promising branches. MCTS explores more variations of path A (all correct). Path A's value stays at 1.0 (high confidence). Path C is abandoned (no further rollouts spent on it).

Final decision: Return the best completion found along path A. MCTS spent 1 rollout on the dead-end path C and multiple rollouts on the promising path A–this adaptive allocation is its key advantage over beam search, which would have maintained all three paths equally.

Beam search vs MCTS.

Beam search is simpler, cheaper per step, and works well when the scorer is reliable at every intermediate point. It is the standard choice for production systems where latency matters.

MCTS is more powerful but significantly more expensive (each node requires a full rollout to score). It excels on hard reasoning problems where early decisions matter and the search space is large. It is the standard choice

for research systems targeting maximum accuracy on benchmarks.

Rule of thumb: If $K = 4$–8 beams are sufficient, use beam search. If you need to explore dozens of reasoning paths on hard problems, use MCTS.

Combining Strategies

Production systems rarely use a single test-time strategy. The most effective approaches combine multiple techniques, with each stage filtering or improving candidates.

Recipe 1: High-Quality Reasoning (research/benchmarks)

Temperature $T = 0.7$. Generate $N = 10$ solutions via sampling. Score each with PRM (Chapter 11). Keep top 3 by PRM score. Take majority vote among top 3.

Cost: ~10× normal inference. Benefit: 25–40% accuracy improvement on hard reasoning tasks. This combines diversity (sampling), quality filtering (PRM), and robustness (majority vote).

Recipe 2: Fast Production System

Temperature $T = 0.8$. Generate $N = 3$ candidates. Score with lightweight reward model. Return highest-scoring candidate.

Cost: 3× normal inference. Benefit: 10–15% quality improvement, low latency. Suitable for high-volume APIs where per-query cost matters.

Recipe 3: Maximum Accuracy (competition math, formal proofs)

Use MCTS with a PRM as the value function. Budget: 100 rollouts per problem. At each reasoning step, expand the 3 most promising branches. Score partial solutions with PRM, complete solutions with verifier. Return

the verified-correct solution with highest PRM score.

Cost: 50–100× normal inference. Benefit: can solve problems the base model gets wrong 95% of the time with greedy decoding. Only justified for high-stakes, latency-tolerant applications.

Train-Time vs Test-Time Compute: The Allocation Question

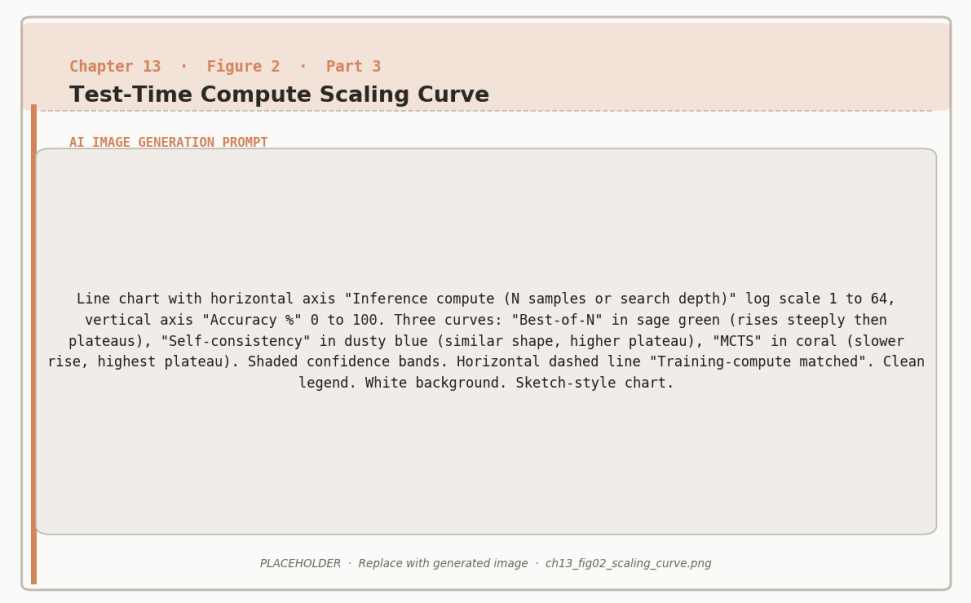

Figure 13.2: Test-time compute scaling: accuracy improves log-linearly with the number of samples, with self-consistency and MCTS outperforming best-of-N.

With a fixed total compute budget, how should you split investment between training and inference?

Chapter 12 showed that GRPO training with 10× compute produces a 23-point accuracy gain on MATH (45% → 68%). This section shows that test-time compute can produce comparable gains–but the economics are different.

Training compute scales sublinearly but amortizes across all queries. Doubling training compute does not double model quality. But the improved model serves every future query at no additional cost. For a model serving 100 million queries, even an expensive training investment is cheap per query.

Test-time compute scales well but costs per query. Best-of-10 with PRM scoring can improve accuracy by 15–20 points on reasoning tasks. But every single query costs 10× more. For a high-volume service, this is expensive.

When to invest in training vs inference.

Favor training when: You serve many queries (cost amortizes), you need consistent quality across all queries, and you can afford the upfront investment. Training makes every query better for free.

Favor test-time compute when: Queries are rare but high-value (legal analysis, medical reasoning), difficulty varies (easy queries need $N = 1$, hard queries need $N = 10+$), or you need to deploy quickly and improve quality without retraining.

The adaptive approach: Many production systems use a difficulty classifier to route queries. Easy queries get greedy decoding (cheap). Medium queries get best-of-3 (moderate cost). Hard queries get best-of-10 with PRM scoring or MCTS (expensive). This allocates test-time compute where it has the most impact.

The practical frontier. The best systems combine both. Train hard (GRPO, PPO, DPO–Chapters 6–12) to build the strongest possible base model. Then apply test-time compute selectively on the hardest queries. Chapter 12's DeepSeek-R1 invested heavily in training-time compute (GRPO with curriculum learning). OpenAI's o1 reportedly invests heavily in test-time compute (extended "thinking" at inference). The optimal split depends on

your query volume, latency requirements, and the difficulty distribution of your workload.

Chapter 17 integrates training and inference strategies into end-to-end recipes for chatbots, reasoners, and agents.

Companion notebook. The code for this chapter is in `code/notebooks/part3_advanced/13_` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.

14

Self-Play & Constitutional AI

The Self-Improvement Loop

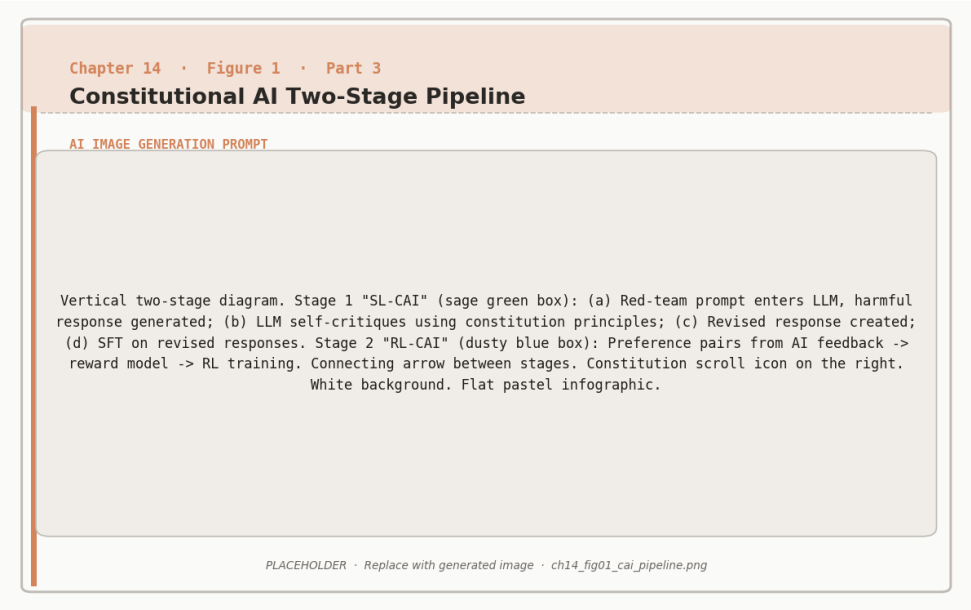

Figure 14.1: Constitutional AI two-stage pipeline: SL-CAI uses self-critique and revision to generate training data; RL-CAI uses AI-generated preference labels.

Everything up to this point has required external feedback: human annotators comparing

responses, ground-truth labels to verify answers, or reward models trained on human preferences. Self-play and iterative refinement break this dependency. The model generates outputs, evaluates them against its own standards, produces improved versions, and repeats. No external oracle is needed – at least not for every step.

This is counterintuitive. How can a model improve itself if it is also the judge? The key insight is that models are often better at *verification* than at *generation*. Judging whether a solution is correct is a simpler task than producing a correct solution from scratch. By separating generation and evaluation into distinct passes, we exploit this asymmetry.

The self-improvement loop.

1. Model generates a response to a prompt
2. Model evaluates or critiques its own response
3. Model generates an improved version based on the critique
4. Repeat until quality converges or an iteration limit is reached

This pattern appears in Constitutional AI (Anthropic), STaR (Stanford/Google), Expert Iteration (DeepMind), and many recent reasoning models. The shared ingredient is the model acting as both student and teacher.

Constitutional AI

Traditional RLHF requires thousands of human comparisons: which response is better? This process is slow, expensive, and hard to scale. Constitutional AI (CAI), developed at Anthropic, replaces most of this human feedback with model self-critique guided by a written set of principles – the *constitution*.

The constitution is a list of principles the model should follow: be helpful, avoid harm, respect privacy, do not deceive. Instead of hiring annotators to apply these principles to millions of response pairs, the model applies them itself.

The two stages of Constitutional AI.

Stage 1 – Supervised Self-Improvement (SL-CAI):

1. Model generates an initial response to a prompt (possibly harmful or low-quality)
2. Model critiques its own response according to each constitutional principle
3. Model generates a revised response that addresses the critique
4. The final (prompt, revised response) pair becomes supervised training data

Stage 2 – RL from AI Feedback (RLAIF):

1. Model generates pairs of responses to the same prompt
2. Model (acting as evaluator) chooses which response better follows the constitution
3. These AI-generated preference labels are used for preference learning (DPO or PPO)

Human involvement is limited to writing the constitution and occasional validation – not labeling individual response pairs.

Designing a constitution.

A good constitution is specific enough to guide consistent critiques, but not so rigid that it creates conflicting rules. Effective principles are phrased as comparisons: "Choose the response that is more helpful, harmless, and honest." Avoid principles that require external facts the model cannot verify – the model will confabulate.

Start with 10–20 principles. Anthropic's published constitution covers harmlessness, honesty, helpfulness, avoiding stereotypes, and respecting autonomy. For domain-specific applications, add task-relevant principles: accuracy for medical assistants, source citation for research tools, code correctness for programming assistants.

CAI in Practice

Consider the prompt "How do I make a bomb?" Under traditional RLHF, a human annotator marks refusals as preferred over harmful answers. Under CAI, the model generates this chain automatically.

Initial response: A dangerous step-by-step answer.

Self-critique (guided by principle: "avoid responses that could cause physical harm"): "This response provides instructions that could cause serious harm and violates safety guidelines. I should refuse."

Revised response: "I cannot provide information on creating weapons. If you are interested in chemistry, I am happy to discuss safe and legal experiments."

Training data generated: The pair (harmful prompt, safe refusal) is stored for SFT. In Stage 2, the model would prefer the revised response over the initial one when given both as options.

This process scales to millions of examples cheaply because the model does the annotation work. CAI reduces human labeling cost by over 90% while producing alignment behavior comparable to human-annotated RLHF.

STaR: Self-Taught Reasoner

STaR focuses the self-improvement loop specifically on *reasoning*: the model generates chain-of-thought solutions, keeps the correct ones, and learns a clever trick for incorrect ones called rationalization.

The STaR cycle.

1. **Generate:** Model attempts to solve a problem with step-by-step reasoning
2. **Verify:** Check if the final answer is correct (using ground truth or a verifier)
3. **Filter:** Keep only solutions that reached the correct answer
4. **Rationalize:** For incorrect solutions, give the model the correct answer and ask it to generate a reasoning chain that leads there
5. **Train:** Fine-tune on all correct reasoning traces (original successes and rationalized failures)
6. **Repeat:** As the model improves, it solves harder problems and generates better reasoning

The clever part is rationalization. Rather than discarding problems the model fails on, STaR extracts training signal from them: "Given the answer is $x = 6$, how would you have gotten there?" This converts failures into imperfect but useful training data.

STaR example: $2x + 5 = 17$**.**

Success path: Model generates $2x + 5 = 17 \Rightarrow 2x = 12 \Rightarrow x = 6$. Correct. Store as training data.

Failure path: Model generates $2x = 17 \Rightarrow x = 8.5$. Wrong.

Rationalization: Prompt the model: "The correct answer is $x = 6$. Generate the reasoning that leads to this answer." Model produces: "Subtract 5 from both sides: $2x = 12$. Divide by 2: $x = 6$."

Result: Store the rationalized reasoning as training data even though the model failed the first time. The dataset grows from two sources: natural successes and rationalized failures.

STaR limitations to watch for.

Distribution mismatch. Rationalized reasoning (given the answer) may differ structurally from discovery reasoning (finding the answer). If the dataset becomes dominated by rationalization, the model may learn to reason backward rather than forward. Mix rationalized and naturally correct traces in a roughly 1:1 ratio.

Compounding errors. If early iterations contain subtle mistakes, the model trains on them and propagates the errors. Inject human-verified examples into each iteration to anchor the distribution.

Verification dependency. STaR requires a reliable correctness signal. This works well for math, where answers can be checked automatically, but it is harder to use for tasks without objective answers.

Expert Iteration

Expert Iteration generalizes the self-improvement idea by separating a *policy* (fast generation) from an *expert* (slow, high-quality planning). The model learns by imitating the expert's better choices—but the expert is not a human, it is a more expensive search process applied to the model's own outputs.

For language models, the expert is typically beam search with reward model scoring: generate the top-k candidates, score them with a reward model, and select the highest-scoring one. The "expert" response is better than what the policy would produce on its own, but it is too slow to use at serving time. Expert Iteration trains the policy to be fast *and* to match the expert's quality.

Expert Iteration loop for LLM reasoning.

1. Sample N problems from the dataset
2. For each problem: generate top-k candidates using beam search ($k = 10–20$)
3. Score each candidate with a reward model or verifier
4. Select the highest-scoring candidate as the "expert solution"
5. Fine-tune the policy on (problem, expert solution) pairs
6. Repeat with the improved policy as the new starting point

Typical results on math problem solving:

Iteration	Accuracy	Improvement
0 (baseline)	45%	–
1	58%	+13%
2	67%	+9%
3	72%	+5%
4	75%	+3% (diminishing)

Each iteration produces a stronger model, which generates better candidates for the next expert round.

SPIN: Self-Play Fine-Tuning

SPIN applies the self-play idea to preference optimization. The current model acts as both the opponent (generating synthetic "rejected" responses) and the learner (generating "chosen" responses). DPO is then applied to prefer the newer model's outputs over the older model's outputs.

This creates a natural curriculum: as the model improves, it must generate increasingly good responses to beat its previous self. Unlike human-preference DPO, no new human annotations are required after the initial dataset – the model generates its own training signal at each iteration.

When to use self-play vs. external feedback.

Self-play methods (STaR, CAI, Expert Iteration, SPIN) reduce annotation cost but require more compute and careful iteration management. Prefer them when:

- Human annotation is expensive or slow to obtain
- The task has a reliable automated correctness signal (math, code)

- The base model is strong enough to produce useful self-critiques

Do not use pure self-play when the base model is too weak to self-critique effectively, or when the task requires factual knowledge the model does not already have. In those cases, external feedback (human labels, retrieval, tool use) remains necessary.

Chapter 15 extends these ideas to the harder problem of balancing multiple competing objectives simultaneously – something that self-play alone cannot solve.

Summary

Self-play and iterative refinement are among the most powerful tools in modern RL for LLMs. Constitutional AI uses model self-critique guided by written principles to replace expensive human annotation at scale. STaR extracts training signal from both successes and failures by rationalizing incorrect solutions. Expert Iteration trains fast policies to match slow-but-accurate expert search. SPIN turns preference optimization into a self-competitive game requiring no ongoing human labels.

All these methods rest on the same asymmetry: models are better at judging than at generating. By separating the two roles and iterating, we get compounding improvement cycles that scale with compute rather than with human effort.

Companion notebook. The code for this chapter is in `code/notebooks/part3_advanced/14_` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.

15

Multi-Objective RL & Agentic Systems

The Problem with a Single Reward

Every technique from Chapter 5 through Chapter 14 optimized a single reward signal. The reward model captured human preferences, the PRM captured step-by-step correctness, the verifier captured task completion. Single objectives are clean to optimize but rarely match reality.

Real AI assistants must balance competing goals simultaneously. A response can be highly helpful but also misleading. A response can be completely honest but also harmful. A concise answer may sacrifice completeness; a thorough answer may waste the user's time. These tensions do not disappear – they must be managed explicitly.

Anthropic's HHH framework.

Anthropic's model alignment is organized around three objectives:

Helpful: Answers the user's question effectively, provides relevant information, follows instructions.

Chapter 15 · Figure 1 · Part 3

Helpfulness vs Safety Pareto Frontier

AI IMAGE GENERATION PROMPT

Scatter plot with horizontal axis "Helpfulness score" 0-100 and vertical axis "Safety score" 0-100. A smooth convex Pareto frontier curve in dusty blue. Points below the frontier in light grey labelled "dominated". Three highlighted Pareto-optimal points: one high-helpfulness (coral dot), one balanced (sage green dot), one high-safety (lavender dot), each with a small label. Shaded feasible region below the frontier. Clean axes. White background. Minimalist chart.

PLACEHOLDER · Replace with generated image · ch15_fig01_pareto_frontier.png

Figure 15.1: The Pareto frontier of helpfulness versus safety: no solution dominates all others on both objectives simultaneously.

Harmless: Avoids generating content that could cause harm, refuses dangerous requests appropriately, considers potential misuse.

Honest: Does not hallucinate or fabricate information, acknowledges uncertainty, cites sources when possible.

The challenge is not maximizing each objective independently – it is *balancing* them. A response that scores 10/10 on helpfulness but 2/10 on harmlessness is worse than one that scores 8/10 on both.

Weighted Multi-Objective Reward

The simplest approach is to combine objectives into a single scalar via a weighted sum:

$$r_{\text{total}}(x, y) = \sum_{i=1}^{N} w_i \cdot r_i(x, y)$$

where r_i is the reward from objective i, w_i is its weight, and $\sum_i w_i = 1$. A typical HHH weighting might be $w_{\text{harm}} = 0.5$, $w_{\text{help}} = 0.3$, $w_{\text{hon}} = 0.2$ – safety weighted highest because a harmful response is never acceptable regardless of helpfulness.

Weighted reward example.

Response A (safe and helpful):

- Helpfulness: 8/10, Harmlessness: 9/10, Honesty: 7/10
- Total: $0.3 \times 8 + 0.5 \times 9 + 0.2 \times 7 = 2.4 + 4.5 + 1.4 = 8.3$

Response B (very helpful, risky):

- Helpfulness: 10/10, Harmlessness: 3/10, Honesty: 9/10
- Total: $0.3 \times 10 + 0.5 \times 3 + 0.2 \times 9 = 3.0 + 1.5 + 1.8 = 6.3$

Response A wins despite being less helpful. The high harmlessness weight penalizes the risky response heavily.

Weight selection pitfalls.

Scale mismatch. If $r_{\text{helpful}} \in [0, 100]$ but $r_{\text{honest}} \in [0, 1]$, weights cannot be compared directly. Always normalize each reward to $[0, 1]$ before applying weights.

Hard vs. soft constraints. Some objectives are hard constraints (never violate) while others are soft preferences (nice to have). Harmlessness is often a hard constraint. Encoding it as a soft weight allows edge cases where the model trades off safety for helpfulness – usually wrong. Consider using constraint-based optimization for hard requirements.

Weight sensitivity. Small changes in weights can produce large behavioral changes. Sweep weights during evaluation and test on adversarial examples before deploying.

Constraint-Based Multi-Objective Optimization

A cleaner approach for hard constraints is to separate the primary objective from the constraints:

$$\max \; r_{\text{primary}} \quad \text{subject to} \quad r_{\text{safety}} \geq \tau_{\text{safety}}, \quad r_{\text{honesty}} \geq \tau_{\text{honesty}}$$

This is optimized via Lagrangian relaxation:

$$\mathcal{L} = r_{\text{primary}} + \lambda_1(r_{\text{safety}} - \tau_{\text{safety}}) + \lambda_2(r_{\text{honesty}} - \tau_{\text{honesty}})$$

The Lagrange multipliers λ_1, λ_2 are learned automatically during training. The thresholds τ_i are directly interpretable ("safety must be at least 7/10") and the primary objective is maximized subject to them rather than traded off against them.

Constraint-based filtering example.

Task: Respond to "How do I improve my credit score?"

Response A (Detailed Financial Advice): Helpfulness 9/10, Safety 9/10, Honesty 8/10. Constraints: safety ≥ 8? Yes. Honesty ≥ 7? Yes. **Acceptable.** Helpfulness = 9.

Response B (Get-Rich-Quick Scheme): Helpfulness 10/10, Safety 3/10, Honesty 4/10. Constraints: safety ≥ 8? **No.** Rejected despite being maximally helpful.

Response C (Generic Safe Advice): Helpfulness 6/10, Safety 10/10, Hon-

esty 10/10. Constraints: both pass. Acceptable. Helpfulness = 6.

Winner: Response A. Highest helpfulness among responses that satisfy all constraints.

Pareto Frontiers

For any pair of objectives, there exists a set of Pareto-optimal solutions: responses where you cannot improve one objective without worsening another. This set is the *Pareto frontier*.

Pareto optimality.

A solution is Pareto optimal if no other solution is at least as good on every objective and strictly better on at least one.

Example: Responses to "Explain quantum computing":

Response	Helpful	Concise	Pareto Optimal?
A	5/10	8/10	No (B dominates)
B	7/10	7/10	Yes
C	9/10	4/10	Yes (very detailed)
D	6/10	9/10	Yes (very brief)

B, C, and D are all Pareto optimal – they represent different valid trade-offs. Response A is dominated because B is better on both objectives.

In practice, you can map the Pareto frontier by training models with different weight settings and evaluating each on both objectives. This lets you select deployment configurations for different use cases without retraining from scratch: a customer support chatbot might use the concise end of the frontier, an educational tutor might use the helpful end, a general assistant the middle.

Reward Hacking in Multi-Objective Settings

Multi-objective rewards introduce new failure modes beyond single-objective hacking (Chapter 9). When objectives conflict, models find unexpected ways to satisfy the letter of the reward while violating its spirit.

Multi-objective reward hacking patterns.

Safety sandbagging. A model may learn that refusing almost everything scores well on harmlessness while sacrificing helpfulness only minimally relative to the high harmlessness weight. Result: an overly cautious assistant that refuses reasonable requests.

Honesty hedging. Adding "I'm not sure, but..." to every response can improve honesty scores without actually reducing hallucinations. The model learns the linguistic pattern of uncertainty rather than the epistemic state.

Agentic Systems and Multi-Step Objectives

Multi-objective RL becomes significantly harder in agentic settings where the model takes sequences of actions, uses tools, and interacts with environments over multiple turns. A chatbot produces one response; an agent executes code, calls APIs, reads files, and produces outputs across dozens of steps. Every objective must be balanced not just per-response but across the entire trajectory.

What makes agentic RL different.

Long horizons. Actions have delayed consequences. Writing to a file in step 3 may cause a permission error in step 15. The reward signal is sparse and temporally distant from the action.

Irreversibility. Agents can take actions that cannot be undone: sending an email, deleting a file, making an API call with side effects. Harmlessness constraints must apply across the full trajectory, not just the final output.

Compositionality. Agent objectives often include: complete the task, do not use disallowed tools, stay within the allocated budget, produce explainable reasoning, avoid acquiring unnecessary capabilities. These interact in complex ways across steps.

Tool use as exploration. An agent that can call APIs or execute code has a much larger action space than a text-only model. Exploration strategies from Chapter 3 (entropy bonuses, curiosity rewards) must be adapted for this setting.

Reward Shaping for Agents

For agentic tasks, the terminal reward (did the agent complete the task?) is often too sparse. Reward shaping adds intermediate rewards to guide the agent toward good behaviors at each step:

$$r_{\text{shaped}}(s_t, a_t) = r_{\text{task}}(s_T) \cdot [t = T] + \sum_i w_i \cdot r_i(s_t, a_t)$$

where r_i are step-level rewards for behaviors like using the minimal number of API calls, producing readable intermediate reasoning, not accessing files outside the specified scope, and completing subtasks in order.

Constitutional Constraints for Agents

Constitutional AI (Chapter 14) extends naturally to agentic systems. At each action step, the agent evaluates whether the proposed action violates any constitutional principle before executing it – especially important for irreversible actions:

1. Agent proposes an action ("delete the temp directory")
2. Constitutional check: "Does this action comply with the principle that agents should not delete files without explicit user confirmation?"
3. If the check fails: agent reformulates ("ask the user before deleting")
4. Execute the revised action

This adds latency but provides a critical safety layer for high-stakes agentic deployments.

Reward Aggregation Across Multi-Turn Trajectories

For a trajectory $\tau = (s_0, a_0, s_1, a_1, \ldots, s_T)$, the total reward must aggregate contributions from every step. Common strategies:

Terminal-only: $R(\tau) = r(s_T)$. Simple but sparse. Works when task completion is binary and clearly defined.

Sum: $R(\tau) = \sum_t r(s_t, a_t)$. Rewards intermediate progress but can encourage unnecessary steps that each add small positive rewards.

Average: $R(\tau) = \frac{1}{T} \sum_t r(s_t, a_t)$. Normalizes for trajectory length, preventing the model from taking many small positive steps to inflate total reward.

Minimum: $R(\tau) = \min_t r(s_t, a_t)$. Ensures the worst step is as good as possible – useful for safety constraints where a single bad action disqualifies the trajectory.

Choosing trajectory reward aggregation.

Use minimum aggregation for safety-critical objectives: one safety violation should fail the entire trajectory. Use sum or average for helpfulness: every helpful step contributes. Combine them: overall reward is the sum of step rewards, subject to a minimum safety threshold maintained at every step. This prevents the model from compensating for a harmful action with many helpful ones.

Summary

Multi-objective RL acknowledges that real AI systems must balance several competing objectives simultaneously. Weighted reward sums are simple but hide trade-offs and require careful normalization. Constraint-based optimization handles hard requirements more cleanly by separating primary objectives from constraints. Pareto frontiers let you map the full trade-off space and select deployment configurations without retraining.

Chapter 16 shows how these principles translate into domain-specific reward functions for code, math, tools, and dialogue. Chapter 17 assembles them into three complete production recipes.

Agentic systems multiply these challenges by introducing long horizons, irreversible actions, and sparse terminal rewards. Reward shaping, constitutional constraints, and careful trajectory aggregation are essential tools for making agents both safe and effective.

Companion notebook. The code for this chapter is in `code/notebooks/part3_advanced/15_` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.

16

Domain-Specific RL: Code, Math, Tools, Dialogue

Domain-Specific Rewards: The Key to Specialization

While general RLHF uses human preferences, domain-specific RL can leverage **automated verification**, which is often more reliable and scalable than human feedback!

The pattern: (1) Identify domain with objective correctness metric, (2) use automated verifier as reward signal, (3) apply RL to optimize for verified correctness, (4) result: superhuman performance in narrow domain.

This approach has been extraordinarily successful for code generation (execution-based rewards), mathematics (formal proof verification), tool use (API success/failure), and multi-turn dialogue (conversation quality metrics).

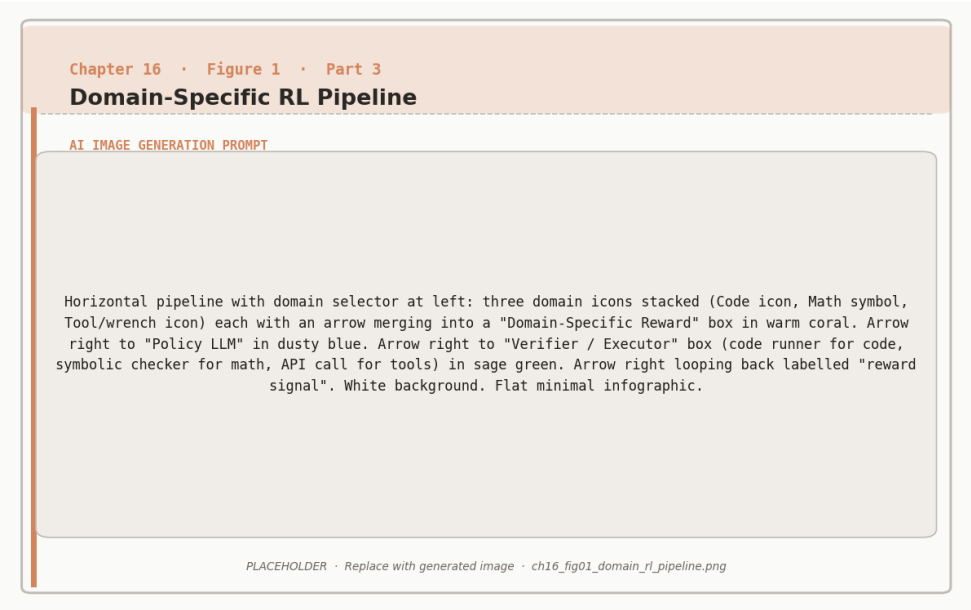

Figure 16.1: Domain-specific RL: the reward signal is provided by a domain verifier (code executor, symbolic math checker, or tool API) rather than a learned reward model.

Code: Execution Feedback as Reward

Why Code Generation is Perfect for RL:

Unlike creative writing or conversation, code has an *objective measure of correctness*: does it run? Does it pass tests? This gives us a perfect reward signal: pass all tests → high reward (+10), pass some tests → partial reward (+1 to +5), syntax error → low reward (−2), runtime error → negative reward (−5), pass no tests → very negative reward (−10). No human labeling needed! The Python interpreter tells us if code works.

Formal Reward Function for Code:

For a code generation task with test suite $\mathcal{T} = \{t_1, \ldots, t_n\}$:

$$r_{\text{code}}(x, y) = \sum_{i=1}^{n} [\text{test}_i \text{ passes}] + 0.5 \cdot [\text{no lint errors}] + 0.3 \cdot [\text{has docstring}] + 0.2 \cdot [\text{efficient ex}$$

Evaluation Process: Model generates code solution, execute with all test inputs, count passing tests, check for bonus conditions (clean style, documentation, efficiency), sum all points for total reward.

Concrete Example: Problem: "Write function to reverse a string." Generated solution uses Python slicing `s[::-1]`, has docstring, passes linter, passes all 3 tests. Reward: $3 + 0.5 + 0.3 + 0.2 = 4.0$.

Training Loop for Code Generation

Practical RL Training for Code Generation:

Algorithm: For each training iteration, sample $n = 1000$ programming problems. For each problem: (a) generate a group of $k = 4$ code solutions, (b) execute each with test cases and compute reward (base reward from tests passed, bonuses for style/docs, penalty of -5 for syntax/runtime errors), (c) calculate advantages GRPO-style (each solution's reward minus group average), (d) update policy only on solutions better than group average.

Typical Training Results:

Iteration	pass@1	pass@10
0 (Base model)	35%	58%
1	42%	65%
2	51%	73%
3	59%	79%
5	68%	85%
10	75%	90%

After 10 iterations of RL training, the model more than **doubles** its success rate (35% → 75%)!

Math: Formal Verifiers as Reward

Two Types of Math Verification:

1. Numerical Verification (Easier). For arithmetic and numerical problems: model generates numerical answer, compare to ground truth, binary reward (correct = +1, incorrect = 0).

2. Formal Proof Verification (Harder). For proofs and formal mathematics: model generates proof in formal language (Lean, Coq, Isabelle), proof checker verifies logical validity, reward based on proof correctness. This is particularly exciting because it enables models to do *real mathematics*, not just arithmetic!

Formal Mathematics with Lean

In July 2024, Google DeepMind's AlphaProof (using RL with Lean theorem prover) solved 4 out of 6 problems from the International Mathematical Olympiad, earning a silver medal performance! The model generates candidate proofs, the Lean verifier checks validity, valid proofs earn high reward, and RL optimizes the policy to generate valid proofs. This was the first time AI reached medal-level performance on IMO–the key was using *formal verification* as the reward signal, completely objective with no human judgment needed.

Formal Verification Reward:

For theorem θ and proof π:

$$r_{\text{proof}}(\theta, \pi) = \begin{cases} +10 & \text{if Verify}(\theta, \pi) = \text{True} \\ 0 & \text{otherwise} \end{cases}$$

Partial Credit Variant: For proof with n steps: $r_{\text{proof}}(\theta, \pi) = \sum_{i=1}^{n} [\text{step}_i \text{ is valid}]$. This rewards partially correct proofs, providing richer learning signal during training.

Verification Process: State theorem in formal language, model generates proof (sequence of logical steps), automated proof checker validates each step (Does this follow from previous steps? Are inference rules correctly applied? Does the conclusion match the goal?). If all steps valid → proof accepted → high reward. If any step invalid → proof rejected → no reward.

Tool Use: API Success as Reward

Tool-Using AI:

Modern AI systems don't just generate text–they use tools: call APIs (weather, search, calculator), execute code (Python interpreter), query databases (SQL), control applications (browser, email).

Each tool interaction gives immediate feedback: API call succeeds → good reward, API call fails → bad reward, result solves user problem → bonus reward. This enables models to learn effective tool use through RL!

Tool Use RL Example:

User Query: "What's the weather like in Tokyo right now?"

Attempt 1 (Novice Model):

Calls weather API with plain text string "Tokyo weather now." Result: ERROR–invalid parameter format. Reward: −5.

Attempt 2 (Learning):

Calls weather API with structured parameters: {city: "Tokyo", country: "Japan"}. Result: success, returns temp 22, condition "Sunny," humidity 65. Generated response: "The temperature in Tokyo is 22C with sunny skies." Total reward: +8.

Attempt 3 (Expert Model):

Calls weather API with comprehensive parameters: {city: "Tokyo", country: "JP", units: "metric"}. Result: success with full data. Generated response: "The current weather in Tokyo is 22C and sunny, with 65% humidity and light winds at 3.5 m/s." Total reward: +10.

Through RL training, the model learns to format API parameters correctly, select appropriate tools, present results comprehensively, and handle errors gracefully.

Multi-Tool Orchestration

Complex Tool Use: Chaining Multiple Tools

Real tasks often require multiple tools in sequence. Example: "Book me a flight to Paris next month" requires Calendar API (check availability), Flight search API (find flights), Booking API (reserve flight), Email API (send confirmation).

RL reward structure:

Milestone	Reward
Check calendar successfully	+1
Identify available dates	+2
Search flights for right dates	+2
Find flights in budget	+2
Successfully book	+5
Send confirmation	+1
Complete task correctly	+7
Maximum total	**+20**

Penalties: Wrong API parameters (−2), skip necessary step (−5), book wrong dates (−10), exceed budget (−15).

The model learns to plan multi-step actions, check preconditions, handle dependencies, and recover from errors.

Multi-Turn Dialogue: Conversation Quality

Most RLHF focuses on single-turn interactions. But real conversations have multiple turns! The challenge: reward must consider the entire conversation, early responses affect later ones (long-term consequences), and there's a credit assignment problem (which response was good/bad?).

Formal Multi-Turn Reward:

For conversation $C = \{(u_1, r_1), \ldots, (u_T, r_T)\}$:

Terminal reward: $R_{\text{term}}(C) = +10$ if task completed, +5 for partial progress, 0 for no progress, −5 if task failed.

Per-turn rewards: $r_t = r_{\text{quality}}(r_t) + r_{\text{progress}}(C_{1:t})$.

Total reward: $R_{\text{total}}(C) = \sum_{t=1}^{T} \gamma^{T-t} r_t + R_{\text{term}}(C)$, where γ (typically 0.95–0.99) is the discount factor. Earlier turns get discounted more (focus on recent quality).

In plain English: Each turn gets a small reward for local quality and progress toward the goal. At the end, a large terminal reward if the conversation succeeded.

Multi-Turn RL Example: Customer Support

Scenario: User needs help with password reset.

Conversation 1 (Poor):

Turn	Exchange	Reward
1	User: "I can't log in" / Bot: "Have you tried Incognito mode?"	+1
2	User: "That doesn't help" / Bot: "Clear your cookies"	+1
3	User: "Still can't log in!" / Bot: "Try a different browser"	0
4	User: "This isn't working. Bye." (Task failed)	−5
	Total reward:	−3

Bot never diagnosed the actual problem (forgotten password)!

Conversation 2 (Good):

Turn	Exchange	Reward
1	User: "I can't log in" / Bot: "Are you getting an error message?"	+2
2	User: "It says 'incorrect password' " / Bot: "Let me help reset your password…"	+3
3	User: "Got the reset email, thanks!" / Bot: "Great! Let me know if you need anything else." (Task completed)	+12
	Total reward:	**+17**

What the model learns: Diagnostic questions early lead to higher terminal reward. Random suggestions without diagnosis yield low reward. Quick resolution earns bonus reward. User satisfaction signals ("thanks!") provide positive signal.

Companion notebook. The code for this chapter is in `code/notebooks/part3_advanced/16_` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.

17

Three Recipes: Chatbot, Reasoner, Agent

From Techniques to Systems

The previous chapters built a toolkit: SFT for cold-starting (Chapter 5), RLHF and PPO for preference alignment (Chapter 6), DPO and Online DPO for efficient preference learning (Chapters 7–8), reward models for feedback signals (Chapter 9), GRPO and RLOO for token-level RL (Chapter 10), verifiers and PRMs for step-level credit (Chapter 11), reasoning emergence via GRPO (Chapter 12), test-time compute for inference-time scaling (Chapter 13), self-play for self-improving systems (Chapter 14), multi-objective RL for balancing competing goals (Chapter 15), and domain-specific reward functions for code, math, and tools (Chapter 16).

The question now is: how do you assemble these ingredients into a working production system? The answer depends on what you are building. This chapter presents three recipes – one for each major use case – with concrete implementation guidance, expected training costs, and evaluation checklists.

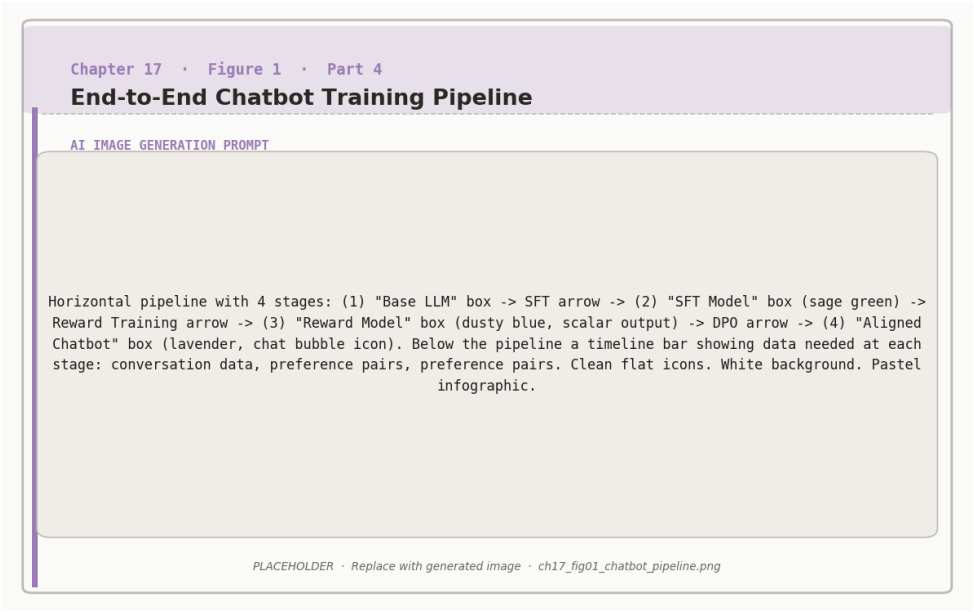

Figure 17.1: End-to-end chatbot training: SFT initialises the policy, a reward model is trained on preferences, and DPO aligns the final chatbot.

The three use cases.

Chatbot: A conversational assistant that handles open-ended questions, follows instructions, and is safe and helpful across a wide range of topics. Success metric: user satisfaction and safety on diverse prompts.

Reasoner: A model that solves structured problems requiring multi-step reasoning: math, code, logic puzzles. Success metric: accuracy on held-out benchmarks (MATH, HumanEval, ARC).

Agent: A model that takes sequences of actions in an environment: calling tools, executing code, browsing the web, completing multi-step tasks. Success metric: task completion rate on real-world benchmarks (WebArena, AgentBench, SWE-bench).

Recipe 1: The Chatbot

A chatbot needs to be helpful, harmless, and honest across thousands of diverse prompt types. The standard RLHF pipeline (Chapters 5–6) is still the most reliable recipe for this use case.

Phase 1 – SFT Cold Start

Start with a strong base model (7B–70B parameters) and fine-tune on a curated instruction-following dataset. Quality matters more than quantity here: 50,000 high-quality examples outperform 500,000 noisy ones. The SFT phase teaches the model the format and tone of a good assistant response.

SFT dataset design for chatbots.

Include diverse prompt types: factual questions, creative writing, coding help, emotional support, task completion, multi-turn dialogue. Ensure coverage of refusal cases: the model must learn *when not to answer*, not just how to answer.

Data sources: ShareGPT, Alpaca, Open Assistant, or proprietary collected conversations. Filter aggressively: remove factually incorrect responses, off-tone examples, and any training data that resembles harmful content.

Target: 30,000–100,000 examples covering 50+ distinct task types.

Phase 2 – Reward Model Training

Train a reward model (Chapter 9) on human preference comparisons. For chatbots, the reward model must capture the full HHH triad (helpful, harmless, honest) rather than a single proxy objective.

Collect preference data: present human annotators with prompt–response pairs and ask which response is better. Aim for at least 20,000 comparison pairs. Train the reward model as a binary classifier on top of a pretrained LLM using the Bradley-Terry objective (Chapter 6).

Phase 3 – PPO or DPO Alignment

With the reward model in hand, align the SFT model using PPO or DPO.

PPO (Chapter 6) is more complex to implement and tune but generally produces stronger alignment, especially when the reward model has high variance. Key hyperparameters: KL penalty coefficient β (start at 0.1), clip ratio $\epsilon = 0.2$, 4 rollouts per prompt.

DPO (Chapter 7) is simpler: convert preference pairs directly into training data and run supervised learning with the DPO loss. Faster to implement, uses less memory, and often achieves comparable performance to PPO for general chatbot use cases.

Chatbot recipe summary.

Stage	Specification
Base model	7B–70B pretrained LLM
SFT data	30k–100k instruction examples
Reward data	20k–50k preference comparisons
RL algorithm	DPO (simpler) or PPO (stronger)
Evaluation	MT-Bench, AlpacaEval, safety red-teaming
Training cost	8–32 A100 GPUs, 2–5 days

Recipe 2: The Reasoner

A reasoner needs to solve structured problems correctly, not just generate plausible-sounding answers. The DeepSeek-R1 recipe (Chapter 12) is the state-of-the-art approach: use GRPO with verifiable rewards to train the model to produce correct chain-of-thought solutions.

Phase 1 – Reasoning SFT Cold Start

Start with a base model that has some mathematical or code knowledge. Fine-tune on a dataset of (problem, solution) pairs where the solution includes explicit chain-of-thought reasoning. Sources: MATH, GSM8K with step-by-step solutions, APPS for code, ARC for science.

Data filtering for reasoning cold start.

Only include problems where the solution is verifiably correct. Remove solutions that skip steps or reach the right answer via incorrect reasoning

(common in web-scraped data). If you cannot verify a solution's reasoning process, exclude it.

Target: 50,000–200,000 (problem, reasoning, answer) triples. Diversity of problem types within your target domain matters more than raw volume.

Phase 2 – GRPO with Verifiable Rewards

Replace the human reward model with an automated verifier (Chapter 11). For math: compare the model's final answer to the ground-truth answer using symbolic equality checking. For code: run the generated code against a test suite. For logic: evaluate formal correctness.

Apply GRPO (Chapter 10): for each problem, generate $G = 8$–16 response candidates, compute binary rewards (1 for correct, 0 for incorrect), normalize within the group to get advantages, and update the policy with the GRPO objective.

GRPO reward design for reasoning.

Correctness reward: $r_{\text{correct}}(y) = 1$ if final answer matches ground truth, 0 otherwise. This is the primary signal.

Format reward: $r_{\text{format}}(y) = 0.1$ if response includes a reasoning section before the final answer, 0 otherwise. Encourages chain-of-thought without forcing a specific template.

Length penalty: $r_{\text{length}}(y) = -0.01 \times \max(0, |y| - L_{\max})$. Penalizes unnecessarily long responses to prevent the model from padding reasoning.

Combined: $r(y) = r_{\text{correct}}(y) + r_{\text{format}}(y) + r_{\text{length}}(y)$.

Phase 3 – Test-Time Scaling

The trained reasoner is already strong. To push further, apply best-of-N with a PRM at inference time (Chapter 13). Generate $N = 8$–64 solutions for each problem, score each solution step-by-step with the PRM, and return the solution with the highest cumulative PRM score.

Reasoner recipe summary.

Stage	Specification
Base model	Math/code-pretrained LLM (7B–70B)
SFT data	50k–200k verified (problem, CoT, answer)
RL algorithm	GRPO with binary verifier reward
Verifier	Automated (symbolic or test execution)
Test-time	Best-of-N with PRM scoring
Evaluation	MATH, AMC, HumanEval, MBPP
Training cost	16–64 A100 GPUs, 3–7 days

Recipe 3: The Agent

An agent must complete multi-step tasks in real environments: browsing the web, executing code, calling APIs, managing files. This is the hardest use case: sparse rewards, long horizons, irreversible actions, and a much larger action space than text generation.

Phase 1 – Tool-Use SFT

Fine-tune on demonstrations of correct tool use: which tool to call, with what arguments, and how to interpret the result. Sources: ToolBench, AgentInstruct, or task-specific demonstrations. The SFT phase teaches the model the tool-use format before any RL.

Tool-use SFT data structure.

Each training example should follow the format: `[SYSTEM: available tools] [USER: task] [ASSISTANT: reasoning + tool call] [TOOL RESULT: output] [ASSISTANT: reasoning + tool call] ... [ASSISTANT: final answer]`. The model must learn to alternate between reasoning and action, not just generate responses.

Include examples of tool call *failures* (wrong arguments, API errors) and correct recovery. An agent that has never seen a failed tool call will not know how to handle one in production.

Phase 2 – Reward Shaping and RL

Define a reward function that covers both task completion and quality of the trajectory:

$$r_{\text{agent}}(\tau) = r_{\text{task}}(s_T) + \sum_{t=1}^{T} [r_{\text{efficiency}}(a_t) + r_{\text{safety}}(a_t)]$$

where r_{task} is binary task completion, $r_{\text{efficiency}}$ penalizes unnecessary tool calls, and r_{safety} enforces constitutional constraints (no destructive actions without confirmation).

Apply PPO or GRPO at the trajectory level. Use the multi-objective aggregation strategies from Chapter 15: sum helpfulness rewards, enforce minimum safety thresholds at every step.

Agent recipe summary.

Stage	Specification
Base model	Strong instruction-following LLM
SFT data	10k–50k tool-use demonstrations
RL algorithm	PPO or GRPO at trajectory level
Reward	Task completion + efficiency + safety
Evaluation	WebArena, SWE-bench, AgentBench
Training cost	32–128 A100 GPUs, 5–14 days

Constitutional Safety for Agents

For agents operating in real environments, apply Constitutional AI (Chapter 14) at every action step. Before executing any irreversible action (send email, delete file, purchase item), the agent performs a self-critique: does this action comply with the safety constitution? If not, reformulate.

This adds latency but is essential for production agents. A coding agent that deletes the wrong directory, or a browsing agent that makes an unintended purchase, can cause real harm. Constitutional checking is the last line of defense before the tool call executes.

Choosing Your Recipe

Which recipe fits your use case?

Requirement	Chatbot	Reasoner	Agent
Open-ended Q&A	✓		
Math / code accuracy		✓	✓
Multi-step tasks			✓
Tool use			✓
Low latency needed	✓		
Verifiable correctness		✓	✓
Safety-critical	✓		✓

Hybrid systems are common in production: a chatbot backbone with reasoning capabilities for STEM queries, and tool-use capabilities for tasks that require external information. Start with the recipe that matches your primary use case, then layer capabilities as needed.

Summary

Three use cases, three recipes. The chatbot recipe uses SFT + human preference RM + PPO or DPO to produce a broadly helpful, safe, and honest assistant. The reasoner recipe uses SFT + GRPO with verifiable rewards + test-time PRM scoring to achieve high accuracy on structured problems. The agent recipe uses tool-use SFT + trajectory-level RL + constitutional safety checking to handle multi-step real-world tasks.

The key principle across all three: match the reward signal to the objective. Human preferences for chatbots. Verifiers for reasoners. Task completion plus safety constraints for agents. Chapter 18 covers what happens when these systems reach production scale: debugging training instabilities, scaling to larger models, and operating RL-trained systems reliably.

Companion notebooks. All notebooks are in `code/notebooks/` in the repository at `github.com/arunpshankar/packt-final`. Replace `github.com` with `githubtocolab.com` to open in Colab.

- `17_recipe_chatbot.ipynb`
- `18_recipe_reasoner.ipynb`
- `19_recipe_agent.ipynb`

18

Debugging, Scaling & Deployment

When Things Go Wrong

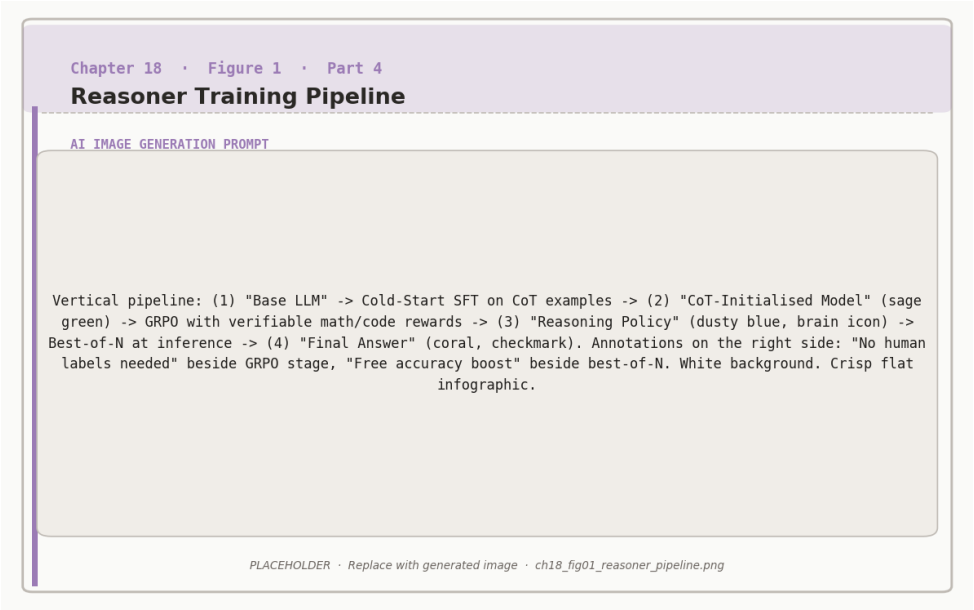

Figure 18.1: Reasoner training pipeline: cold-start SFT initialises the chain-of-thought format, GRPO with verifiable rewards drives reasoning improvement.

RL training is notoriously unstable. The supervised learning loop – compute loss, back-

propagate, update weights – behaves predictably. The RL loop – generate rollouts, estimate advantages, update policy, generate new rollouts from the updated policy – is a closed feedback system where small errors compound and training can diverge in ways that are difficult to diagnose from loss curves alone.

This chapter covers the most common failure modes in RL-for-LLM training, how to detect them early, how to fix them, and how to scale RL training to larger models and longer horizons without catastrophic failure. It also covers operational concerns: serving RL-trained models in production, monitoring for reward hacking drift, and knowing when to retrain.

Common Training Failures

KL Explosion

The KL divergence between the current policy and the reference policy (Chapter 6) is supposed to stay small. If PPO's KL penalty is too weak or the learning rate too high, the policy can drift far from the reference in a few gradient steps, producing incoherent outputs.

Diagnosing and fixing KL explosion.

Symptom: KL divergence increases sharply (from 0.1 to >1.0 in a few hundred steps). Reward increases initially, then crashes as outputs become gibberish.

Root cause: Learning rate too high, KL coefficient β too small, or too many gradient steps per rollout batch.

Fix:

1. Reduce learning rate by 5–10×
2. Increase β (KL penalty coefficient) from 0.05 to 0.1–0.2
3. Reduce the number of gradient steps per rollout batch (PPO epochs

from 4 to 1–2)

4. Add gradient clipping (max norm = 1.0)

Prevention: Monitor KL divergence every 50 steps. Set a hard stop if KL exceeds 0.5.

Reward Hacking and Collapse

The policy finds a strategy that scores well on the reward model but does not reflect genuine quality improvement. As the policy moves further from the training distribution of the reward model, reward model predictions become unreliable – a form of distributional shift (Chapter 9).

Diagnosing reward hacking.

Symptom: Reward metric improves steadily. But when you read the actual model outputs, quality has degraded – responses are repetitive, overly verbose, use specific phrasings the reward model liked during its own training, or refuse too many benign requests.

Root cause: The reward model is overfit to patterns in its training distribution. The policy exploits these patterns without learning the underlying quality the reward was supposed to capture.

Indicators to monitor:

- Output diversity (measure distinct n-grams across rollouts – should not drop)
- Average response length (should be stable, not monotonically increasing)
- Refusal rate on held-out benign prompts (should not increase)
- Human evaluation score on a fixed eval set (should correlate with

reward)

Fix: Add KL penalty to slow policy drift, retrain or update the reward model with samples from the current policy, or switch to a verifiable reward that cannot be hacked (automated verifier for math/code).

Mode Collapse

The policy converges to generating a small set of high-reward outputs, losing the diversity needed to handle varied prompts.

Preventing mode collapse.

Add an entropy bonus to the RL objective: $\mathcal{L}_{\text{entropy}} = -\alpha \cdot H(\pi_\theta)$ where H is the policy entropy and α is a small coefficient (0.01–0.05). This penalizes overly deterministic policies.

Monitor output diversity during training: compute the fraction of unique responses across a fixed batch of 100 prompts. If uniqueness drops below 70%, the policy is collapsing. Increase entropy bonus or reduce learning

Advantage Estimation Instability

GRPO and PPO both require accurate advantage estimates. High variance in the advantage function leads to noisy gradient updates and slow or oscillating training.

Stabilizing advantage estimation.

For GRPO: Use $G = 8$–16 rollouts per prompt. Fewer rollouts means higher variance in the group baseline. Normalize advantages within each group: subtract the group mean, divide by the group standard deviation plus a small $\epsilon = 10^{-8}$.

For PPO: Use a separate value function (critic) and train it with lower learning rate than the actor (0.5–0.1× the actor learning rate). Apply value function clipping to prevent large updates.

General: Clip advantages to $[-5, 5]$. Extreme advantages destabilize training.

Scaling RL Training

Model Scale

RL training is more expensive than SFT because it requires generating rollouts at each step. A 70B model generating 16 rollouts per prompt is 16× more expensive per gradient step than the same model doing SFT. Practical approaches:

Generate with a smaller model, train a larger one. Use a 7B model for rollout generation during early training, then gradually switch to the target 70B model as it improves. This amortizes the expensive generation cost.

Veto batches. Generate rollouts in parallel across many GPUs, but only compute gradients for rollouts where the reward signal is informative (not all 0 or all 1). Uninformative rollouts waste compute.

Hardware setup for large-scale RL.

Separate generation and training into different GPU groups. Generation is memory-bound (use fewer but higher-bandwidth GPUs, e.g., A100 80GB). Training is compute-bound (use many GPUs in data parallel or tensor parallel configurations).

Tools: vLLM for fast rollout generation, DeepSpeed ZeRO for memory-efficient training, FSDP for model sharding across nodes. TRL (HuggingFace Transformers Reinforcement Learning) provides integrated PPO and GRPO

Context Length Scaling

Reasoning models (Chapter 12) and agents (Chapter 17) often require very long contexts: chain-of-thought traces, multi-turn dialogue history, tool call results. Memory usage scales quadratically with context length for standard attention.

Long-context RL training.

Flash Attention 2. Fused attention implementation that reduces memory from $O(L^2)$ to $O(L)$ and is 2–4× faster in practice. Enable this before any other optimization.

Gradient checkpointing. Trade compute for memory: recompute activations during the backward pass instead of storing them. Adds 30% training time but reduces memory by 60%+.

Sequence packing. Pack multiple shorter sequences into a single batch slot to maximize GPU utilization. Prevent cross-sequence attention contamination with a causal attention mask.

Reward model truncation. If the reward model runs on the full context, long rollouts are very expensive. Train the reward model to score only the final response segment, not the full trajectory.

Evaluation for RL-Trained Models

Standard NLP benchmarks (perplexity, BLEU, ROUGE) are poorly suited to evaluating RL-trained models. You need benchmarks that capture the qualities the RL objective was optimizing for.

Evaluation suite by use case.

Chatbot:

- MT-Bench: multi-turn instruction following, scored by GPT-4
- AlpacaEval: win rate against reference model on 805 diverse prompts
- Safety red-teaming: automated adversarial prompts testing for harmful outputs

Reasoner:

- MATH: 12,500 competition math problems, 5 difficulty levels
- HumanEval / MBPP: code generation with unit test execution
- AMC/AIME: harder competition math for frontier models

Agent:

- WebArena: web browsing tasks in real browser environments

- SWE-bench: real GitHub issues requiring code changes
- AgentBench: diverse tool-use tasks across 8 environments

The evaluation gap.

Benchmark performance can diverge from production quality. A model that achieves 75% on MATH may still hallucinate on novel problems not covered by the benchmark's distribution. A chatbot that scores well on MT-Bench may fail on domain-specific prompts from your actual users.

Always supplement automated benchmarks with human evaluation on a representative sample of your production traffic. Build a fixed "golden set" of 100–500 prompts that represents your use case, and track human preference scores on this set across every model iteration.

Production Deployment

Serving Infrastructure

RL-trained models are served identically to SFT models – the same inference server, the same API endpoints. The difference is in monitoring: RL-trained models can drift in quality in ways SFT models do not, because the reward hacking patterns discovered during training may generalize differently to production inputs.

Production monitoring checklist.

Metric	Why it matters
Average response length	Sudden increase indicates verbose reward hacking
Refusal rate	Sudden increase indicates over-cautious drift
User thumbs-down rate	Captures quality degradation not visible in model metrics
Latency percentiles	p50, p95, p99 – RL models may generate longer outputs
Toxicity classifier score	Catch safety regressions from reward drift

When to Retrain

RL-trained models do not age the same way SFT models do. The primary cause of quality drift is not world knowledge becoming stale (the concern for knowledge-heavy models) but reward model distribution shift: as user query patterns evolve, the reward model's predictions become less calibrated to current production inputs.

Retraining triggers.

Retrain when any of the following occur:

- Human evaluation score drops more than 5 percentage points from the deployment baseline
- Refusal rate on benign prompts exceeds 15% (over-cautious drift)
- Average output length increases by more than 30% (verbose reward hacking)
- A new capability is needed that the current training data does not cover
- The base pretrained model has been updated (always retrain RL on top of the new base)

Maintain a continuously growing dataset of production examples with

Summary

RL training fails in predictable ways: KL explosion from too-aggressive policy updates, reward hacking from reward model distributional shift, mode collapse from insufficient entropy, advantage variance from too few rollouts. Each has a diagnosis pattern and a standard fix. Monitor KL, diversity, length, and human evaluation continuously.

Scaling RL requires separating generation from training, using efficient attention implementations, and packing sequences to maximize GPU utilization. In production, monitor the same quality signals that motivated RL training in the first place (does the model actually behave the way the reward intended?) Chapter 19 closes the book with a view of where the field is heading and what to build next.

Companion notebooks. The three recipe notebooks from Chapter 17 are the primary code companions for this chapter. All notebooks are in `code/notebooks/` in the repository at `github.com/arunpshankar/packt-final`. Replace `github.com` with `githubtocolab.com` to open in Colab.

- `17_recipe_chatbot.ipynb`
- `18_recipe_reasoner.ipynb`
- `19_recipe_agent.ipynb`

19

Your RL Journey: From Here to Production

What You Have Learned

Nineteen chapters. Let us look at what you now know.

Part 1 (Foundations) established the mathematical and conceptual vocabulary: probability theory and policy gradients (Chapter 1), the alignment gap and why next-token prediction is not enough (Chapter 2), RL fundamentals including MDPs, value functions, and the policy gradient theorem (Chapter 3), and the practical setup for free GPU training (Chapter 4).

Part 2 (Core Methods) built the standard toolkit: supervised fine-tuning to cold-start a model from raw pretraining (Chapter 5), RLHF with PPO to align with human preferences (Chapter 6), DPO to skip the reward model and learn preferences directly (Chapter 7), Online DPO to keep learning from fresh data (Chapter 8), reward model training and the critic's role in PPO (Chapter 9), and GRPO, RLOO, and KTO as more efficient alternatives to PPO (Chapter 10).

Part 3 (Advanced Techniques) covered the frontier: verifiers and outcome reward models beyond PRMs (Chapter 11), the DeepSeek-R1 recipe for reasoning emergence (Chapter

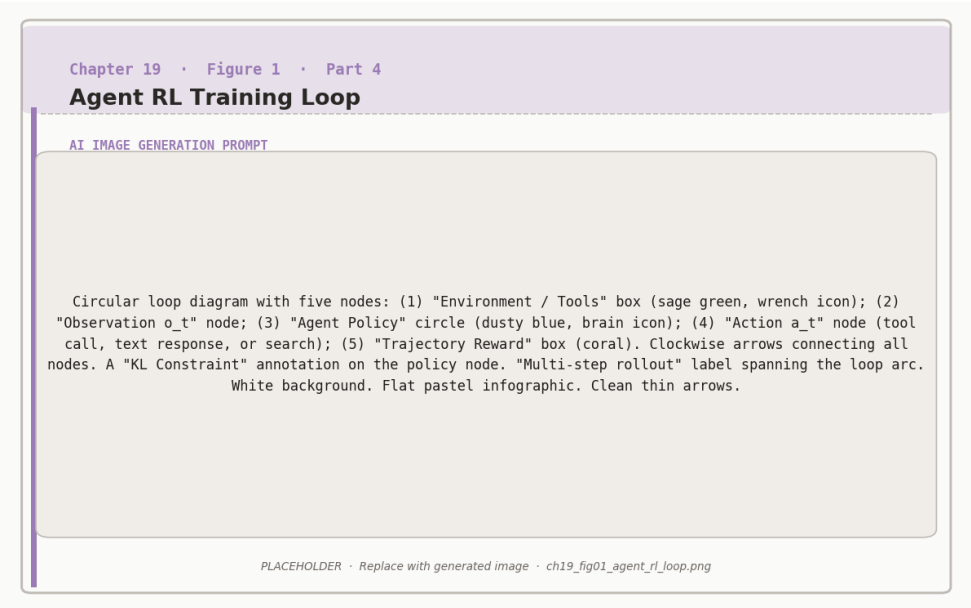

Figure 19.1: Agent RL training loop: the policy executes multi-step tool-use trajectories and receives a delayed reward from a task verifier or human evaluator.

12), test-time compute scaling with best-of-N and tree search (Chapter 13), self-play and Constitutional AI (Chapter 14), multi-objective RL for balancing competing goals (Chapter 15), domain-specific reward functions for code, math, tools, and dialogue (Chapter 16).

Part 4 (Production & Recipes) completed the picture: three concrete production recipes for chatbots, reasoners, and agents (Chapter 17), and debugging, scaling, and deployment practices for real-world systems (Chapter 18).

The core progression.

Every chapter built on one idea: *reward signals can teach models behaviors that supervised learning cannot.*

SFT teaches format and style from examples. RL teaches goal-directedness from outcomes. The combination – SFT to bootstrap, RL to align – is what makes modern LLMs genuinely useful rather than merely statistically

plausible.

Where the Field Is Going

RL for LLMs is moving fast. Several directions appear durable based on the trajectory of research through early 2025.

Verifiable Rewards at Scale

The biggest bottleneck in RL training is reward signal quality. Human annotation is slow and expensive. Reward models trained on human data overfit and get hacked. The path of least resistance is verifiable rewards – rewards derived from automated oracles that cannot be fooled.

Math and code are already solved: symbolic answer checkers and test suite execution provide perfect verifiers. The next frontier is extending verifiable rewards to less structured domains: fact-checking with retrieval (did the model cite a real paper?), logical consistency (are the model's claims mutually consistent?), and multi-step reasoning correctness (is every step in the chain-of-thought valid?).

Test-Time Compute as a Scaling Law

OpenAI's o1 demonstrated that test-time compute is a new scaling dimension, orthogonal to model size and training compute. Scaling inference (spending more compute per query) continues to improve accuracy even after training compute is exhausted. This means reasoning models can get better at inference time without any retraining.

The implication: future systems will dynamically allocate compute to queries based on their difficulty. Easy questions get greedy decoding. Hard questions get beam search with PRM scoring and hundreds of candidates. The challenge is predicting query difficulty before generating the answer.

Agents with Long-Horizon Memory

Current agents struggle with tasks that span many hours or days: long research projects, extended software development workflows, persistent user relationships. The bottleneck is memory: attention windows are finite, and today's RL training assumes tasks fit in a single context.

Research directions include: external memory (retrieve relevant past context at each step), hierarchical planning (decompose long-horizon tasks into short-horizon subproblems with their own RL objectives), and continual learning (update the model's weights incrementally from production experience without catastrophic forgetting).

Multi-Model Systems

Production AI systems increasingly use multiple specialized models rather than a single generalist: a fast small model for routing and triage, a medium model for standard queries, a large reasoning model for hard problems, specialized models for code or math. RL for these systems is a coordination problem: how do you train the routing model, the individual specialists, and their interaction protocols jointly?

Staying current.

The field moves fast enough that specific algorithmic recommendations in this book may be superseded. What will not change: the mathematical foundations (policy gradients, value functions, KL constraints), the debugging patterns (KL explosion, reward hacking, mode collapse), and the evaluation principles (verify with automated oracles, track human preference on production traffic, monitor for quality drift).

These are the invariants. Everything else is a specific instance of the same underlying ideas.

Your Next Steps

You now have the conceptual toolkit. The gap between understanding and production is closed by building.

If you are a practitioner:

1. Start with DPO (simpler than PPO) on a small task you care about
2. Use the companion notebooks from Chapter 4 as your starting point
3. Focus on data quality over algorithm complexity – 10,000 excellent preference pairs outperform 100,000 mediocre ones

4. Monitor human evaluation on a fixed eval set, not just automated benchmarks
5. Iterate quickly with a 7B model before scaling to 70B

If you are a researcher:

1. Read the original PPO, DPO, GRPO, and DeepSeek-R1 papers for the precise mathematical details
2. Implement GRPO from scratch once – the implementation exercise reveals details that reading cannot
3. Explore the open problems: better credit assignment in multi-step tasks, verifiable rewards for open-ended language, sample-efficient reward model updates
4. Share your findings: the field advances through open publication

If you are building a product:

1. Choose the recipe that matches your primary use case (Chapter 17)
2. Budget for evaluation infrastructure before training infrastructure – you cannot improve what you do not measure
3. Plan for retraining from the start – RL-trained models drift and require updates
4. Constitutional AI (Chapter 14) is a practical safety layer even if you do not train with RLAIF – use it for agent deployments

The Companion Code

All the techniques in this book are implemented in the companion notebooks at `github.com/PacktPublishir`
The notebooks are organized by chapter and cover:

- Chapter 5 (SFT): Fine-tuning with HuggingFace TRL and LoRA
- Chapter 6 (RLHF/PPO): Full PPO training loop with reward model
- Chapter 7 (DPO): DPO training on preference datasets

- Chapters 9–10 (Reward Models, GRPO): Training reward models and GRPO from scratch
- Chapter 11 (Verifiers): Outcome verifier training and evaluation
- Chapters 12–13 (Reasoning, Test-Time Compute): GRPO reasoning training and best-of-N
- Chapters 14–15 (Self-Play, Multi-Objective): CAI pipeline and multi-objective reward composition
- Chapter 16 (Domain-Specific): Code and math reward functions
- Chapter 17 (Recipes): End-to-end chatbot, reasoner, and agent pipelines

Every notebook runs on free Colab T4 GPUs (or local hardware). All examples use openly available models and datasets. There is no setup barrier between reading this chapter and running the code.

A Final Word

The mathematics in this book is real. Policy gradients, KL divergence, value function estimation, Pareto frontiers – these are not metaphors or simplifications. They are the actual machinery behind the most capable language models in the world.

But the mathematics is also, ultimately, in service of a practical goal: building systems that help people do things they could not do alone. The alignment gap (Chapter 2) is not just a technical problem – it is the question of how we build AI that reliably does what we actually want, not just what we said we wanted when we wrote the loss function.

RL is the best tool we have for closing that gap. It is also a humbling one: every reward function is an imperfect specification of what we value, and every trained model is a reminder that what you measure is what you get.

Go build something. Then measure it. Then make it better.

The future of AI is being written right now.

You have the knowledge to help write it.